

lab1

▼ 前序解释部分

- 操作系统分为用户态和内核态，在这里，完成xv6的lab，也就是在test测试样例下，观察用户态调用的某些封装了内核态函数的指令，是否可以由合适的内核态函数给出符合要求的输出。
- test测试样例本质上是用户态的函数调用，即使是用户态的函数，实际上只是对内核态一些函数的包装而已，所以真正要实现的部分，是xv6-k210中内核态的同义函数。

接下来用getcwd为例，展示一下从调用测试样例到执行到需要我們进行修改部分的流程。

- 例： `getcwd.c` 对应 `getcwd_test.py` 用来打分。调用时，用户态运行 `getcwd.c`

```
#include "stdio.h"
#include "stdlib.h"
#include "unistd.h"
#include "string.h"

/*
 * 测试通过时输出:
 * "getcwd OK."
 * 测试失败时输出:
 * "getcwd ERROR."
 */
void test_getcwd(void){
    TEST_START(__func__);
    char *cwd = NULL;
    char buf[128] = {0};
    cwd = getcwd(buf, 128);
    if(cwd != NULL) printf("getcwd: %s successfully!\n", buf);
    else printf("getcwd ERROR.\n");
    TEST_END(__func__);
}

int main(void){
```

```
test_getcwd();
return 0;
}
```

其中的 `getcwd()` 为 `syscall.c` 中声明的函数，这里则是用户态下声明的函数，可以看到，这里的参数为 `0: char *buf 1: size_t size` 即表明需要两个参数。但这个函数仅起到一个封装的作用，真正底层运行的是 `syscall()`。

```
char *getcwd(char *buf, size_t size){
    return syscall(SYS_getcwd, buf, size);
}
```

`syscall()` 其实是一个宏，这个宏在 `syscall.h` 中被定义，这些宏的作用包括将用户态的参数传入寄存器 `a0-a5`，并执行 `ecall` 指令*。

而这里的第一个参数 `SYS_getcwd` 实际上是一个宏，也即用户态下声明好的，倘若希望调用 `getcwd()` 函数，则需要启用内核态进行操作，而执行这个操作的编码，就是 `SYS_getcwd` 这个宏声明好的id。

```
#define SYS_fremovexattr 16
#define SYS_getcwd 17
#define SYS_lookup_dcookie 18
```

`ecall` 是由 `trampoline.S`（汇编码）的 `uservec` 来直接执行，这会进行用户态的寄存器保存（保护现场）将相关内容存进了 `trapframe` 中，这时，我们发现文件夹已经切换到了xv6-k210下的kernel中了，这说明此时一旦结束了 `ecall`，就彻底陷入了内核态。

```
trampoline:
.align 4
.globl uservec
uservec:
    #
    # trap.c sets stvec to point here, so
    # traps from user space start here,
    # in supervisor mode, but with a
    # user page table.
    #
    # sscratch points to where the process's p
```

```

->trapframe is
    # mapped into user space, at TRAPFRAME.
    #

    # swap a0 and sscratch
    # so that a0 is TRAPFRAME
    csrrw a0, sscratch, a0

    # save the user registers in TRAPFRAME
    sd ra, 40(a0)
    sd sp, 48(a0)
    ...
    sd t6, 280(a0)

    # save the user a0 in p->trapframe->a0
    csrr t0, sscratch
    sd t0, 112(a0)

    # restore kernel stack pointer from p->trapframe->kernel_sp
    ld sp, 8(a0)

    # make tp hold the current hartid, from p->trapframe->kernel_hartid
    ld tp, 32(a0)

    # load the address of usertrap(), p->trapframe->kernel_trap
    ld t0, 16(a0)

    # restore kernel page table from p->trapframe->kernel_satp
    ld t1, 0(a0)
    csrw satp, t1

    # sfence.vma zero, zero
    sfence.vma

```

```

        # a0 is no longer valid, since the kernel
page
        # 也就是说，这里的a0寄存器只能用于存储内核页的指
针，而不可再用
        # table does not specially map p->tf.

        # jump to usertrap(), which does not retur
n
        # 跳转至usertrap()，可以发现，这里汇编码中并不设
计ret
        # 也就是自此以后进入了内核态的C语言层了
        jr t0

```

`ecall` 的执行结束，会从 `uservec` 直接跳转至C代码层的 `usertrap()` 函数进行后续的执行。

`usertrap()` 定义在 `trap.c` 中，会检查这次跳转至内核态的申请，是否来自于用户态，否则会导致panic，接下来进行用户上下文如进程指针的保存。下一步，根据异常类型（`trap==8` 即为用户态调用了内核态来获取某些信息）分发，首先修改 `epc`，使其返回后执行 `ecall` 的下一条指令（4字节后，因为是RISCV架构）；其次，允许中断（`intr_on()`），正式调用内核态的 `syscall()`。

```

void
usertrap(void)
{
    // printf("run in usertrap\n");
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    // send interrupts and exceptions to kerneltrap
(),
    // since we're now in the kernel.
    w_stvec((uint64)kernelvec);

    struct proc *p = myproc();

```



```

// save user program counter.
p->trapframe->epc = r_sepc();

if(r_scause() == 8){
    // system call
    if(p->killed)
        exit(-1);
    // sepc points to the ecall instruction,
    // but we want to return to the next instruction.
    p->trapframe->epc += 4;
    // an interrupt will change sstatus & c registers,
    // so don't enable until done with those registers.
    intr_on();
    syscall();
}

...

usertrapret();
}

```

- 接下来的部分将在内核态进行，上面的“用户态→内核态”切换过程大致粗略地结束了。

`syscall()` 会进行当前进程信息的提取，并检查以及分发任务，实际上，他像是从进程中抽取要执行的内核态函数id，并检查id是否可用，转到执行对应内核态函数的分拣中转站。既然涉及到一些结构体，就应该解释一下 `xv6-k210/kernel/syscall.c` 整个脚本*的一些关键定义部分了。

▼ `xv6-k210/kernel/syscall.c`

```

#include "include/sysnum.h"
#include "include/types.h"
#include "include/param.h"
#include "include/memlayout.h"
#include "include/riscv.h"

```

```

#include "include/spinlock.h"
#include "include/proc.h"
#include "include/syscall.h"
#include "include/sysinfo.h"
#include "include/kalloc.h"
#include "include/vm.h"
#include "include/string.h"
#include "include/printf.h"
#include "include/sbi.h"

// Fetch the uint64 at addr from the current process.
int fetchaddr(uint64 addr, uint64 *ip)
{
    struct proc *p = myproc();
    if (addr >= p->sz || addr + sizeof(uint64) > p->sz)
        return -1;
    // if(copyin(p->pagetable, (char *)ip, addr, sizeof(*ip)) != 0)
    if (copyin2((char *)ip, addr, sizeof(*ip)) != 0)
        return -1;
    return 0;
}

// Fetch the nul-terminated string at addr from the current process.
// Returns length of string, not including nul, or -1 for error.
int fetchstr(uint64 addr, char *buf, int max)
{
    // struct proc *p = myproc();
    // int err = copyinstr(p->pagetable, buf, addr, max);
    int err = copyinstr2(buf, addr, max);
    if (err < 0)
        return err;
}

```

```

    return strlen(buf);
}

static uint64
argraw(int n)
{
    struct proc *p = myproc();
    switch (n)
    {
        case 0:
            return p->trapframe->a0;
        case 1:
            return p->trapframe->a1;
        case 2:
            return p->trapframe->a2;
        case 3:
            return p->trapframe->a3;
        case 4:
            return p->trapframe->a4;
        case 5:
            return p->trapframe->a5;
    }
    panic("argraw");
    return -1;
}

// Fetch the nth 32-bit system call argument.
int argint(int n, int *ip)
{
    *ip = argraw(n);
    return 0;
}

// Retrieve an argument as a pointer.
// Doesn't check for legality, since
// copyin/copyout will do that.
int argaddr(int n, uint64 *ip)
{

```

```

    *ip = argraw(n);
    return 0;
}

// Fetch the nth word-sized system call argumen
t as a null-terminated string.
// Copies into buf, at most max.
// Returns string length if OK (including nul),
-1 if error.
int argstr(int n, char *buf, int max)
{
    uint64 addr;
    if (argaddr(n, &addr) < 0)
        return -1;
    return fetchstr(addr, buf, max);
}

extern uint64 sys_chdir(void);
extern uint64 sys_close(void);
...

static uint64 (*syscalls[])(void) = {
    [SYS_fork] sys_fork,
    [SYS_exit] sys_exit,
    ...
};

static char *sysnames[] = {
    [SYS_fork] "fork",
    [SYS_exit] "exit",
    ...
};

void syscall(void)
{
    int num;
    struct proc *p = myproc();

```

```

    num = p->trapframe->a7;
    if (num > 0 && num < NELEM(syscalls) && syscalls[num])
    {
        p->trapframe->a0 = syscalls[num]();
        // trace
        if ((p->tmask & (1 << num)) != 0)
        {
            printf("pid %d: %s -> %d\n", p->pid, sysnames[num], p->trapframe->a0);
        }
    }
    else
    {
        printf("pid %d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

uint64
sys_test_proc(void)
{
    int n;
    argint(0, &n);
    printf("hello world from proc %d, hart %d, arg %d\n", myproc()->pid, r_tp(), n);
    return 0;
}

uint64
sys_sysinfo(void)
{
    uint64 addr;
    // struct proc *p = myproc();

    if (argaddr(0, &addr) < 0)
    {

```

```

        return -1;
    }

    struct sysinfo info;
    info.freemem = freemem_amount();
    info.nproc = procnum();

    // if (copyout(p->pagetable, addr, (char *)&info, sizeof(info)) < 0) {
    if (copyout2(addr, (char *)&info, sizeof(info)) < 0)
    {
        return -1;
    }

    return 0;
}

uint64 sys_shutdown(void)
{
    // printf("Shutdown hear\n");
    sbi_shutdown();
    return 0;
}

```

- `syscall.c` 的构成大概如是：
 - include
 - 封装了 `copyin2()`、`copyinstr2()` 等安全使用用户态内容的 fetch 函数
 - 解析 `trapframe` 的参数读取与转换
 - 外部 `sys_*` 函数声明（`syscall.c` 实现了部分的声明，即 `sys_shutdown()`，不过后面应该还是把它放在别的文件里实现才行）
 - 系统调用表与名字表
 - `syscall()` 分发函数
 - 后续实例调用实现&辅助函数

- 这里简要说一下大概都什么作用
 - `fetch`函数部分：安全地在内核与用户虚拟地址空间之间拷贝或读取数据，避免直接解引用用户指针导致内核崩溃或越权。
 - 这里的 `copyin2` 等函数，来自于头文件 `vm.h`，是安全从用户态空间做地址转换后拷贝到内核态的函数
 - `argraw` / `argint` 等：解析当前进程（`*p = myproc()`）的 `trapframe` 结构体，并将其内容转换成内核态可以直接用的变量，都使用指针来进行参数传递以及结果传递，不是直接返回结果。
 - 这里的 `trapframe` 是一个结构体，上文提到过要把 `epc` 向后移动4 Byte，`epc` 实际上就是用户态的 `pc` 指针，指着要执行的指令。后面还存了一大堆寄存器。

```
struct trapframe {
    /* 0 */ uint64 kernel_satp;    // k
    /* 8 */ uint64 kernel_sp;      // t
    /* 16 */ uint64 kernel_trap;   // u
    /* 24 */ uint64 epc;           // s
    /* 32 */ uint64 kernel_hartid; // s
    /* 40 */ uint64 ra;
    /* 48 */ uint64 sp;
    ...
    /* 112 */ uint64 a0;
    /* 120 */ uint64 a1;
    /* 128 */ uint64 a2;
    /* 136 */ uint64 a3;
    /* 144 */ uint64 a4;
    /* 152 */ uint64 a5;
    /* 160 */ uint64 a6;
    /* 168 */ uint64 a7;
    ...
}
```

```
/* 280 */ uint64 t6;
};
```

- `argraw`：一个结构体，实际上干了一个小函数的工作，将 `trapframe` 存的几个a寄存器内容直接放在 `argraw` 里。
- `argint(int n, int *ip)`：取系统调用时传递的第n个参数 (a_n 寄存器内容)，这里的 `ip` 指向一个 `argraw` 结构体，直接让它完成对于 `trapframe` 的参数取用。
- `argaddr(int n, uint64 *ip)`：同上面的逻辑，但这里的 `*ip` 是一个64位无符号整数指针，用于将 `trapframe` 中的第n参数地址存在 `*ip` 指针中传出。
- `argstr(int n, char *buf, int max)`：检查一下传入的地址（第n个word-size的位置）是否合法，将其传入 `buf` 开好的数组内，最大传 `max` 个，并返回传递了多少个char。
- 外部 `sys_*` 函数声明：告诉 `syscall()` 这些函数在外部实现过了，同时给后面的调用表提供一个函数接口。（这块就是，如果实现什么新的内核态函数，要在这里添加声明）。
- 系统调用表和名字表：
 - `static uint64 (*syscalls[])(void)`：这是一个存放函数指针的结构体，把系统调用号（宏 `SYS_fork`）与真正要调用的内核态函数建立映射。
 - `static char *sysnames[]`：名字表，一个char数组，用于将系统调用号和名字建立映射。
- `syscall()`：核心分发函数。
- `syscall()`：核心分发函数，从 `trapframe` 的 a_7 中取得系统调用号（`SYS_*`）
`p->trapframe->a0 = syscalls[num]();` 这一步则直接调用了系统调用表中的内核态函数，并将返回值放入了 a_0 寄存器中。
- 至此，我们完成了从用户态运行测试样例，到最终执行内核态函数的过程。

▼ 其他脚本解释

- `sysnum.h`：声明了系统调用号的宏（`#define SYS_getcwd 17`），如果有新的函数，要在这声明系统调用号

- `init.c`：这是用户态执行的第一个脚本，这里就包含了执行测试样例的部分，对于 `tests[]` 中的每个测试名，`fork()` 出一个子进程，子进程 `exec` 一个 `fs.img` 镜像中的可执行文件，也就是测试样例（例：`getcwd.c`）。最后依次执行完毕，执行一个 `shutdown()` 关闭系统调用。

```
char *argv[] = {0};
char *tests[] = {
    "getcwd",
    "write",
    "getpid",
    "times",
    "uname",
};
...
int main(void)
{
    int pid, wpid;

    // if(open("console", O_RDWR) < 0){
    //     mknod("console", CONSOLE, 0);
    //     open("console", O_RDWR);
    // }
    dev(O_RDWR, CONSOLE, 0);
    dup(0); // stdout
    dup(0); // stderr

    for (int i = 0; i < counts; i++)
    {
        printf("init: starting %s\n", tests[i]);
        pid = fork();
        if (pid < 0)
        {
            printf("init: fork failed\n");
            exit(1);
        }
        if (pid == 0)
        {
            // printf("IIIinit.c testing\n");
        }
    }
}
```

```

        exec(tests[i], argv);
        printf("init: exec %s failed\n", tests[i]);
        exit(1);
    }

    for (;;)
    {
        // this call to wait() returns if the shell exits,
        // or if a parentless process exits.
        wpid = wait((int *)0);
        if (wpid == pid)
        {
            // the shell exited; restart it.
            break;
        }
        else if (wpid < 0)
        {
            printf("init: wait returned an error\n");
            exit(1);
        }
        else
        {
            // it was a parentless process; do nothing.
        }
    }
}
shutdown();
return 0;
}

```

- `fs.img`：一个镜像，挂载测试样例的所有可执行文件。
- `usys.pl`（perl脚本）：生成 `usys.S`，这个汇编代码会把系统调用号load进入 a_7 寄存器，包含了 `sysnum.h` 头文件，直接将宏对应整数加载到寄存器中。如果有新的函数，必须在这里注册 `entry`，只有这样，才能为我们创建的函数生成一个函数符号，用户态（执行测试样例）在进行 `ecall` 时，前一步的 `syscall(SYS_getcwd, buf, size);` 将用户态的系统调用号传入 a_7 ，但如果用户态这一端没有 `usys.S` 中对应 `entry`，就找不到要链接哪个函数符号了。

```

print "# generated by usys.pl - do not edit\n";

print "#include \"kernel/include/sysnum.h\"\n";

# 下面这个sub entry就是后面产生.S文件的生成器，每个后面声明的
entry都会有一个对应的.S条目

sub entry {
    my $name = shift;
    print ".global $name\n";
    print "${name}:\n";
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
...

```

▼ getcwd

- 用户态的两个参数之一（`size`），就通过 `trampoline` 在保护现场的同时，存入了 `trapframe` 中，这样我们就可以用 `argint(1, &size)` 直接得到 `size` 这个int类型参数了
- 这里看到 `getcwd()` 有两个参数，一个是预留下来用于存储后面地址的char数组 `buf`，一个就是这个char数组的大小。如果成功getcwd，`cwd` 将不是NULL。

```

void test_getcwd(void){
    TEST_START(__func__);
    char *cwd = NULL;
    char buf[128] = {0};

```

```

        cwd = getcwd(buf, 128);
        if(cwd != NULL) printf("getcwd: %s successfully!\n", buf);
        else printf("getcwd ERROR.\n");
        TEST_END(__func__);
    }

```

- 查找内核态对应的内核态函数 `sys_getcwd()`，这是改完之后的样子，要加入第二个参数size的判断逻辑，并修改错误情况返回的值，将其设为 `NULL`。

```

uint64
sys_getcwd(void)
{
    uint64 addr;
    int size;
    if (argaddr(0, &addr) < 0 || argint(1, &size) < 0)
        return NULL;
    // if (argaddr(0, &addr) < 0)
    //     return -1;

    struct dirent *de = myproc()->cwd;
    char path[FAT32_MAX_PATH];
    char *s = path + sizeof(path) - 1;
    int len;

    if (de->parent == NULL)
    {
        s = "/";
    }
    else
    {
        s = path + FAT32_MAX_PATH - 1;
        *s = '\0';
        while (de->parent)
        { // 这步是递归地向上寻找父目录，直到根目录为止。每找到一个父目录，就把当前目录的名字写到路径字符串的前面。
            len = strlen(de->filename);
            s -= len;

```

```

        if (s <= path) // can't reach root "/"
            return NULL;
        strncpy(s, de->filename, len);
        s--;
        if (s <= path) // can't reach root "/"
            return NULL;
        *s = '/';
        de = de->parent;
    }
}

// if (copyout(myproc()->pagetable, addr, s, strlen
(s) + 1) < 0)
    if (size < strlen(s) + 1) // check buffer size
        return NULL;
    if (copyout2(addr, s, strlen(s) + 1) < 0)
        return NULL;

    return addr;
}

```

▼ shutdown

- 在init.c的main()最后，应该调用一个shutdown()，否则不能正常通过测试点，退出系统。而这个sys_shutdown()并不是syscall.c中注册过的函数，因此需要为这个函数进行注册。这个函数的具体实现，实际上封装一下sbi_shutdown()就行了。

```

uint64
sys_shutdown(void)
{
    // printf("Shutdown hear\n");
    sbi_shutdown();
    return 0;
}

```

- 为什么不能直接在 `init.c` 中直接调用 `sbi_shutdown()`：`init.c` 是用户态的脚本存放在xv6-user文件夹下，但sbi相关函数是平台/固件接口，是内核态的函

数，相关头文件 `sbi.h` 存放在kernel文件夹下，修改这个逻辑会导致 `pid 1`
`initcode: unknown sys call 8`

▼ times

- `times()` 传入一个参数，也就是 `tms` 结构体实例 `mytimes` 的地址，最后不会检查这个结构体中是否有正常的数据，但是会检查用户态调用的`times()`是否正确引起了内核态函数 `sys_times`，并取得了 ≥ 0 的 `ret`。

```
struct tms
{
    long tms_utime;
    long tms_stime;
    long tms_cutime;
    long tms_cstime;
};

struct tms mytimes;

void test_times()
{
    TEST_START(__func__);

    int test_ret = times(&mytimes);
    assert(test_ret >= 0);
    printf("mytimes success\n{tms_utime:%d, tms_stime:%d, tms_cutime:%d, tms_cstime:%d}\n",
           test_ret, mytimes.tms_utime, mytimes.tms_stime, mytimes.tms_cutime, mytimes.tms_cstime);
    TEST_END(__func__);
}
```

- `syscall()`：前面我们知道，内核态函数的返回值，在这次调用结束后，由 `syscall()` 的 `p->trapframe->a0 = syscalls[num]();` 处理后会出现在 `trapframe` 中的 `a0` 变量中，并最后恢复上下文时，存在 `a0` 寄存器里，
- `copyout2`：它把内核缓冲区的数据复制到进程的用户虚拟地址空间（由 `addr` 指定），写入的是用户内存，不会写入 `trapframe`。如果 `copyout2` 成功，用户进程在自己的地址 `addr` 上就能读到那段数据。在这个测试样例中，这个地址也就是指向 `tms` 结构体的指针。

- 也就是说如果能够让内核态函数实现正确返回0，这个测试样例就有分，甚至不用管是否给 `&mytimes` 里正确赋值。这样实测就可以通过。但是正确的逻辑是，使用 `timer.c` 里的 `ticks`，并在 `timer.h` 中声明一个跟用户态 `tms` 结构体一样的结构，用于对应地址指针 `&mytimes` 中的各部分，最后用 `copyout2`，将正确答案传回。
- 不过值得注意的是：`sys_times()` 在内核态并未被声明，因此我们需要给他从头到尾注册一下。

```
uint64
sys_times(void)
{
    // uint64 addr;
    // if (argaddr(0, &addr) < 0)
    //     return -1;

    // struct tms t;
    // acquire(&tickslock);
    // t.utime = ticks;
    // t.stime = ticks;
    // t.cutime = ticks;
    // t.cstime = ticks;
    // release(&tickslock);
    // if (copyout2(addr, (char *)&t, sizeof(t)) < 0)
    //     return -1;
    return 0;
}
```

▼ uname

- 跟times的要求一样，只要能正常引发内核态调用 `sys_uname()` 并返回0就行了。

```
struct utsname {
    char sysname[65];
    char nodename[65];
    char release[65];
    char version[65];
    char machine[65];
    char domainname[65];
};
```

```

};

struct utsname un;

void test_uname() {
    TEST_START(__func__);
    int test_ret = uname(&un);
    assert(test_ret >= 0);

    printf("Uname: %s %s %s %s %s %s\n",
        un.sysname, un.nodename, un.release, un.version,
        un.machine, un.domainname);

    TEST_END(__func__);
}

```

- 至于 `utsname` 的内容，自己写合适的就好。

```

uint64
sys_uname(void)
{
    uint64 addr;
    if (argaddr(0, &addr) < 0)
        return -1;
    struct utsname un;
    strncpy(un.sysname, "Miss", sizeof(un.sysname));
    strncpy(un.nodename, "DSY", sizeof(un.nodename));
    strncpy(un.release, ",", sizeof(un.release));
    strncpy(un.version, "I", sizeof(un.version));
    strncpy(un.machine, "love", sizeof(un.machine));
    strncpy(un.domainname, "you!", sizeof(un.domainname));
    if (copyout2(addr, (char *)&un, sizeof(un)) < 0)
        return -1;
    return 0;
}

```


lab2

要求：跟lab1差不多意思，但是要实现这些测试样例：

`test_wait` `test_clone` `test_fork` `test_execve` `test_getppid` `test_exit` `test_yield`
`test_waitpid` `test_gettimeofday` `test_sleep`

▼ 实验前置：

- 我们在init.c里添加这几个测试样例名：

```
char *tests[] = {  
    "getcwd",  
    "write",  
    "getpid",  
    "times",  
    "uname",  
  
    // lab2  
  
    "wait",  
    "clone",  
    "fork",  
    "execve",  
    "getppid",  
    "exit",  
    "yield",  
    "waitpid",  
    "gettimeofday",  
    "sleep",  
};
```

- 执行make fast_renew：
 - 我在Makefile里新添加了这个用法，不知道逻辑对不对，是跟着助教的视频一步一步抠出来的，每次git push之前都make fast_renew一下，提交到平台上也是正常有分的。

```

fast_renew:
    @make clean
    @make build
    @make dump
    @make fs
    @make clean
    @make run

```

- 然后观察输出，会看到有这么几个不明的系统调用号，查询 https://github.com/oscomp/testsuite-for-oskernel/blob/main/oscomp_syscalls.md 这个文档，我们可以找到这几个调用号对应的定义：

- 220: `#define SYS_clone 220`
- 260: `#define SYS_wait4 260`
- 173: `#define SYS_getppid 173`
- 169: `#define SYS_gettimeofday 169`

```

===== START test_wait =====
pid 7 wait: unknown sys call 220
pid 7 wait: unknown sys call 260
--- Assert Fatal ! ---
init: starting clone
===== START test_clone =====
pid 8 clone: unknown sys call 220

--- Assert Fatal ! ---
init: starting fork
===== START test_fork =====
pid 9 fork: unknown sys call 220

--- Assert Fatal ! ---
init: starting execve
===== START test_execve =====
    I am test_echo.
execve success.
===== END main =====

```

```

init: starting getppid
===== START test_getppid =====
pid 11 getppid: unknown sys call 173
    getppid error.
===== END test_getppid =====
init: starting exit
===== START test_exit =====
pid 12 exit: unknown sys call 220

    --- Assert Fatal ! ---
init: starting yield
===== START test_yield =====
pid 13 yield: unknown sys call 220
pid 13 yield: unknown sys call 220
pid 13 yield: unknown sys call 220
pid 13 yield: unknown sys call 260
pid 13 yield: unknown sys call 260
pid 13 yield: unknown sys call 260
===== END test_yield =====
init: starting waitpid
===== START test_waitpid =====
pid 14 waitpid: unknown sys call 220

    --- Assert Fatal ! ---
init: starting gettimeofday
===== START test_gettimeofday =====
pid 15 gettimeofday: unknown sys call 169
pid 15 gettimeofday: unknown sys call 169
gettimeofday error.
===== END test_gettimeofday =====
init: starting sleep
===== START test_sleep =====
pid 16 sleep: unknown sys call 169

    --- Assert Fatal ! ---

```

- 在 `sysnum.h` 中添加这些调用号以后，在 `syscall.c` 中进行函数声明：

```
extern uint64 sys_clone(void);
extern uint64 sys_wait4(void);
extern uint64 sys_getppid(void);
extern uint64 sys_gettimeofday(void);
```

进行系统调用号对应函数指针的系统调用表补充：

```
[SYS_clone] sys_clone,
[SYS_wait4] sys_wait4,
[SYS_getppid] sys_getppid,
[SYS_gettimeofday] sys_gettimeofday,
```

进行名字表的对应：

```
[SYS_clone] "clone",
[SYS_wait4] "wait4",
[SYS_getppid] "getppid",
[SYS_gettimeofday] "gettimeofday",
```

- 别忘了在 `usys.pl` 中为这些函数名注册，否则用户态在链接时，就不知道要怎么链接函数名了：

```
entry("clone");
entry("getppid");
entry("wait4");
entry("gettimeofday");
```

- 接下来就可以挨个函数进行实现了。

▼ wait (wait4):

- 在 `sysproc.c` 这个文件中，我们搜索一下 `wait`，看看有没有已经实现的内容。在之后的所有样例实现之前，我都会搜索一下，看看我们要做的是仅仅对齐系统调用号还是要重头定义并实现一个函数呢。

```
uint64
sys_wait(void)
{
    uint64 p;
```

```

    if (argaddr(0, &p) < 0)
        return -1;
    return wait(p);
}

```

- 这里已经给出了一个 `wait` 的函数，其实这个就是测试仓库里之前查到的 `wait4` 那个调用号对应的实现效果，我们也把它完全复制过来，然后改一下函数名，获得我们自己的 `wait4()` 函数。
- 再看看具体需要我们做什么：

```

// wstatus表示当前子进程的退出状态
int cpid, wstatus;
cpid = fork();
if(cpid == 0){
    printf("This is child process\n");
    exit(0);
}
else{
    pid_t ret = wait(&wstatus);
    assert(ret != -1);
    if(ret == cpid)
        printf("wait child success.\nwstatus: %d\n",
wstatus);
    else
        printf("wait child error.\n");
}

```

- 可以看到这个测试样例的运行过程是这样：先进行 `fork`，父进程中将子进程的 `pid` 存放在 `cpid` 变量中，而子进程的运行从 `fork()` 下一句开始，子进程的 `cpid=0`；在 `printf`（这个过程会比较慢，所以会切换到父进程的运行中）之后，子进程 `exit(0)` 结束；在父进程中，`ret` 为调用 `wait(&wstatus)` 的返回值，当父进程等待到这个子进程状态变化，返回的 `pid` 后，如果这个 `pid==cpid`，则测试成功。
- 看一下最关键的 `wait(&wstatus)` 到底是怎么定义的吧：

```

int wait(int *code)
{

```

```
return waitpid((_int)-1, code, 0);
```

又套了一个waitpid:

```
int waitpid(int pid, int *code, int options)
{
    return syscall(SYS_wait4, pid, code, options,
0);
}
```

发现确实最后都是使用的 `SYS_wait4` 这个系统调用。看看这两个函数的逻辑，`wait()` 传的参数为指向 `wstatus` 变量的指针，这个指针会作为 `waitpid()` 的第二个参数 `code`，并作为 `syscall` 传递给内核的第三个参数 `code`。

- 然后我们再回到内核态，也就是我们自己要实现的函数这里来，这里也有一个 `wait(p)` 函数，看一看是啥：

```
// 等待一个子进程exit, 并返回其pid
// Return -1 如果这个进程没有子进程
int wait(uint64 addr)
{
    struct proc *np;
    int havekids, pid;
    struct proc *p = myproc(); // 这里是获取当前的进程，调用了myproc

    // 获取当前进程（父进程）的锁，父进程会在后面的整个循环中一直持有这个锁
    // 直到子进程退出时尝试唤醒父进程
    // 父进程会阻塞子进程，直到它进入sleep()
    acquire(&p->lock);

    for (;;)
    {
        // 查询全局进程表proc, 找自己的子进程, np指针指向proc数组头，直至到最大限额NPROC为止
        havekids = 0;
        for (np = proc; np < &proc[NPROC]; np++)
```

```

    {
        // this code uses np->parent without holding np
        ->lock.
        // acquiring the lock first would cause a deadl
        ock,
        // since np might be an ancestor, and we alread
        y hold p->lock.
        // 检查哪个进程是p的子进程np
        if (np->parent == p)
        {
            // np->parent can't change between the check
            and the acquire()
            // because only the parent changes it, and w
            e're the parent.
            acquire(&np->lock);
            havekids = 1;
            // 找到kid之后, 如果这个子进程的状态是ZOMBIE, 则父进
            程为其处理后事
            if (np->state == ZOMBIE)
            {
                // Found one.
                pid = np->pid;
                if (addr != 0 && copyout2(addr, (char *)&np
                ->xstate, sizeof(np->xstate)) < 0)
                {
                    // 如果一开始给wait函数的传参地址是可以使用
                    的, 那就把这个僵尸子进程的退出状态码xstate
                    // 粘进去, 然后释放子进程的锁, 释放父进程的锁
                    release(&np->lock);
                    release(&p->lock);
                    // 粘贴失败就返回-1
                    return -1;
                }
                // 彻底清除np进程的页表, 解除trapframe等等
                freeproc(np);
                release(&np->lock);
                release(&p->lock);
                // 粘贴成功, 返回子进程pid
            }
        }
    }

```

```

        return pid;
    }
    release(&np->lock);
}
}

// No point waiting if we don't have any children.
if (!havekids || p->killed)
{
    release(&p->lock);
    return -1;
}

// 查了一圈以后发现没有子进程退出了，就把父进程sleep，让
被阻塞的子进程继续运行
sleep(p, &p->lock); // D0C: wait-sleep
}
}

```

- 然后这里提到了一些有关进程的结构体，我们不妨直接看一看，这样后面处理任何有关进程的东西，就不会一抹黑了。

- `struct proc`：进程结构体

```

struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    struct proc *parent;           // Parent process
    void *chan;                    // If non-zero,
    sleeping on chan
    int killed;                    // If non-zero,
    have been killed
    int xstate;                    // Exit status to
    be returned to parent's wait
    int pid;                       // Process ID
}

```



```

    // these are private to the process, so p->lock
    need not be held.
    uint64 kstack;                // Virtual address
    ss of kernel stack
    uint64 sz;                    // Size of process
    memory (bytes)
    pagetable_t pagetable;        // User page table
    pagetable_t kpagetable;       // Kernel page table
    struct trapframe *trapframe; // data page for
    trampoline.S
    struct context context;        // swtch() here
    to run process
    struct file *ofile[NFILE];    // Open files
    struct dirent *cwd;           // Current directory
    char name[16];                // Process name
    (debugging)
    int tmask;                    // trace mask
};

```

- `myproc()` : 从cpu中获取当前进程的指针

```

struct proc *
myproc(void)
{
    push_off();
    struct cpu *c = mycpu();
    struct proc *p = c->proc;
    pop_off();
    return p;
}

```

- `struct proc proc[ NPROC ]` : 全局进程表 `proc`

- 回归正题，我们现在需要实现的 `wait4` 到底是什么功能？在 `wait.c` 的测试样例中，我们知道，用户态 `wait` 的本质实际是在调用一个传入 `pid=int(-1)` 的

`waitpid`，而`waitpid`其实就是等待一个指定 `pid` 的进程结束。那么我们现有的内核态 `wait(uint64 addr)` 就不够看了，我们起码要添加一个参数，这个参数是指定等待的子进程 `pid`，也就是由用户态 `syscall(SYS_wait4, pid, code, options, 0)`；这一句传来的第二个参数。

- 调整一下我们内核态的`wait()`，让它符合这个要求：`int wait(uint64 addr, int wait_pid)`
- 对了，你还得记着调整`kernel/include/proc.h`里的声明奥：`int wait(uint64, int)`；不然编译会报错的。
- 只需要微调一下现有函数的内容，使之符合这个判断的逻辑：

```
if (np->parent == p)
{
    // np->parent can't change between the check
    and the acquire()
    // because only the parent changes it, and w
    e're the parent.
    acquire(&np->lock);

    if (wait_pid == -1 || np->pid == wait_pid)
    {
        havekids = 1;
    }

    if (np->state == ZOMBIE)
    {
        ...
    }
}
```

- 随后在`sysproc.c`里调整自带的`sys_wait`函数，以及我们的`sys_wait4`函数：

```
uint64
sys_wait(void)
{
    uint64 p;
    if (argaddr(0, &p) < 0)
        return -1;
    return wait(p, -1);
}
```

这里你要注意：系统自带的 `wait` 函数对应的是正常情况下用户态的 `wait` 调用，因此第一个参数也就是 `wstatus` 的指针，但我们自己的 `wait4` 要注意到用户态的 `waitpid` 实际上第二个参数才是 `wstatus` 的指针，所以在这里要对顺序进行调整，第一个参数是 `pid` 才对

```
uint64
sys_wait4(void)
{
    uint64 addr;
    int wait_pid;
    if (argint(0, &wait_pid) < 0 || argaddr(1, &addr) <
    0)
        return -1;
    return wait(addr, wait_pid);
}
```

▼ clone:

- 这个在 `sysproc.c` 里没有，我们得自己实现一个
- 看看我们该做什么：

```
size_t stack[1024] = {0};
static int child_pid;

static int child_func(void){
    printf("  Child says successfully!\n");
    return 0;
}

void test_clone(void){
    TEST_START(__func__);
    int wstatus;
    child_pid = clone(child_func, NULL, stack, 1024,
    SIGCHLD);
    assert(child_pid != -1);
    if (child_pid == 0){
        exit(0);
    }else{
```

```

        if(wait(&wstatus) == child_pid)
            printf("clone process successfully.\npid:%d\n", child_pid);
        else
            printf("clone process error.\n");
    }

    TEST_END(__func__);
}

```

- 就是要执行 `clone()`，在child进程（因为clone和fork有所区分）中执行 `exit`，然后再检查父进程的 `wait` 是否等到了自己 `clone` 出来的child进程的 `exit`
- 但这里的stack, child_func都只是很像形式上的占位，但是如果我们再看看clone_test.py，就发现这个child_func是需要被正确执行才可以的，但是如果我们只是给sys_clone()套壳成fork()，是拿不到测试点全部的3分的
- 看一下所谓的clone在用户态是怎么定义的：

```

pid_t clone(int (*fn)(void *arg), void *arg, void
*stack, size_t stack_size, unsigned long flags)
{
    if (stack)
        stack += stack_size;    // 由于地址增长
        放下向低地址，这里要还回去，变成堆顶指针

    return __clone(fn, stack, flags, NULL, NULL, N
ULL);
    //return syscall(SYS_clone, fn, stack, flags,
    NULL, NULL, NULL);
}

```

- 这里我觉得应该把第一个 `return` 注释掉，用第二个 `return`，这样我们才能正常的让它链接到我们自己的内核态 `clone` 逻辑？其实并不是的，这里的 `__clone`其实是一个非常精妙的汇编封装，我们在 `testsuits-for-oskernel/riscv-syscalls-testing/user/src/oscomp/clone.s`找到了有关 `__clone`的功能：

```

# __clone(func, stack, flags, arg, ptid, tls, ctid)
#          a0,    a1,    a2,  a3,   a4,  a5,   a6

# syscall(SYS_clone, flags, stack, ptid, tls, ctid)
#          a7    a0,    a1,   a2,  a3,   a4

.global __clone
.type __clone, %function
__clone:
    # Save func and arg to stack
    addi a1, a1, -16
    sd a0, 0(a1)
    sd a3, 8(a1)

    # Call SYS_clone
    mv a0, a2
    mv a2, a4
    mv a3, a5
    mv a4, a6
    li a7, 220 # SYS_clone
    ecall

    beqz a0, 1f
    # Parent
    ret

    # Child
1:    ld a1, 0(sp)
    ld a0, 8(sp)
    jalr a1

    # Exit
    li a7, 93 # SYS_exit
    ecall

```

我们不需要读太明白这个汇编代码的意思，需要注意的是上面人为写的注释，这告诉我们传参的情况，以及这个汇编封装到底做了什么。

- 首先最关键的一步，在整段__clone开始的第一个部分中，它将a0和a3的参数存进了a1参数为地址头的第0个和第8个位置，我们看一下具体的逻辑：

```
# __clone(func, stack, flags, arg, ptid, tls, c
tid)
#          a0,    a1,    a2,  a3,    a4,  a5,
a6

# syscall(SYS_clone, flags, stack, ptid, tls, c
tid)
#          a7      a0,    a1,    a2,  a3,
a4

.global __clone
.type __clone, %function
__clone:
    # Save func and arg to stack
    addi a1, a1, -16    # 这里先把指向stack的栈顶
                        # 指针向低地址移动了16字节，相当于开辟了一块16字节的栈空间
    sd a0, 0(a1)        # 将第一个参数func（一个8
                        # 字节大小的函数指针）放在栈的开头
    sd a3, 8(a1)        # 将第四个参数arg放在func
                        # 后面的栈空间里
```

- 随后就是调整参数位置，执行ecall指令并进入SYS_clone的系统调用。其实在后面这里，相当于运行了一次 `syscall(SYS_clone, flags, stack, ptid, tls, ctid)` 指令，那么我们就不能在 `trapframe` 中再直接取到 `func` 参数了，那就不能直接执行这个 `func`
- 然后在 `ecall` 执行完毕返回这个汇编封装后，parent和child各自执行自己的逻辑，这里是经典的分流操作，如果a0是子进程，也就是 `beqz`（是否等于0）为真，则跳转至前方的1标签处

```
beqz a0, 1f          # 检查ecall系统调用SYS_clone以
后，a0作为系统调用的返回值是什么
```

parent: 直接 `ret` 了

child: 从栈指针的0字节位置取出, 并存入`a1`, 从8字节位置取出并存入`a0`, 然后再跳转执行`a1`中的指令, 最后再系统调用一次 `exit` 就结束啦

```
1:      ld a1, 0(sp)
        ld a0, 8(sp)
        jalr a1

        # Exit
        li a7, 93 # SYS_exit
        ecall
```

- 然后我们回到内核态, 在助教给的文档中, 说到了 `clone` 和 `fork` 其实只有细微的差别, 但是差别是什么我们先不管, 先看看真正的 `fork` 是啥样的:

```
int fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    // 这个allocproc的逻辑其实就是搜索全局进程表proc (我们在wait里分析过了), 找一个能用的进行初始化
    if ((np = allocproc()) == NULL)
    {
        return -1;
    }

    // 将父进程的页表全部真正拷贝到一块新的np进程空间中
    if (uvmcopy(p->pagetable, np->pagetable, np->kpagetable, p->sz) < 0)
    {
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    // 子进程与父进程size一样
```

```

np->sz = p->sz;
    // 设置父进程为p
np->parent = p;

// copy tracing mask from parent.
np->tmask = p->tmask;

// 由于子进程与父进程共享同样的用户态上下文，因此直接全部粘过去
*(np->trapframe) = *(p->trapframe);

// 为了区分系统调用结束后谁是子进程（注意这个fork就是一次系统调用），
// 把子进程的a0强行返回0，父进程的a0则返回子进程的pid
np->trapframe->a0 = 0;

// 共享并增加一次打开文件的计数器
for (i = 0; i < NOFILE; i++)
    if (p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
np->cwd = edup(p->cwd);

safestrcpy(np->name, p->name, sizeof(p->name));

pid = np->pid;

// 现在子进程可以被运行，并释放锁
np->state = RUNNABLE;

release(&np->lock);

return pid;
}

```

- 然后阅读一下[clone \(2\) - Linux 手册页面](#) --- [clone\(2\) - Linux manual page](#)，有一大堆很复杂的形容，大致意思就是 `clone` 可以更精细地通过 `flags` 参数进行选择，是软拷贝还是硬拷贝父进程的一些结构等等的。

- 不过我们管他那么多的，我们根本不用处理 `flags`，我们只需要在 `clone` 之后子进程能够直接执行 `child_func` 不就得了。
- 不过在此之前，我们先试试看如果直接用 `fork` 的逻辑原封不动地跑这个样例会怎么样：

```
uint64
sys_clone(void)
{
    return fork();
}
```

```
init: starting clone
===== START test_clone =====
usertrap(): unexpected scause 0x0000000000000002 p
id=10 clone
sepc=0x0000000000000000 stval=0x0000000000000000
clone process successfully.
pid:10
===== END test_clone =====
```

发现报错了：sepc=0表示引发异常的那条指令的地址，而scause=2则表示这是执行了非法指令导致的报错。也就是说在子进程结束SYS_clone以后返回汇编封装时，它执行了那两条从栈顶取数据的指令，但是在跳转a1并执行时，发现这个在0x0处的指令完全是非法的，这是因为在0x0的地址处，大部分都是空映射0，而RISCV的指令集编码中，全0是不属于任何有效指令的。

- 其实这个原因很明确，我们的 `__clone` 本来只是把 `func` 这条指令存到了 `stack` 变量中，这个 `stack` 是最后我们希望在 `ecall` 结束并返回后，`sp` 指向的位置，这个 `sp` 指向这里的话，就会从 `stack` 的顶端得到我们想要的 `func` 指令，然后再正常执行 `child_func`，完成 `printf` 通过测试点，可是传统的 `fork` 逻辑里，

```
// 由于子进程与父进程共享同样的用户态上下文，因此直接全部粘过去
*(np->trapframe) = *(p->trapframe);
```

这一步把所有的父进程 `trapframe` 全都给子进程了，子进程的 `sp` 这时候实际上指向的仍然是父进程的旧栈，而不是我们要的 `stack` 变量顶，所以在最后 `ecall` 结束返回时，`sp` 就只能从父进程的旧栈读数据，然后就报错了

- 然后我们的操作就是，从

```
# syscall(SYS_clone, flags, stack, ptid, tls, ctid)
#          a7      a0,    a1,   a2,   a3,   a4
# Call SYS_clone
    mv a0, a2
    mv a2, a4
    mv a3, a5
    mv a4, a6
    li a7, 220 # SYS_clone
    ecall
```

这一部分，找到我们的 `stack` 变量究竟被存在哪一个参数里，交给了我们的内核，其实显而易见就在 `a1` 里面了，然后我们就可以添加这样一个逻辑（注意其他的代码都是直接复制 `fork` 的代码），这样就可以实现我们的要求

```
*(np->trapframe) = *(p->trapframe);    // 这是原本fork
的代码
// xv6中，如果调用clone传了有效的stack地址，则必须重置子进程
栈顶和相关的a1参数！
    if (stack != 0)
    {
        np->trapframe->sp = stack;
    }
np->trapframe->a0 = 0;                    // 这是原本fork
的代码
```

然后在 `proc.c` 中，我们的 `clone` 函数长成这个样子：

```
int clone(uint64 stk)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();
    uint64 stack = stk;
```

```

...    // 同于fork()
*(np->trapframe) = *(p->trapframe);

// 除了这个if, 其他都是fork的原封不动
if (stack != 0)
{
    np->trapframe->sp = stack;
}

np->trapframe->a0 = 0;
...
}

```

在 `proc.h` 里声明我们这个函数 `int clone(uint64 stack);` 这一步有点像前面的 `wait` 的操作, 最后在 `sysproc.c` 里, 把我们的 `sys_clone` 完善一下就OK了

```

uint64
sys_clone(void)
{
    uint64 stack;
    if (argaddr(1, &stack) < 0)
        return -1;
    return clone(stack);
}

```

▼ fork:

好像正常实现了wait之后, fork直接就通过了

▼ execve:

- 查了一下发现 `sysproc.c` 里有这个实现, 然后看看我们该做什么:

```

void test_execve(void){
    TEST_START(__func__);
    char *newargv[] = {"test_echo", NULL};
    char *newenviron[] = {NULL};
    execve("test_echo", newargv, newenviron);
    printf("  execve error.\n");
}

```

```

        //TEST_END(__func__);
    }

```

虽然说这里我们什么都不用管，直接 `make fast_renew` 就能通过，但是我们还是看看这个传参逻辑吧：

- 看一下 `execve` 在用户态的实现，就是一个很简单的传参，然后执行 `syscall` 进行系统调用

```

int execve(const char *name, char *const argv[], c
har *const argp[])
{
    return syscall(SYS_execve, name, argv, argp);
}

```

这里有一个情况，我们在用户端调用的分明是 `SYS_execve` 信号，为啥到了内核态，却是由 `SYS_exec` 来接手下面的操作呢？这是因为在用户端，如果我们查询这个 `SYS_execve` 信号的定义，它其实是一个对应221数字的调用号，而我们在内核态，只是看到有人在调用221这个调用号，那在内核态来看，不就是我们定义的 `#define SYS_exec 221` 吗，那就用这个 `SYS_exec` 及其对应的函数什么的来响应好了

- 那我们看看内核态的 `execve` 函数吧

```

uint64
sys_exec(void)
{
    char path[FAT32_MAX_PATH], *argv[MAXARG];
    int i;
    uint64 uargv, uarg;

    // 非常正常的取参数过程，跟用户态的syscall调用完全可以对齐
    if (argstr(0, path, FAT32_MAX_PATH) < 0 || argadr(1, &uargv) < 0)
    {
        return -1;
    }
    memset(argv, 0, sizeof(argv));
    // 循环搬运参数数组，把这些以NULL结尾的字符串指针一个一

```

个搬过来

```
for (i = 0;; i++)
{
    if (i >= NELEM(argv))
    {
        goto bad;
    }
    if (fetchaddr(uargv + sizeof(uint64) * i, (uint64 *) &uarg) < 0)
    {
        goto bad;
    }
    if (uarg == 0)
    {
        argv[i] = 0;
        break;
    }
    argv[i] = kalloc();
    if (argv[i] == 0)
        goto bad;
    if (fetchstr(uarg, argv[i], PGSIZE) < 0)
        goto bad;
}
// 包装了一下，把用户态的参数都规规矩矩处理好，交给exec真正进行执行
int ret = exec(path, argv);

for (i = 0; i < NELEM(argv) && argv[i] != 0; i++)
    kfree(argv[i]);

return ret;

bad:
for (i = 0; i < NELEM(argv) && argv[i] != 0; i++)
    kfree(argv[i]);
```

```
    return -1;
}
```

- 接下来的exec具体是啥我们就不看了，总之是执行了“test_echo”这个可执行文件，得到了我们需要的测试样例结果

```
#include "stdio.h"

/*
 * for execve
 */

int main(int argc, char *argv[]){
    printf(" I am test_echo.\nexecve success.\n");
    TEST_END(__func__);
    return 0;
}
```

▼ getpid:

- 看看具体做什么吧

```
int test_getppid()
{
    TEST_START(__func__);
    pid_t ppid = getppid();
    if(ppid > 0) printf(" getppid success. ppid : %d\n", ppid);
    else printf(" getppid error.\n");
    TEST_END(__func__);
}
```

就只需要得到>0的 `ppid` 就行了，甚至你都不用 `getppid`，直接调用 `getpid` 都通过了，这里会得到 `pid=14`

但是我们还是修改一下ppid的逻辑，万一后面有用

- 编写一个 `sys_ppid` 函数，这就可以了，这里会得到 `ppid=1`，其实就是我们的 `init.c` 得到的初始进程，这也比较明显，因为 `init.c` 中，我们执行循环，然

后 `fork` 出来其他的进程，我们总共进行了9个测试样例，到这里是第10个，从1往后数，这是第11个，但是我们在 `clone` 那看到，这时 `clone` 出来的子进程占用了 `pid=10`（之前的 `wait` 也占用了2个pid），随后 `fork` 样例执行后又占用了两个 `pid`，到这里 `getppid` 测试样例的进程，正好到 `pid=14` 了

```
uint64
sys_getppid(void)
{
    return myproc()->parent ? myproc()->parent->pid :
    0;
}
```

▼ exit:

- 好像跟wait.c要求差不多？能正常退出就行

```
void test_exit(void){
    TEST_START(__func__);
    int cpid, waitret, wstatus;
    cpid = fork();
    assert(cpid != -1);
    if(cpid == 0){
        exit(0);
    }else{
        waitret = wait(&wstatus);
        if(waitret == cpid) printf("exit OK.\n");
        else printf("exit ERR.\n");
    }
    TEST_END(__func__);
}
```

这个exit在内核态我们就不用额外实现了，`sysproc.c` 里面的 `sys_exit` 也就是对 `exit` 做了一个提取参数的前置套壳工作，`exit()` 函数在 `proc.c` 里有实现，我们就不多说了，直接可以正常通过测试点

▼ yield:

- 如果我们直接执行make fast_renew，会看到这里抛出这样的报错

```
init: starting yield
===== START test_yield =====
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 18 yield: unknown sys call 124
I am child process: 18. iteration 0.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 19 yield: unknown sys call 124
I am child process: 19. iteration 1.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
pid 20 yield: unknown sys call 124
I am child process: 20. iteration 2.
===== END test_yield =====
```

去查一下手册，看看这个124是什么调用号，发现对应在用户态的，这个是 `#define SYS_sched_yield 124`，那我们就需要在内核态这一端，也声明一个124的系统调用号 `#define SYS_yield 124`，然后再进行一圈注册，回到 `sysproc.c`里，现在有一个纯为了能通过编译的函数 `sys_yield()`

- 看看我们需要做啥

```
/*
理想结果：三个子进程交替输出
*/

int test_yield(){
    TEST_START(__func__);

    for (int i = 0; i < 3; ++i){
        if(fork() == 0){
            for (int j = 0; j < 5; ++j){
                sched_yield();
                printf(" I am child process: %d. iteration %d.\n", getpid(), i);
            }
            exit(0);
        }
    }
    wait(NULL);
    wait(NULL);
    wait(NULL);
    TEST_END(__func__);
}
```

fork出来3个子进程，然后每个子进程都进行一次sched_yield，如果理想，我们能得到15行交替的输出

- 然后我们查一查 `proc.c`，经过之前fork、clone样例，我们知道这个脚本下面存着很多现成的函数，这些函数基本上都是经过 `sysproc.c` 处理传参问题，然后最终调用的底层函数，我们发现确实有 `yield`

```
void yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
}
```

```

    release(&p->lock);
}

```

它主要还是获取了当前进程的锁，并设置当前进程状态为 `RUNNABLE` 而非 `RUNNING`，这就是让出进程的前置操作，然后进行了 `sched()`，这个 `sched` 主要是检查当前的状态是不是可以进行进程切换，然后交给外部定义的 `swtch` 切换进程，这个样例就结束了，所以我们最终只需要得到一个这样的 `sys_yield`

```

uint64
sys_yield(void)
{
    yield();
    return 0;
}

```

▼ waitpid:

- 看看我们该做什么

```

int i = 1000;
void test_waitpid(void){
    TEST_START(__func__);
    int cpid, wstatus;
    cpid = fork();
    assert(cpid != -1);
    if(cpid == 0){
        while(i--);
        sched_yield();
        printf("This is child process\n");
        exit(3);
    }else{
        pid_t ret = waitpid(cpid, &wstatus, 0);
        assert(ret != -1);
        if(ret == cpid && WEXITSTATUS(wstatus) == 3)
            printf("waitpid successfully.\nwstatus: %
x\n", WEXITSTATUS(wstatus));
        else
            printf("waitpid error.\n");
    }
}

```

```

    }
    TEST_END(__func__);
}

```

这个又很像之前的wait要求我们做的事情，只不过这里强行把子进程的退出码设成3了，如果实现了yield和wait，这里我们完全不用进行任何操作，因为这里的系统调用函数 `waitpid` 实际上底层也是进行 `wait4` 的 `syscall`

```

int waitpid(int pid, int *code, int options)
{
    return syscall(SYS_wait4, pid, code, options, 0);
}

```

▼ gettimeofday:

- 看看我们该做什么

```

/*
 * 测试通过时的输出:
 * "gettimeofday success."
 * "start:[num], end:[num]"
 * "interval: [num]"    注: 数字[num]的值应大于0
 * 测试失败时的输出:
 * "gettimeofday error."
 */
void test_gettimeofday() {
    TEST_START(__func__);
    int test_ret1 = get_time();
    volatile int i = 12500000; // qemu时钟频率1250000
    0
    while(i > 0) i--;
    int test_ret2 = get_time();
    if(test_ret1 > 0 && test_ret2 > 0){
        printf("gettimeofday success.\n");
        printf("start:%d, end:%d\n", test_ret1, test_
ret2);
        printf("interval: %d\n", test_ret2 -
test_ret1);
    }
}

```

```

    }else{
        printf("gettimeofday error.\n");
    }
    TEST_END(__func__);
}

```

可以看到调用了两次系统调用函数 `get_time()`，第一次调用以后，经过一个 qemu 时钟周期后再次调用，最终要检查这两次调用得到的结果相减，看看是否间隔 > 0 ，这样才算完全通过，如果你在这里只是让 `sys_gettimeofday` 返回 0，就只能拿到 2 分，具体为什么，可以看一下 `gettimeofday_test.py`，它最后有一行 `interval` 与 0 的大小比较

- 看看核心函数是怎样的

```

int64 get_time()
{
    TimeVal time;
    int err = sys_get_time(&time, 0);
    if (err == 0)
    {
        return ((time.sec & 0xffff) * 1000 + time.
usec / 1000);
    }
    else
    {
        return -1;
    }
}

int sys_get_time(TimeVal *ts, int tz)
{
    return syscall(SYS_gettimeofday, ts, tz);
}

```

如果系统调用后，不产生 err，则返回一个 TimeVal 结构体中的内容，我们看看这个 TimeVal 是啥

```

typedef struct
{

```

```

uint64 sec; // 自 Unix 纪元起的秒数
uint64 usec; // 微秒数
} TimeVal;

```

那我们在内核态也应该实现这样的一个结构体，并且往里面填东西

- 发现 `syscall.c` 和 `proc.c` 中都没有我们想要的现成函数，只能自己实现一下，不过在此之前，我们先了解一下系统中内核态的 `time`、`tick` 与真实时间的关系
 - `time`：RISC-V 中硬件的 `mtime` 寄存器，只要一开机，就会按照固定的物理频率雷打不动一直+1，这个+1的速度，就是上面提到的，qemu主板的时钟频率12,500,000（12.5MHz），也就是说真实时间过1s，`mtime` 寄存器会增加12,500,000
 - `tick`：这是内核态自己维护的整数变量，定义在 `timer.c` 中，内核并不像 `mtime` 那样实时更新 `tick`，而是定了一个闹钟，这个闹钟是设定好的宏 `INTERVAL`，等 `mtime` 过了这么久以后，产生一个 `TimerInterrupt` 中断，CPU停止手里的工作，进入到 `trap.c` 然后是 `timer.c`，执行 `timer_tick`，为 `ticks+1`，随后设定下一次中断的时间，假如设置 `INTERVAL` 使得 `tick` 在每秒可触发100次时钟中断，那一个 `tick` 就是真实时间1/100秒

```

struct spinlock tickslock;
uint ticks;

void timerinit() {
    initlock(&tickslock, "time");
}

void
set_next_timeout() {
    sbi_set_timer(r_time() + INTERVAL);
}

void timer_tick() {
    acquire(&tickslock);
    ticks++;
    wakeup(&ticks);
    release(&tickslock);
}

```

```

        set_next_timeout();
    }

```

- 之后，其实有这么一个函数r_time()，是作为查看 `mtime` 的函数接口使用的，自此，我们就可以使用r_time()获取硬件时间，然后经过真实时间的换算，得到确切的时间了

```

uint64
sys_gettimeofday(void)
{
    uint64 addr;
    if (argaddr(0, &addr) < 0)
        return -1;

    uint64 t = r_time();
    struct
    {
        uint64 sec;
        uint64 usec;
    } tv;

    tv.sec = t / 12500000;
    tv.usec = (t % 12500000) * 1000000 / 12500000;

    if (copyout2(addr, (char *)&tv, sizeof(tv)) < 0)
        return -1;
    return 0;
}

```

▼ sleep

- 如果我们直接执行make fast_renew的话，会发现这样的报错

```

init: starting sleep
===== START test_sleep =====
pid 24 sleep: unknown sys call 101
-- Assert Fatal ! ---

```

不过这次我们不直接查手册了，我们直接看看我们应该做什么，也就是说测试样例会调用什么东西吧

- 看看我们要做什么

```
/*
 * 测试通过时的输出:
 * "sleep success."
 * 测试失败时的输出:
 * "sleep error."
 */
void test_sleep() {
    TEST_START(__func__);

    int time1 = get_time();
    assert(time1 >= 0);
    int ret = sleep(1);
    assert(ret == 0);
    int time2 = get_time();
    assert(time2 >= 0);

    if(time2 - time1 >= 1){
        printf("sleep success.\n");
    }else{
        printf("sleep error.\n");
    }
    TEST_END(__func__);
}
```

调用了 `get_time`，并观察两次得到的时间是否差距 ≥ 1 ，因为在中间，调用了核心的系统调用函数 `sleep(1)`，这里的 `get_time` 逻辑，我们已经在 `gettimeofday` 实现过了，我们看看这个 `sleep` 的具体实现

```
int sleep(unsigned long long time)
{
    TimeVal tv = {.sec = time, .usec = 0};
    if (syscall(SYS_nanosleep, &tv, &tv))
        return tv.sec;
```

```

    return 0;
}

```

然后直接查看 `SYS_nanosleep` 对应的系统调用号，发现 `#define SYS_nanosleep 101`，于是我们就知道，在内核态这一端，我也需要一个101的 `SYS_nanosleep` 调用号，不过在此之前，经过之前的 `SYS_sched_yield` 和 `SYS_yield` 的关系，我们还是留一个心眼，万一这里其实已经实现好了 `SYS_sleep` 这个调用号，只是它的号码没对齐为101呢？

- 。但其实并不行，因为实现好的 `sys_sleep` 的逻辑是这样的

```

uint64
sys_sleep(void)
{
    int n;
    uint ticks0;

    if (argint(0, &n) < 0)    // 取第一个参数，并将第一个
    参数看做int整数
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while (ticks - ticks0 < n)    // 执行对第一个参数的
    睡眠
    {
        if (myproc()->killed)
        {
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}

```

可是在我们 `syscall` 传参的时候，第一个参数 `&tv` 是一个uint64的地址指针，如果把它看做一个需要睡眠的时间 `n`，系统就像直接卡死在这了，因

为这个地址换算成十进制，差不多有上百亿大。因此我们还是需要自己实现一个 `sys_nanosleep`

- 然后我们注册好自己的 `sys_nanosleep`，并实现这个函数吧
 - 不过我们需要逻辑上真正的实现这个要求的秒+微秒的逻辑，所以在照搬 `sys_sleep` 之后，我们还需要计算一下这里 `ticks` 与真实时间秒的换算关系

```
uint64
sys_nanosleep(void)
{
    uint64 tvaddr;
    uint ticks0;

    if (argaddr(0, &tvaddr) < 0)
        return -1;

    struct TimeVal
    {
        uint64 sec;
        uint64 usec;
    } tv;
    // 把tv的地址当成uint64读出，而不是int
    if (copyin2((char *)&tv, tvaddr, sizeof(tv)) <
        0)
        return -1;

    acquire(&tickslock);
    ticks0 = ticks;
    // 这里要进行单位换算!
    while (ticks - ticks0 < tv.sec * 200 + tv.usec *
        200 / 1000000)
    {
        if (myproc()->killed)
        {
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
}
```

```

    release(&tickslock);
    return 0;
}

```

- 我们查询一下宏定义的 `INTERVAL`，看一看一秒内会产生多少个ticks（从这往下的这些小目录其实不太用看了，只要修改 `INTERVAL` 就行）

```

#define INTERVAL      (390000000 / 200)    // timer
interrupt interval (K210: 390MHz)

```

- 不过我们发现这个和我们之前在gettimeofday中提到的不一样啊？这里的分母并不是我们qemu用的“*qemu时钟频率12,500,000*”啊，那么这个更大的390,000,000是啥。查一下，我们知道这个390,000,000实际上是真实物理硬件K210的RISC-V双核芯片的频率**390MHz**，而我们在QEMU这个虚拟平台上模拟，为了节省宿主主机性能而设计的**12.5MHz**。
- 我想看看这个**12.5MHz**是在哪设定的，实际上这个并不是显式设定在C语言源码，而是QEMU自己写了一个虚拟主板，当运行 `qemu-system-riscv64 -machine virt ...` 这么一个指令时，这个QEMU在内存里凭空捏造了一块代号为 `virt` 的RISC-V主板，其参数会由设备树传递给操作系统内核

```

cpus {
#address-cells = <0x01>;
#size-cells = <0x00>;
timebase-frequency = <0x989680>;    // <--- 就是这里!!!

```

- 但是这里的 $0x989680 = 10,000,000$ 是**10MHz**啊，也不是代码里说的“*qemu时钟频率12,500,000*”，这是因为在官方的QEMU实现中，`timebase-frequency` 一直都是定死的10MHz，但在这里的xv6 K210实验中，早就把qemu-system-riscv64进行了魔改，因此目前我们使用的系统，它的 `mtime` 确实就是按照**12.5MHz**进行的，即使设备树里仍然是**10MHz**，这也就是为什么助教在测试样例中有这句声明

```

volatile int i = 12500000; // qemu时钟频率12500000

```

- 一句话：我们应该把 `INTERVAL` 改成，这样才能在逻辑上符合经修改后 QEMU 虚拟的 RISC-V 系统的逻辑

```
#define INTERVAL (12500000 / 200) // timer interrupt interval (QEMU: 12.5MHz)
```

- 不过实际上计算 ticks 与真实时间秒的关系，也用不上分母的数，我们看分子就行，分母的数是 `mtime` 的频率，分子才是一秒产生多少次时钟异常（产生多少个 ticks）
- 这还有一个事，虽然要求 `nanosleep`，但用户态调用的 `sleep` 那里，定义依然是 `usec`（微秒），而 $1\text{sec} = 1,000,000\text{usec}$ ，这也是为什么我们要进行 `/1,000,000` 操作

lab3

要求：实现 `brk` `mmap` `munmap` 测试样例的要求

▼ 实验前置：

- 修改 `init.c`，添加这几个测试样例点

```
char *tests[] = {
    // "getcwd",
    // "write",
    // "getpid",
    // "times",
    // "uname",

    // // lab2

    // "wait",
    // "clone",
    // "fork",
    // "execve",
    // "getppid",
    // "exit",
    // "yield",
    // "waitpid",
    // "gettimeofday",
    // "sleep",

    // // lab3

    "brk",
    "mmap",
    "munmap",
};
```

- 执行 `make fast_renew`，看看结果

```

init: starting brk
===== START test_brk =====
pid 2 brk: unknown sys call 214
Before alloc,heap pos: -1
pid 2 brk: unknown sys call 214
pid 2 brk: unknown sys call 214
After alloc,heap pos: -1
pid 2 brk: unknown sys call 214
pid 2 brk: unknown sys call 214
Alloc again,heap pos: -1
===== END test_brk =====
init: starting mmap
===== START test_mmap =====
file len: 0
pid 3 mmap: unknown sys call 222
mmap error.
===== END test_mmap =====
init: starting munmap
===== START test_munmap =====
file len: 0
pid 4 munmap: unknown sys call 222
mmap error.
===== END test_munmap =====

```

查询https://github.com/oscomp/testsuits-for-oskernel/blob/main/oscomp_syscalls.md这个文档，我们可以找到这几个调用号对应的定义：

- 214: `#define SYS_brk 214`
- 222: `#define SYS_mmap 222`
- 进行一系列注册过程，需要修改的脚本有： `sysnum.h` `syscall.c` `usys.pl`
- 接下来为了能通过编译，我们需要写几个占位用的函数
 - 进入 `sysproc.c`，挨个搜索函数，我们发现有一个系统实现的 `sys_sbrk()`，于是我们自己建一个 `sys_brk()`，先直接调用这个实现好的函数

```

uint64
sys_brk(void)

```

```
{
    return sys_sbrk();
}
```

- 对于mmap，没有发现已实现好的函数，我们就写一个这个应付一下

```
uint64
sys_mmap(void)
{
    return 0;
}
```

- 然后我们继续 `make fast_renew`，结果是这样

```
init: starting brk
===== START test_brk =====
Before alloc,heap pos: 20480
After alloc,heap pos: 41024
Alloc again,heap pos: 82112
===== END test_brk =====
init: starting mmap
===== START test_mmap =====
file len: 0
mmap content: (null)
pid 3 mmap: unknown sys call 215
===== END test_mmap =====
init: starting munmap
===== START test_munmap =====
file len: 0
pid 4 munmap: unknown sys call 215
munmap return: -1
-- Assert Fatal ! ---
```

- 我们检查一下，215调用号对应的是 `#define SYS_munmap 215`，这样我们仍然用占位置的函数注册 `sys_munmap()` 以后，再运行就会得到这样的结果，接下来就可以挨个实现这些调用了

```

init: starting brk
===== START test_brk =====
Before alloc,heap pos: 20480
After alloc,heap pos: 41024
Alloc again,heap pos: 82112
===== END test_brk =====
init: starting mmap
===== START test_mmap =====
file len: 0
mmap content: (null)
===== END test_mmap =====
init: starting munmap
===== START test_munmap =====
file len: 0
munmap return: 0
munmap successfully!
===== END test_munmap =====

```

▼ brk:

- 检查测试样例:

```

/*
 * 测试通过时应输出:
 * "Before alloc,heap pos: [num]"
 * "After alloc,heap pos: [num+64]"
 * "Alloc again,heap pos: [num+128]"
 *
 * Linux 中brk(0)只返回0, 此处与Linux表现不同, 应特殊说明.
 */
void test_brk(){
    TEST_START(__func__);
    intptr_t cur_pos, alloc_pos, alloc_pos_1;

    cur_pos = brk(0);
    printf("Before alloc,heap pos: %d\n", cur_pos);
    brk(cur_pos + 64);
    alloc_pos = brk(0);

```

```

    printf("After alloc,heap pos: %d\n",alloc_pos);
    brk(alloc_pos + 64);
    alloc_pos_1 = brk(0);
    printf("Alloc again,heap pos: %d\n",alloc_pos_1);
    TEST_END(__func__);
}

```

- 这里的brk(n)会调用用户态的

```

int brk(void *addr)
{
    return syscall(SYS_brk, addr);
}

```

而这里的传参n被包装成一个addr形式，而观察测试样例的要求，三次输出的结果应该是逐次递增64的。

- 分析一下目前直接调用的sys_sbrk():

```

uint64
sys_sbrk(void)
{
    int addr;
    int n;

    if (argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    if (growproc(n) < 0)
        return -1;
    return addr;
}

```

其中，`myproc()` 是一个定义在 `proc.c` 下的 `struct proc *` 指针，它的作用是返回当前cpu正运行的进程地址，这个时候我们去看一下 `struct proc` 的结构，每一个成员都是什么含义：

```

// Per-process state
struct proc

```



```

{
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    struct proc *parent;  // Parent process
    void *chan;           // If non-zero, sleeping on chan
    int killed;            // If non-zero, have been killed
    int xstate;            // Exit status to be returned to parent's wait
    int pid;               // Process ID

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;         // Virtual address of kernel stack
    uint64 sz;             // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    pagetable_t kpagetable; // Kernel page table
    struct trapframe *trapframe; // data page for trapframe.S
    struct context context;      // switch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct dirent *cwd;          // Current directory
    char name[16];              // Process name (debugging)
    int tmask;                  // trace mask
};

```

需要持有 `p->lock` 才能访问的成员：

- `enum procstate state`：进程当前的状态（例如：`UNUSED` 未使用、`SLEEPING` 睡眠/等待中、`RUNNABLE` 就绪可运行、`RUNNING` 正在运行、

`ZOMBIE` 僵尸状态)。

- `struct proc *parent`：指向当前进程的父进程的指针。用于在子进程退出时通知父进程。
- `void *chan`：通道标识 (channel)。如果非 0，表示当前进程正在睡眠，且正在等待某个特定的事件 (通常是某个内核数据结构的内存地址，比如锁的地址、磁盘缓冲区的地址等)。
- `int killed`：进程是否已被杀死的标志位。如果非 0，表示有其他进程请求终止该进程 (如通过 `kill` 系统调用)，它会在下一次陷入内核或从内核返回用户态前退出。
- `int xstate`：退出状态码 (Exit status)。当进程调用 `exit(status)` 退出成为僵尸进程 (ZOMBIE) 后，其退出码存放在此，等待父进程通过 `wait()` 收集。
- `int pid`：进程的全局唯一标识符 (Process ID)。

进程的私有成员，不需要持有锁：

- `uint64 kstack`：进程的内核栈 (Kernel stack) 的虚拟地址。当进程从用户态陷入内核态运行系统调用或处理中断时，使用的就是这个栈。
- `uint64 sz`：进程当前占用的用户内存大小，以字节 (bytes) 为单位。当使用 `sys_sbrk()` 等扩展内存时，这个值会改变。
- `pagetable_t pagetable`：用户态页表指针。由于每个进程有自己独立的用户虚拟地址空间，这个页表记录了该进程的用户虚拟地址到物理内存地址的映射。
- `pagetable_t kpagetable`：内核态页表指针。这个项目 (可能修改自标准的 xv6) 中，每个进程都拥有独立的内核页表结构，以便在内核态也维持精细的内存隔离或映射层级。
- `struct trapframe *trapframe`：陷入帧 (Trapframe)。一个指向专门的数据页的指针，用于在用户态发生系统调用或中断陷入内核态 (如 `trampoline.S` 中执行操作) 时，保存用户态的寄存器状态，以便之后可以恢复执行。
- `struct context context`：进程上下文。用于内核线程 (进程) 的切换。当在内核中调用 `swtch()` 函数挂起进程时，会将 CPU 的被调用者保存寄存器 (如 `s0-s11, ra, sp` 等) 保存在这里。

- `struct file *ofile[NOFILE]`：文件描述符表。这是一个指针数组，记录了进程当前打开的所有文件资源。数组索引就是文件描述符（fd）。
 - `struct dirent *cwd`：当前工作目录（Current working directory）。这代表了进程所处的目录节点。
 - `char name[16]`：进程名称。系统通常允许给进程指定一个简短的名字，主要方便调试代码或通过 `ps` 等工具查看进程情况。
 - `int tmask`：系统调用追踪掩码（Trace mask）。用于实现类似 `strace` 的功能，拦截和输出特定的系统调用，调试工具会用到。
- 综合两部分的内容，我们其实分析出来：`sys_sbrk()` 的主要作用是将用户态传的参数 `n` 解码为扩张该运行中进程内存的大小 `n` 个Byte，如果成功扩张内存，则返回扩张内存之前的进程内存大小。
- 这里其实还有一个很关键的信息，就是如果我们深入了解一下xv6系统的内存分配方式，我们可以发现所有进程的虚拟空间都是从0开始向上增长的，这个信息可以查询 `Makefile` 或者我们直接从 `sys_sbrk()` 中，可以推测出。
 - 由于 `addr = myproc() -> sz`，我们可以发现，这里的 `sz` 实际上被解码为一个地址。因此我们得到一个关键信息，就是在xv6的进程内存分配中，所有虚拟地址都是从0开始向上增长，而管理进程用的结构体 `struct proc`，其中的 `sz` 变量，实际上也是整个进程空间的堆顶地址。
- 观察目前我们直接调用 `sys_sbrk()` 的结果：

```
Before alloc,heap pos: 20480
After alloc,heap pos: 41024
Alloc again,heap pos: 82112
```

其实就可以发现 $41024 = 20480 + 20480 + 64$ ，也就是说，如果我们直接使用 `sbrk` 的逻辑，那么我们的 `brk` 测试样例实际上做了这样一件事：

```
cur_pos = brk(0);    // 分配了0个Byte，并返回了当前的堆顶地址
printf("Before alloc,heap pos: %d\n", cur_pos);
brk(cur_pos + 64);    // 又分配了堆顶地址+64大小的Byte
```

```

    alloc_pos = brk(0); // 分配0个Byte, 这一步实际上
    就是获取当前堆顶地址
    printf("After alloc,heap pos: %d\n",alloc_pos);
    brk(alloc_pos + 64); // 又分配了堆顶地址+64大小的Byte
    alloc_pos_1 = brk(0);
    printf("Alloc again,heap pos: %d\n",alloc_pos_1);

```

因此返回的内容完全不是相差64。

- 既然已经知道了问题所在，我们只需要修改一点 `sbrk` 的逻辑即可满足 `brk` 的要求，在这里，`brk` 传入的参数不同于 `sbrk`，后者传入的是需要扩展的Byte数，而前者则是直接传入一个希望扩展到的地址：

```

uint64
sys_brk(void)
{
    int addr;
    int endaddr;

    if (argint(0, &endaddr) < 0)
        return -1;
    if (endaddr == 0)
        return myproc()->sz;
    addr = myproc()->sz;
    if (growproc(endaddr - addr) < 0)
        return -1;
    return endaddr;
}

```

这里要注意：正如助教在测试样例中写的——Linux 中 `brk(0)` 只返回0，此处与Linux表现不同，应特殊说明。——这里的 `brk(0)` 事实上是要返回当前堆顶地址。

▼ mmap:

- 先查看测试样例：

```

/*
 * 测试成功时输出:
 * " Hello, mmap success"
 * 测试失败时输出:
 * "mmap error."
 */
static struct kstat kst;
void test_mmap(void){
    TEST_START(__func__);
    char *array;
    const char *str = " Hello, mmap successfully!";
    int fd;

    fd = open("test_mmap.txt", O_RDWR | O_CREATE);
    write(fd, str, strlen(str));
    fstat(fd, &kst);
    printf("file len: %d\n", kst.st_size);
    array = mmap(NULL, kst.st_size, PROT_WRITE | PROT
_READ, MAP_FILE | MAP_SHARED, fd, 0);
    //printf("return array: %x\n", array);

    if (array == MAP_FAILED) {
        printf("mmap error.\n");
    }else{
        printf("mmap content: %s\n", array);
        //printf("%s\n", str);

        munmap(array, kst.st_size);
    }

    close(fd);

    TEST_END(__func__);
}

```

接下来我们挨个看这个测试样例的调用情况。

▼ open:

- 检查 `open` 的调用号，我们发现 `sys_open` 事实上是文档中的 `#define SYS_openat 56`，而我们再调查一下，发现事实上我们的测试中是有 `open` 和 `openat` 这两个测试点的！所以我们还是一个一个实现一下。
- 检查 `open.c` 的测试要求：

```
void test_open() {
    TEST_START(__func__);
    // O_RDONLY = 0, O_WRONLY = 1
    int fd = open("./text.txt", 0);
    assert(fd >= 0);
    char buf[256];
    int size = read(fd, buf, 256);
    if (size < 0) {
        size = 0;
    }
    write(STDOUT, buf, size);
    close(fd);
    TEST_END(__func__);
}
```

要求就是打开一个文件名，然后 `read` 出这个文件中的内容，如果 `open` 成功，会返回给我们一个文件描述符 `fd`，以后我们就可以用 `fd` 来访问这个文件了，随后，我们用这个 `fd` 将读出来的东西写进标准输出 `STDOUT` 中即可。

- 然后我们观察一下传参情况：

```
int open(const char *path, int flags)
{
    return syscall(SYS_openat, AT_FDCWD, path,
        flags, 0_RDWR);
}
```

这里的系统调用号实际上还是 `openat` 的调用号，而我们又可以在 `sysfile.c` 里直接找到 `sys_open` 因此我们只需要在内核态继续完善 `sys_open` 就行了，传给内核态 `sys_open` 的第一个参数是 `AT_FDCWD`，经过查询，我们得知它的值是 -100，它在这个参数的含义是：请帮我打开这个 `path` 文件。如果这是一个相对路径的话，就在当前进程的工作目录下找它（因为我传的是 -100 即 `AT_FDCWD`），这个参数的全称其

实是"**At File Descriptor Current Working Directory**"。这里的后两个参数我们参考文档，也解释一下，这个会帮助我们在后面完成 `openat` 的全流程：

- `flags`（打开方式标志）
它告诉内核你当前操作文件的意图。通常由多个宏通过按位或（`|`）组合而成：

读写意图：`O_RDONLY`（只读），`O_WRONLY`（只写），`O_RDWR`（读写）。这三个必须选且只能选其一。

附加行为：
`O_CREAT`：文件如果不存在，请帮我创建它。
`O_TRUNC`：如果文件已存在，打开时顺便把里面清空（截断长度设为 0）。
`O_APPEND`：追加模式，每次写入都会强行写到文件末尾。
- 最后一个 `mode`（文件创建权限）表示文件的读写执行权限，只有在 `flags` 中包含了 `O_CREAT` 时才是有效且必须的，而这个位置的参数通常是一个八进制数（比如 `0600`，`0666`，`0777`）。

- 接下来我们开始研究原有的 `sys_open()`

```
uint64
sys_open(void)
{
    char path[FAT32_MAX_PATH];
    int fd, omode;
    struct file *f;
    struct dirent *ep;

    if (argstr(0, path, FAT32_MAX_PATH) < 0 || argint(1, &omode) < 0)
        return -1;

    // 要在物理磁盘上新建一个文件。
    // 后面有关ep的这部分其实都是在处理创建文件的事，我们就不看了。

    if (omode & O_CREATE)
    {
        ep = create(path, T_FILE, omode);
```

```

    if (ep == NULL)
    {
        return -1;
    }
}
else
{
    if ((ep = ename(path)) == NULL)    // 这一步是
    查找数据盘上该路径是否已有文件占用
    {
        return -1;
    }
    elock(ep);
    if ((ep->attribute & ATTR_DIRECTORY) && omode
    != O_RDONLY)
    {
        eunlock(ep);
        eput(ep);
        return -1;
    }
}

```

// 在一个全局系统文件表中为f分配一个file结构体，即一个打开文件句柄

// 括号中的左边如果正常执行，则左侧判断为0，右侧开始给fd分配一个文件描述符

```

    if ((f = filealloc()) == NULL || (fd = fdalloc
    (f)) < 0)
    {
        if (f)
        {
            fileclose(f);
        }
        eunlock(ep);
        eput(ep);
        return -1;
    }

```



```

// trunc相关逻辑，也不用动

    if (!(ep->attribute & ATTR_DIRECTORY) && (omode
& O_TRUNC))
    {
        etrunc(ep);
    }

// 调整f句柄的一些状态量，如权限等

    f->type = FD_ENTRY;
    f->off = (omode & O_APPEND) ? ep->file_size : 0;
    f->ep = ep;
    f->readable = !(omode & O_WRONLY);
    f->writable = (omode & O_WRONLY) || (omode & O_R
DWR);

    eunlock(ep);
// 最终返回一个文件描述符
    return fd;
}

```

- `struct file *f;`
`struct dirent *ep;`：这两个要放在一起考虑，经过查询定义，我们知道下面的 `struct dirent` 实际上代表着在物理磁盘上，这个文件的真实情况，也就是说操作系统的全局中，它的信息只对应一个文件，这是唯一的，又称为这个文件的元数据。
而 `struct file` 相当于一个句柄，关联的是进程对一个文件的打开状态和当前读写权限，这个并不唯一，如果进一步看，这里的 `f->ep`，倘若两个进程都打开了“test.txt”，则会指向同一个 `dirent` 实例。
- 这块分配 `fd` 时，会在一个 `proc.h`（进程的定义头文件）中定义好的进程打开文件表 `struct file *ofile[NOFILE];` 中查找未被占用的条目，并将这个 `f` 安置在这个位置，并返回当前这个位置的索引值作为 `fd`。
- 所以我们其实首先做的第一个事，就是改变一下内核态读取参数的位置，因为在用户态传参中，第一个参数传的是 `AT_FDCWD` 而非 `path`。此外，我们发现这里的 `omode` 实际上就是我们在用户态传的 `flags`，`omode` 就是 `open`

mode，总之我们要明白这个 `omode` 其实就是用户态指定的文件打开权限。

```
if (argstr(1, path, FAT32_MAX_PATH) < 0 || argint(2, &omode) < 0)
    return -1;
```

只需要这样小小修改，就可以通过open的测试点啦！其实还有意外之喜，如果我们留心观察一下open.c的测试样例，我们可以发现它还调用了read，再看一眼测试样例中，确实有read.c，而这个测试样例实际上又依赖open的实现，我们什么都不用改，直接把read也放在init.c的测试样例中，read就可以一并通过了。

▼ openat:

- 看一下这个脚本：

```
void test_openat(void) {
    TEST_START(__func__);
    //int fd_dir = open(".", O_RDONLY | O_CREATE);
    int fd_dir = open("./mnt", O_DIRECTORY);
    printf("open dir fd: %d\n", fd_dir);
    int fd = openat(fd_dir, "test_openat.txt", O_CREATE | O_RDWR);
    printf("openat fd: %d\n", fd);
    assert(fd > 0);
    printf("openat success.\n");
    close(fd);

    TEST_END(__func__);
}
```

观察一下，我们发现在第一次调用 `open` 时，传入的第二个参数 `flags` 是一个之前我们没见过的 `O_DIRECTORY`，这会导致我们在测试时得到这样的报错

```
===== START test_openat =====
open dir fd: -1
```

```
openat fd: -1
-- Assert Fatal ! ---
```

这是因为我们当前的 `open` 逻辑实际上是这样的，就是说如果这个 `ep` 在数据盘中只作为一个目录，并且传来的 `flags` (`omode`) 不是单纯的只读模式 (`O_RDONLY`)，就返回-1，相当于一个错误。

```
if ((ep->attribute & ATTR_DIRECTORY) && omode != O_RDONLY)
{
    eunlock(ep);
    eput(ep);
    return -1;
}
```

这就是导致测试时得到-1的原因，我们打开了一个目录，但是却传入了内核态没见过的 `O_DIRECTORY` 参数。

- 我们在用户态查找所有有关打开方式的参数

```
#define O_RDONLY 0x000
#define O_WRONLY 0x001
#define O_RDWR 0x002 // 可读可写
// #define O_CREATE 0x200
#define O_CREATE 0x40
#define O_DIRECTORY 0x0200000

#define DIR 0x040000
#define FILE 0x100000

#define AT_FDCWD -100
```

并对比在内核态下的定义：

```
#define O_RDONLY 0x000
#define O_WRONLY 0x001
#define O_RDWR 0x002
#define O_APPEND 0x004
```

```
#define O_CREATE 0x200
#define O_TRUNC 0x400
```

我们让内核态的定义都和用户态对其才行，于是修改后得到这样

```
#define O_RDONLY 0x000
#define O_WRONLY 0x001
#define O_RDWR 0x002
#define O_APPEND 0x004
#define O_CREATE 0x40 // 修改，原本为0x200
#define O_TRUNC 0x400

#define O_DIRECTORY 0x0200000
#define AT_FDCWD -100
```

- 做完这些以后，我们再修改一下刚才会产生报错的部分，让它兼容性更高，可以返回一个目录的文件描述符

```
else
{
    if ((ep = ename(path)) == NULL)
    {
        return -1;
    }
    elock(ep);

    // 【修改点 1】：如果参数要求必须是目录，但找到的实体不是目录，返回-1
    if ((omode & O_DIRECTORY) && !(ep->attribute & ATTR_DIRECTORY))
    {
        eunlock(ep);
        eput(ep);
        return -1;
    }

    // 【修改点 2】：放宽限制，只要没有要求写权限 (O_WRONLY / O_RDWR)，不强求 omode == 0，这让宏标签共存成为可能
```

```

        if ((ep->attribute & ATTR_DIRECTORY) && ((omode & 3) != 0_RDONLY))
        {
            eunlock(ep);
            eput(ep);
            return -1;
        }
    }
}

```

做到这一步我们再运行测试样例，就可以通过了。

- 但事实上，如果只是这样写的话，就完全没有考虑到之前提到的参数 `AT_FDCWD` 和 `dirfd`，而这两个参数才是我们实现的功能更强的 `openat` 的关键：即如果传入的第一个参数是代表着“**At File Descriptor Current Working Directory**”即在当前进程运行的文件目录下查找 `path` 指定的文件并打开，而如果第一个参数传进去的并非 `AT_FDCWD`，则查找 `dirfd` 下的 `path` 进行打开。

```

int open(const char *path, int flags)
{
    return syscall(SYS_openat, AT_FDCWD, path, flags, 0_RDWR);
}

int openat(int dirfd, const char *path, int flags)
{
    return syscall(SYS_openat, dirfd, path, flags, 0600);
}

```

因此我们接下来将这个 `dirfd` 参数加入函数逻辑中。

- 这时候我们仍然要把在 `brk` 中看过的 `proc` 结构体拿出来，分析一下我们该做什么。

```

// Per-process state
struct proc
{
    ...
}

```

```

    // these are private to the process, so p->lock
    need not be held.
    uint64 kstack;                // Virtual address
    ss of kernel stack
    uint64 sz;                    // Size of process
    ss memory (bytes)
    pagetable_t pagetable;        // User page table
    pagetable_t kpagetable;       // Kernel page table
    struct trapframe *trapframe; // data page for
    trampoline.S
    struct context context;        // switch() here
    to run process
    struct file *ofile[NFILE];    // Open files
    struct dirent *cwd;            // Current directory
    char name[16];                // Process name
    (debugging)
    int tmask;                    // trace mask
};

```

在这里，我们注意到有这样的 `dirent` 实例 `cwd`，它对应着目前这个进程正在运行的物理位置，即这个文件或目录在内存中缓存的物理元数据（类似Linux中的inode，在我们ics中讲到过这个）。

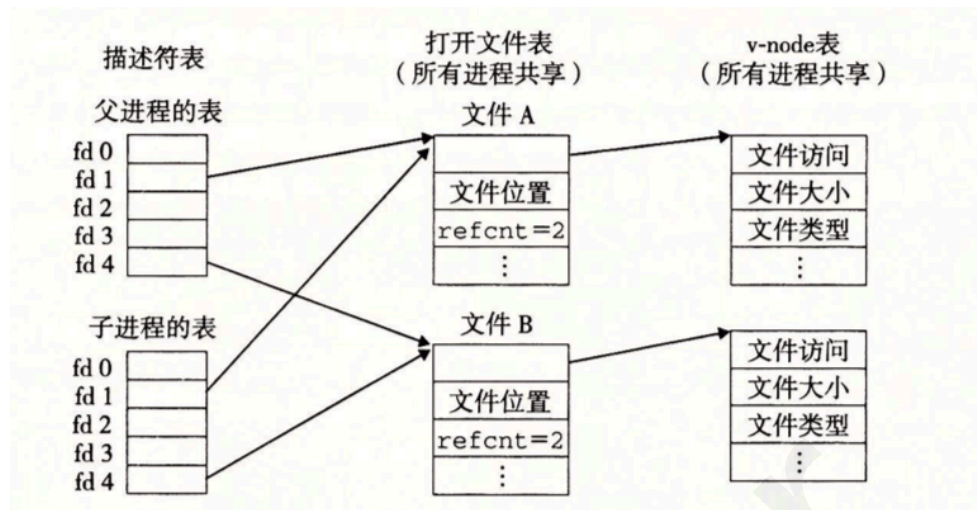


图 10-14 子进程如何继承父进程的打开文件。初始状态如图 10-12 所示

可以认为dirent就是vnode表的条目，file就是打开文件表的条目，fd就是描述符表的条目

总之，深一步理解 `struct dirent`，它就是一个目录结构体，这个结构体直接对应着真实内存的位置，而之前的 `struct file`，则是它的一个句柄，用以记录对应的 `dirent` 的真实文件在当前打开时的状态（如读写等）。

这里的 `file*` 数组，则是该进程打开文件的对应表，一个进程最多只能同时打开 `NOFILE` 个文件，也就是说最多情况下，每个条目的 `file*` 都会指向一个全局进程共用的 `file` 实例，并对应一个真实 `dirent`。

- 而在用户态，我们第一次调用 `int fd_dir = open("./mnt", O_DIRECTORY);` 时，已经将需要查找文件名的路径 `./mnt` 加载到了该进程中，如果观察返回结果，我们直到在该进程中，`./mnt` 目录的 `fd=3`（即 `stdin`、`stdout`、`stderr` 之后的第一个文件描述符）。

也就是说在这个进程的 `struct proc` 下的 `ofile[]` 中，已经分配好了一个条目，这个条目指向的是全局中 `./mnt` 的 `file` 实例，并对应着物理空间中的一个 `dirent`。这一步，是依靠这一个调用实现的

```
if ((f = filealloc()) == NULL || (fd = fdalloc(f)) < 0)
```

于是我们就可以通过 `myproc()->ofile[fd]` 这样的办法，取到 `./mnt` 这个目录对应的 `*file` 了

- 接下来，我们先 `printf` 一下，看看是不是和分析的一样：

```

uint64
sys_open(void)
{
    char path[FAT32_MAX_PATH];
    int fd, omode, dirfd;
    struct file *f;
    struct dirent *ep;

    if (argint(0, &dirfd) < 0 || argstr(1, path,
    FAT32_MAX_PATH) < 0 || argint(2, &omode) < 0)
        return -1;

    printf("sys_open: dirfd=%d, path=%s, omode=%d
    \n", dirfd, path, omode);
    ...
}

```

结果是这样，但其实fd=4对应的到底是什么，我们并不知道它是否为一个合法的文件。

```

sys_open: dirfd=-100, path=./mnt, omode=2097152
open dir fd: 3
sys_open: dirfd=3, path=test_openat.txt, omode=
66
openat fd: 4
openat success.

```

- 在原先的逻辑中，我们关注这一句，这里的 `ename` 事实上就是整个 `open` 的寻址函数关键

```

if ((ep = ename(path)) == NULL)

```

检查定义，注意到了这两个函数：

```

static struct dirent *lookup_path(char *path, i
nt parent, char *name)
{
    struct dirent *entry, *next;

```



```

// 检查是否为绝对路径
if (*path == '/') {
    entry = edup(&root);
} else if (*path != '\0') {
    entry = edup(myproc()->cwd); // 默认从
进程当前运行中的目录开始查找
} else {
    return NULL;
}
while ((path = skipelem(path, name)) != 0)
{
    elock(entry);
    if (!(entry->attribute & ATTR_DIRECTOR
Y)) {
        eunlock(entry);
        eput(entry);
        return NULL;
    }
    if (parent && *path == '\0') {
        eunlock(entry);
        return entry;
    }
    if ((next = dirlookup(entry, name, 0))
== 0) {
        eunlock(entry);
        eput(entry);
        return NULL;
    }
    eunlock(entry);
    eput(entry);
    entry = next;
}
if (parent) {
    eput(entry);
    return NULL;
}
return entry;
}

```

```

struct dirent *ename(char *path)
{
    char name[FAT32_MAX_FILENAME + 1];
    return lookup_path(path, 0, name);
}

```

这两个函数的作用，其实是把一串给人看的路径

(如"./mnt/user/src")，沿着目录树找下去，最终转化为系统操作文件
系统所需要的 内核数据结构 `struct dirent *`，而我们看到

`lookup_path` 的默认行为是先检查路径开头是否为"/"，即是否为一个从
根目录开始的绝对路径，如果不是，则从 `myproc` 的运行路径开始查找
相对路径path对应的 `dirent`。

- 所以在调用`ename(path)`之前，我们就要对path进行逻辑上的处理了，如
果在调用`open()`时，指定了`dirfd`，且传入的path本身就不是以"/"开头的绝
对路径，那么我们就调整这个path，在调用`ename`之前，将其补全为
绝对路径即可。

```

uint64
sys_open(void)
{
    char path[FAT32_MAX_PATH];
    int fd, omode, dirfd;
    struct file *f;
    struct dirent *ep;

    if (argint(0, &dirfd) < 0 || argstr(1, path, FAT
32_MAX_PATH) < 0 || argint(2, &omode) < 0)
        return -1;

    // printf("sys_open: dirfd=%d, path=%s, omode=%d
\n", dirfd, path, omode);

    if (path[0] == '/' || dirfd == AT_FDCWD)
    { // 绝对路径或者没有指定 dirfd (AT_FDCWD)，直接使用
path 进行查找
        ;
    }
}

```

```

else
{
    struct file *dirf;
    if (argfd(0, 0, &dirf) < 0 || !(dirf->ep->attribute & ATTR_DIRECTORY))
    {
        // 这一步检查 dirfd 是否有效，并且对应一个目录dirf。如果不满足条件，返回 -1。
        return -1;
    }

    struct dirent *curr = dirf->ep;
    char fullpath[FAT32_MAX_PATH];

    // 把指针 p_out 放到数组的最后面，准备从后往前写
    char *p_out = fullpath + FAT32_MAX_PATH - 1;
    *p_out = '\0'; // 字符串结尾符

    // 1. 把用户传进来的相对路径塞到最尾部
    int path_len = strlen(path);
    if (path_len >= FAT32_MAX_PATH)
        return -1;
    p_out -= path_len;
    // 这里用 memmove 只移动字符，不带 \0
    memmove(p_out, path, path_len);

    // 2. 循环向上回溯，不断把父目录名拼接到前面
    while (curr != NULL && curr->parent != curr && curr->parent != NULL)
    {
        int len = strlen(curr->filename);

        if (p_out - fullpath < len + 1)
        {
            return -1; // 路径太长
        }

        // 往前移动指针并写入 '/'

```

```

        p_out--;
        *p_out = '/';

        // 往前移动指针并写入当前目录名
        p_out -= len;
        memmove(p_out, curr->filename, len); // 同样
        只移动字符

        curr = curr->parent;
    }

    // 3. 处理根目录的 '/'
    if (p_out > fullpath)
    {
        p_out--;
        *p_out = '/';
    }
    safestrcpy(path, p_out, FAT32_MAX_PATH);
}

printf("sys_open: full path=%s, omode=%d\n", path, omode);

...
}

```

这样以来整个openat的逻辑链就完整了，这时，我们再进行检查，输出就会是这样

```

init: starting openat
===== START test_openat =====
sys_open: full path=./mnt, omode=2097152
open dir fd: 3
sys_open: full path=/mnt/test_openat.txt, omode=66
openat fd: 4
openat success.
===== END test_openat =====

```

顺手，我们把close测试点也直接加进去，一并就都通过了。

▼ fstat:

- 看看需要做什么:

```
#define AT_FDCWD (-100) //相对路径

//Stat *kst;
static struct kstat kst;
void test_fstat() {
    TEST_START(__func__);
    int fd = open("./text.txt", 0);
    int ret = fstat(fd, &kst);
    printf("fstat ret: %d\n", ret);
    assert(ret >= 0);

    printf("fstat: dev: %d, inode: %d, mode: %d, n
link: %d, size: %d, atime: %d, mtime: %d, ctime: %
d\n",
        kst.st_dev, kst.st_ino, kst.st_mode, ks
t.st_nlink, kst.st_size, kst.st_atime_sec, kst.st_
mtime_sec, kst.st_ctime_sec);

    TEST_END(__func__);
}
```

其实是要检查一下test.txt在这个进程中打开时的状态，我们看看kstat是什么结构体

```
struct kstat {
    uint64 st_dev;
    uint64 st_ino;
    mode_t st_mode;
    uint32 st_nlink;
    uint32 st_uid;
    uint32 st_gid;
    uint64 st_rdev;
    unsigned long __pad;
    off_t st_size;
```

```

uint32 st_blksize;
int __pad2;
uint64 st_blocks;
long st_atime_sec;
long st_atime_nsec;
long st_mtime_sec;
long st_mtime_nsec;
long st_ctime_sec;
long st_ctime_nsec;
unsigned __unused[2];
};

```

由于这个系统调用号在内核态的 `sysnum.h` 中已经定义过，我们可以直接查找 `sysfile.c` 中的 `sys_fstat()` 来看看这里的实现情况

- 这是一个很直观的实现，在外面的 `sys_fstat()` 只做套壳，解读 `fd` 为 `f`，并进行 `filestat` 的调用

```

uint64
sys_fstat(void)
{
    struct file *f;
    uint64 st; // user pointer to struct stat

    if (argfd(0, 0, &f) < 0 || argaddr(1, &st) <
0)
        return -1;
    return filestat(f, st);
}

```

我们直接看 `filestat()` 的实现：

```

// Get metadata about file f.
// addr is a user virtual address, pointing to
a struct stat.
int
filestat(struct file *f, uint64 addr)
{
    // struct proc *p = myproc();

```

```

struct stat st;

if(f->type == FD_ENTRY){
    elock(f->ep);
    estat(f->ep, &st);
    eunlock(f->ep);
    // if(copyout(p->pagetable, addr, (char *)&
    st, sizeof(st)) < 0)
    if(copyout2(addr, (char *)&st, sizeof(st))
    < 0)
        return -1;
    return 0;
}
return -1;
}

```

我们直接看 `stat` 这个结构体，按理说，它应该与用户态的 `struct kstat` 完全相同才行

```

struct stat {
    char name[STAT_MAX_NAME + 1];
    int dev;    // File system's disk device
    short type; // Type of file
    uint64 size; // Size of file in bytes
};

```

这才是内核态的 `stat`，那么这就完全错误了，当用户态已经在用相当完整的 `kstat`，内核态的 `sys_fstat` 还在返回一个指向小小 `stat` 的指针，这就导致我们运行这个测试样例时，结果全是0。

- 于是我们要定义一个与 `kstat` 完全一样的结构体才可以。注意不要删除原有的 `stat` 结构，因为在 `filestat()` 中，已经实现了向 `stat` 中传递参数的逻辑 `estat`，而 `stat` 中的这几个现有成员，也是 `kstat` 中有的几个成员，我们不需要额外的逻辑去调整这个 `estat` 了。

```

typedef unsigned int mode_t;
typedef long int off_t;

struct kstat

```

```

{
    uint64 st_dev;
    uint64 st_ino;
    mode_t st_mode;
    uint32 st_nlink;
    uint32 st_uid;
    uint32 st_gid;
    uint64 st_rdev;
    unsigned long __pad;
    off_t st_size;
    uint32 st_blksize;
    int __pad2;
    uint64 st_blocks;
    long st_atime_sec;
    long st_atime_nsec;
    long st_mtime_sec;
    long st_mtime_nsec;
    long st_ctime_sec;
    long st_ctime_nsec;
    unsigned __unused[2];
};

```

- 此后，我们在原有的filestat基础上，做一定的调整，使得原本的stat成员可以直接传给更大的kstat结构体

```

int
filestat(struct file *f, uint64 addr)
{
    // struct proc *p = myproc();
    struct stat st;

    if(f->type == FD_ENTRY){
        elock(f->ep);
        estat(f->ep, &st);
        eunlock(f->ep);

        struct kstat kst;
        memset(&kst, 0, sizeof(kst));
    }
}

```



```

kst.st_dev = st.dev;
kst.st_size = st.size;

// if(copyout(p->pagetable, addr, (char *)&
st, sizeof(st)) < 0)
    if(copyout2(addr, (char *)&kst, sizeof(kst)) < 0)
        return -1;
    return 0;
}
return -1;
}

```

但其他的kstat成员我们还是不知道怎么实现，这时候我们不妨看看测试样例的要求：

```

def test(self, data):
    self.assert_ge(len(data), 2)
    res = re.findall("fststat ret: (\d+)", data[0])
    if res:
        self.assert_equal(res[0], "0")
        res = re.findall(r"fststat: dev: \d+, inode: \d+, mode: (\d+), nlink: (\d+), size: \d+, atime: \d+, mtime: \d+, ctime: \d+", data[1])
        if res:
            self.assert_equal(res[0][1], "1")

```

这里虽然要求了输出中这些位置的参数都是大于等于0的整数，但最终第二条判断却只检验了 `nlink` 这一项是否=1，`nlink` 事实上就是硬链接数量

而 `nlink` 代表什么呢：

当前文件系统中，有多少个“文件名（目录项）”共同指向这一个物理实体（`dirent`）。

- 普通文件：当你新建一个普通文件（比如 `touch my.txt`）时，系统上只有一个名字指向这块数据，所以它的 `nlink` 是 1。如果你给它建了一个硬链接，`nlink` 就会变成 2。

- 空目录：假设你新建了一个空目录 `mkdir mydir`，它的 `nlink` 初始就是 2！为什么是 2？因为有两个目录名指向它：父目录里叫作 `mydir` 的那个目录项。它自己里面的 `.`（代表当前目录）这个专属隐藏项。

（如果 `mydir` 里面又建了一个子目录 `sub`，因为 `ubuntu` 也会指向上级目录，那么 `mydir` 的 `nlink` 就会变成 3，也就是多了一个来自 `sub` 的 `..`）

这样，我们就知道应该在这里根据情况不同设置初始的 `nlink`，而在这里，`stat` 还有一个成员 `type` 我们需要用，它是由 `st->type = (de-attribute & ATTR_DIRECTORY) ? T_DIR : T_FILE;` 决定的，这个标明了该文件的类型是目录还是文件，所以我们可以对齐到 `kstat` 中

```
struct kstat kst;
memset(&kst, 0, sizeof(kst));
kst.st_dev = st.dev;
kst.st_size = st.size;
if (st.type == T_DIR)
{
    kst.st_mode = DIR;
    kst.st_nlink = 2; // 目录至少有两个链接：一个来自父目录，另一个来自 . 自身
}
else if (st.type == T_FILE)
{
    kst.st_mode = FILE;
    kst.st_nlink = 1;
}
else
    kst.st_mode = 0;
```

注意这里要在 `file.c` 的开头添加 `#include "include/fcntl.h"` 以便我们使用其中定义的宏 `DIR` 和 `FILE`，这两个宏的定义，又需要在 `fcntl.h` 中进行补充，仿照用户态的定义，有：

```
#define DIR 0x040000
#define FILE 0x100000
```

- 这里我们没有太过关注完全的 `kstat` 定义，只是实现了通过测试点的部分。

▼ vma的管理

至此，我们完成了 `mmap` 的前置任务，可以开始真正实现 `mmap` 了。

而 `mmap`，实际上就是将磁盘上的一个文件（在测试样例中是 `test_mmap.txt`）直接映射到用户态的一个进程的虚拟内存空间中，这样我们在读写文件时，就不必依靠慢吞吞的读写文件函数，而是可以直接在这个进程的虚拟空间中，像读写内存空间一样读写文件的内容，随后，在 `munmap` 时，再把修改写回磁盘

- 首先我们分析一下 `mmap` 调用的参数情况

```
void *mmap(void *start, size_t len, int prot, int
flags, int fd, off_t off)
{
    return syscall(SYS_mmap, start, len, prot, fla
gs, fd, off);
}
int munmap(void *start, size_t len)
{
    return syscall(SYS_munmap, start, len);
}
```

传了6个参数，而如果我们再看一下它下面的 `munmap` 调用，我们发现在取消这个映射时，并没有传入 `fd` 参数，也就是说当用户态告知内核取消映射时，内核必须要自己记忆这个 `array` 的地址本身是归属于哪一个文件、以及何种权限才行。

而在我们目前的 `struct proc` 中，没有这样的成员，可以帮助内核将 `mmap` 传入的这些参数一一记录。所以要去创建一个这样的结构

```
// 仿照用户态的宏定义，确保内核也能看懂这些 flag
// for mmap
#define PROT_NONE 0
#define PROT_READ 1
#define PROT_WRITE 2
#define PROT_EXEC 4
#define PROT_GROWSDOWN 0X01000000
#define PROT_GROWSUP 0X02000000
```

```

#define MAP_FILE 0
#define MAP_SHARED 0x01
#define MAP_PRIVATE 0x02
#define MAP_FAILED ((void *) -1)

struct vma
{
    int valid;          // 槽位是否可用：0代表空闲，1代表正在使用
    uint64 addr;        // 分配给用户的虚拟地址起始点（账本记录的地址）
    int length;         // 映射的长度
    int prot;           // 权限（PROT_READ, PROT_WRITE）
    int flags;          // 标志（MAP_SHARED 等）
    struct file *f;     // 指向被映射的打开文件
    int offset;         // 文件的偏移量
};

// Per-process state
struct proc
{
    ...
    struct vma vmas[NVMA];    // 进程的虚拟内存区域表
};

```

这里的 `NVMA`，我们在 `param.h` 中给出它的定义 `#define NVMA 16`，为什么是这个数，还记得在之前我们看到实现 `openat` 过程中，关注到 `struct proc` 中，打开文件数的上限 `NOFILE` 其实也是16，也就是说最多最多，每个被进程打开的文件都会被 `mmap` 到一个 `vma` 上。

- 接下来，由于在 `proc` 结构体中多出来了这个条目，我们需要在现有的这些调用中，把这部分内容的初始化与回收逻辑进行补充。
 - 这一部分是在找到空闲进程时，应该同时将该进程的 `valid` 位设为0

```

// Look in the process table for an UNUSED pro
C.
// If found, initialize state required to run i

```

```

n the kernel,
// and return with p->lock held.
// If there are no free procs, or a memory allo
cation fails, return 0.
static struct proc *
allocproc(void)
{
    ...
found:
    p->pid = allocpid();
    ...

    for (int i = 0; i < NVMA; i++)
    {
        p->vmass[i].valid = 0;
    }

    p->kstack = VKSTACK;

    ...
    return p;
}

```

- 在初始化做完之后，一个进程的变化还涉及到 `fork`、`exit`、`exec` 这些调用，因此我们也需要一一补充内容

这部分是对 `fork` 的补充，我们需要调用 `filedup()` 来在子进程中增加一次文件的引用，防止父进程死掉以后把子进程的文件也收回了。

```

// Create a new process, copying the parent.
// Sets up child kernel stack to return as if f
rom fork() system call.
int fork(void)
{
    ...

    for(i = 0; i < NVMA; i++) {
        if(p->vmass[i].valid) {
            np->vmass[i] = p->vmass[i];

```

```

        if(np->vmass[i].f) {
            filedup(np->vmass[i].f);
        }
    }
}

np->state = RUNNABLE;

release(&np->lock);

return pid;
}

```

在 `exec` 的过程中，我们到 `exec.c` 中阅读原有的 `exec` 函数逻辑，就可以知道，在 `exec` 要执行一个新的程序时，首先要做的就是将旧页表 `p->kpagetable` 移到一个新的地方去，让现有的 `myproc` 的 `kpagetable` 位置空出来给新的执行程序让位置，所以对应的 `vma` 也需要被清除掉。

```

int exec(char *path, char **argv)
{
    ...
    p->trapframe->sp = sp;           // initial stack pointer

    // Clear VMAs across exec, unmap and writeback if needed
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid)
        {
            // Note: xv6 original does not write back on exec as standard POSIX expects exec to unmap
            if ((p->vmass[i].flags & MAP_SHARED) && (p->vmass[i].prot & PROT_WRITE))
            {
                filewrite(p->vmass[i].f, p->vmass[i].addr, p->vmass[i].length);
            }
            fileclose(p->vmass[i].f);
        }
    }
}

```

```

        p->vmass[i].valid = 0;
    }
}
proc_freepagetable(oldpagetable, oldsz);

...
}

```

- 在exit时，这个进程的所有内容都会被清理掉，而 `vma` 也会被清理，但要想实现懒分配，在exit清理掉 `vma` 对于文件修改的记录以前，我们就需要对这个 `vma` 记录的改动进行真实的回写操作。

什么是直接分配在直接分配中，如果使用 `mmap` 映射了一块物理内存到虚拟地址中，内核会开始使用 `uvmalloc`，这个包装好的函数则会调用若干次真实物理内存分配函数 `kalloc`，极力寻找空闲的物理内存页，映射到页表上，并将这些虚拟地址分配给这个 `mmap` 映射的文件，将这个文件中的所有内容读到些页表上去。但实际上，如果用户态需要映射一个1GB大小的文档，却只需要查看开头几KB部分的内容并修改，这种直接分配的办法就会显得很不妥当。

那什么是懒分配：而在懒分配中，`vma` 只负责记录，在第一次调用 `mmap` 时，内核不会分配内存给这个映射的文件，也不会读这个文件的内容，只是登记：这一块虚拟内存的地址空间已经被这个文件占用了。直到用户第一次对这个文件实行读写操作时，会用之前登记到的虚拟地址来访问真实的物理空间，但由于此前这个位置根本没有被分配物理空间，用虚拟地址对应的valid位根本就是0，于是触发缺页中断，但内核只会为其 `kalloc` 一页（4KB）的空间，挂到页表上然后将valid位设为1，之后程序继续运行。

- exit时，进程的 `p->state` 会变成 `ZOMBIE`，由父进程的 `wait` 等待，并执行 `freeproc` 的彻底清理函数。

```

int wait(uint64 addr, int wait_pid)
{
    ...
    if (np->state == ZOMBIE)
    {
        // Found one.
        pid = np->pid;
        int status = np->xstate << 8;
    }
}

```

```

        if (addr != 0 && copyout2(addr, (char
*)&status, sizeof(status)) < 0)
        {
            release(&np->lock);
            release(&p->lock);
            return -1;
        }
        freeproc(np);    // <-就是这个地方调用了
freeproc()
        release(&np->lock);
        release(&p->lock);
        return pid;
    }
    ...
}

```

而 `freeproc()` 会调用更底层一点的 `proc_freepagetable(p->pagetable, p->sz)`，我们查看这个函数的定义，会发现它实际上调用了2次 `vmunmap` 分别解除了跳板（`TRAMPOLINE`）和陷阱帧（`TRAPFRAME`）的映射，而查看 `uvmfree`，我们发现它实际上也是在调用 `vmunmap(pagetable, 0, PGROUNDUP(sz) / PGSIZE, 1)`；解除从0开始到堆顶sz这部分的映射后，才进行最底层的 `kfree`。这说明如果我们希望释放进程，就必须调用 `vmunmap` 将其中页表的映射关系进行解除。

```

void proc_freepagetable(pagetable_t pagetable,
uint64 sz)
{
    vmunmap(pagetable, TRAMPOLINE, 1, 0);
    vmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmfree(pagetable, sz);
}

```

- 。为了满足懒分配，在正式 `exit` 之前，进程的 `mmap` 区域（即 `vma` 账本的位置）会出现很多空洞（虽然分配了虚拟内存地址，但未分配真实物理内存页），而在现有的 `vmunmap` 函数中，如果有这样大量的缺页情况，它会一直爆出 `panic`，所以为了补充懒分配的漏洞，我们要调整一下 `vmunmap` 的逻辑。


```

// Remove npages of mappings starting from va.
va must be
// page-aligned. The mappings must exist.
// Optionally free the physical memory.
// 这个函数的作用就是检查页表中的每一个pte（页表项）是
// 否可以被
void vmunmap(pagetable_t pagetable, uint64 va,
uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if ((va % PGSIZE) != 0)
        panic("vmunmap: not aligned");

    for (a = va; a < va + npages * PGSIZE; a += P
GSIZE)
    {
        if ((pte = walk(pagetable, a, 0)) == 0)
            // panic("vmunmap: walk");
            continue; // 没有pte, 说明根本没有分配物理内
存, 直接继续下一个地址
        if ((*pte & PTE_V) == 0)
            // panic("vmunmap: not mapped");
            continue; // 没有映射, 说明根本没有分配物理内
存, 直接继续下一个地址
        if (PTE_FLAGS(*pte) == PTE_V)
            panic("vmunmap: not a leaf");
        if (do_free)
        {
            uint64 pa = PTE2PA(*pte);
            kfree((void *)pa);
        }
        *pte = 0;
    }
}

```

- 接下来，`exit` 之前，还需要把所有记录了dirty的 `vma` 进行真正物理内存写回的逻辑。`write` 其实已经被实现过了，而它的最终逻辑是调用底层的 `filewrite`

```
uint64
sys_write(void)
{
    struct file *f;
    int n;
    uint64 p;

    if (argfd(0, 0, &f) < 0 || argint(2, &n) < 0
    || argaddr(1, &p) < 0)
        return -1;

    return filewrite(f, p, n);
}
```

我们查看这个`filewrite`是否可以直接被用来做我们的`exit`前`vma`写回

```
// Write to file f.
// addr is a user virtual address.
int filewrite(struct file *f, uint64 addr, int
n)
{
    int ret = 0;
    ... // 我们在这里只用关注写回的file类型为entry（即文
件条目）时的情况
    else if (f->type == FD_ENTRY)
    {
        elock(f->ep);
        if (ewrite(f->ep, 1, addr, f->off, n) == n)
        {
            ret = n;
            f->off += n;
        }
    }
    else
    {

```

```

        ret = -1;
    }
    eunlock(f->ep);
}
else
{
    panic("filewrite");
}

return ret;
}

```

而这里的 `ewrite` 才是最终的写回核心函数，而在 `ewrite` 中，内核试图将 `addr` 开始的 `n` 大小的空间中进行写回，但也许在这些空间中存在由于懒分配导致的缺页，这就会有问题。

所以我们还必须得像 `vmunmap` 那样一页一页查询缺页情况，过滤掉未分配的物理页。我们最好自己实现一个查询与写回的函数。

我们不能调用 `vmunmap` 中那个逐页查询缺页情况的 `walk` 函数，这个函数只会返回一个 `pte` 指针，但我们需要快速获得一个虚拟地址 `va` 对应的物理地址 `pa`，用来查看这个物理页是否已经被分配出去了，所以这里我们选择使用 `walkaddr`。注意：要记得在 `proc.h` 中声明这个函数。

```

// 将 VMA 指定范围内的共有且可写数据写回文件
void vma_writeback(pagetable_t pagetable, uint64
addr, uint64 length, struct vma *v)
{
    if ((v->flags & MAP_SHARED) && (v->prot & PROT_WRITE)) // 只有共享映射且具有写权限的内容需要写回底层文件
    {
        for (uint64 va = addr; va < addr + length; va += PGSIZE) // 从vma账本上分配给用户的虚拟地址起点开始
        { // 逐页进行查询
            uint64 pa = walkaddr(pagetable, va);
            if (pa != 0) // 如果这个物理页确实存在
            {
                uint64 n = PGSIZE;
            }
        }
    }
}

```

```

        if (va + n > addr + length) // 检查这个
            地址加上新页大小是否超过了给分配出的地址
            n = (addr + length) - va;

        elock(v->f->ep);
        ewrite(v->f->ep, 1, va, v->offset + (va
- v->addr), n); // 真正的ewrite
        eunlock(v->f->ep);
    }
}
}
}

```

- 接下来我们在exit中进行这一步的修改，这里与A神的博客存在不一样的地方，在解除映射时，没有检测flags是否为SHARED，这里Gemini的回应是：

无论是 MAP_SHARED 还是 MAP_PRIVATE 或者匿名映射（匿名映射可能都没背后的文件）：只要是进程通过 mmap 分配出的物理内存页面，在它 exit (或者 exec) 的时候，都必须把物理内存释放掉 (do_free = 1)，交还给内核的空闲内存池。

MAP_SHARED 只是说，释放前这块数据需要被 writeback 写回到底层的文件而已（这步正是 vma_writeback 做的）。而 MAP_PRIVATE 的数据只是被私有修改，死的时候直接扔掉（无需写回），但底层的物理 RAM 还给内核这个事实是毫无区别的。

```

void exit(int status)
{
    struct proc *p = myproc();

    if (p == initproc)
        panic("init exiting");

    // Close all open files.
    for (int fd = 0; fd < NOFILE; fd++)
    {
        if (p->ofile[fd])
        {
            struct file *f = p->ofile[fd];

```

```

        fclose(f);
        p->ofile[fd] = 0;
    }
}

// 清理 VMA 并将共享映射的数据写回文件
for (int i = 0; i < NVMA; i++)
{
    if (p->vmass[i].valid)
    {
        vma_writeback(p->pagetable, p->vmass[i].addr, p->vmass[i].length, &p->vmass[i]);
        // 这里要记着, 把这个vma条目映射的物理空间解绑
        vmunmap(p->pagetable, p->vmass[i].addr, PG
ROUNDUP(p->vmass[i].length) / PGSIZE, 1);
        if (p->vmass[i].f)
        {
            // 如果这个vma映射的是一个文件, 就关闭它
            fclose(p->vmass[i].f);
            p->vmass[i].f = NULL;
        }
        p->vmass[i].valid = 0;    // 标记为未使用
    }
}

eput(p->cwd);
p->cwd = 0;

...
}

```

- 自此 `exit` 的逻辑我们补充完整了, 但在 `exec` 中, 我们其实还需要再做一些调整, 因为不论是 `exit` 的“完全抹除”还是 `exec` 的“夺舍转生”, 旧的内容都会完全被摧毁, 那么在 `exec` 时, 我们其实也需要将 `mmap` 的映射内容进行写回的判断。

```

int exec(char *path, char **argv)
{
    ...
    p->trapframe->sp = sp;          // initial stack
    pointer                          // Clear VMAs across
    exec, unmap and writeback if needed
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid)
        {
            // Note: xv6 original does not write back on
            exec as standard POSIX expects exec to unmap
            vma_writeback(oldpagetable, p->vmass[i].addr,
            p->vmass[i].length, &p->vmass[i]);
            vmunmap(oldpagetable, p->vmass[i].addr, PGR0U
            NDUP(p->vmass[i].length) / PGSIZE, 1);
            if (p->vmass[i].f)
            {
                fclose(p->vmass[i].f);
                p->vmass[i].f = NULL;
            }
            p->vmass[i].valid = 0;
        }
    }
    proc_freepagetable(oldpagetable, oldsz);
    ...
}

```

- 现在我们终于可以对mmap动手了

```

uint64
sys_mmap(void)
{
    uint64 addr, length, offset;
    int prot, flags, fd;

    if (argaddr(0, &addr) < 0 || argaddr(1, &length) <

```

```

0 ||
    argint(2, &prot) < 0 || argint(3, &flags) < 0 |
|
    argint(4, &fd) < 0 || argaddr(5, &offset) < 0)
{
    return -1;
}

struct proc *p = myproc();
struct file *f = NULL;

// fd描述符在范围内且进程的ofile表中存在这个文件的指针
if (fd >= 0 && fd < NOFILE && p->ofile[fd])
{
    f = p->ofile[fd];
}
else
{
    // fd < 0 (通常为 -1) 可能是匿名映射，但如果存在文件映射
    则需要合法的 fd
    if (fd != -1)
        return -1;
}

if (f)
{
    if ((prot & PROT_READ) && !f->readable) // 用户要
    求读文件但这个文件不可读
        return -1;
    if ((prot & PROT_WRITE) && (flags & MAP_SHARED) &
    & !f->writable) // 用户要求写这个文件，并且不是私有的映射，
    但文件不可写
        return -1;
}

// 找到该进程中一个可用的vma项
struct vma *v = NULL;
for (int i = 0; i < NVMA; i++)

```

```

{
    if (p->vmass[i].valid == 0)
    {
        v = &p->vmass[i];
        break;
    }
}

if (v == NULL)
    return -1;

if (addr == 0)
{
    // 向下寻找空闲的虚拟地址空间
    addr = TRAPFRAME;
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid && p->vmass[i].addr < addr)
        {
            // 这里的逻辑是：跳过之前的所有在该进程空间中已分配的v
            ma, 找到新的需要分配的vma地址的天花板
            addr = p->vmass[i].addr;
        }
    }
    // 向下分配并且对齐
    addr -= PGROUNDUP(length);
    addr &= ~(PGSIZE - 1);
}

// 检查现在选定的新的mmap块起始地址是否低于目前的堆顶地址p-
>sz
if (addr < PGROUNDUP(p->sz))
{
    return -1;
}

v->valid = 1;
v->addr = addr;
v->length = length;

```



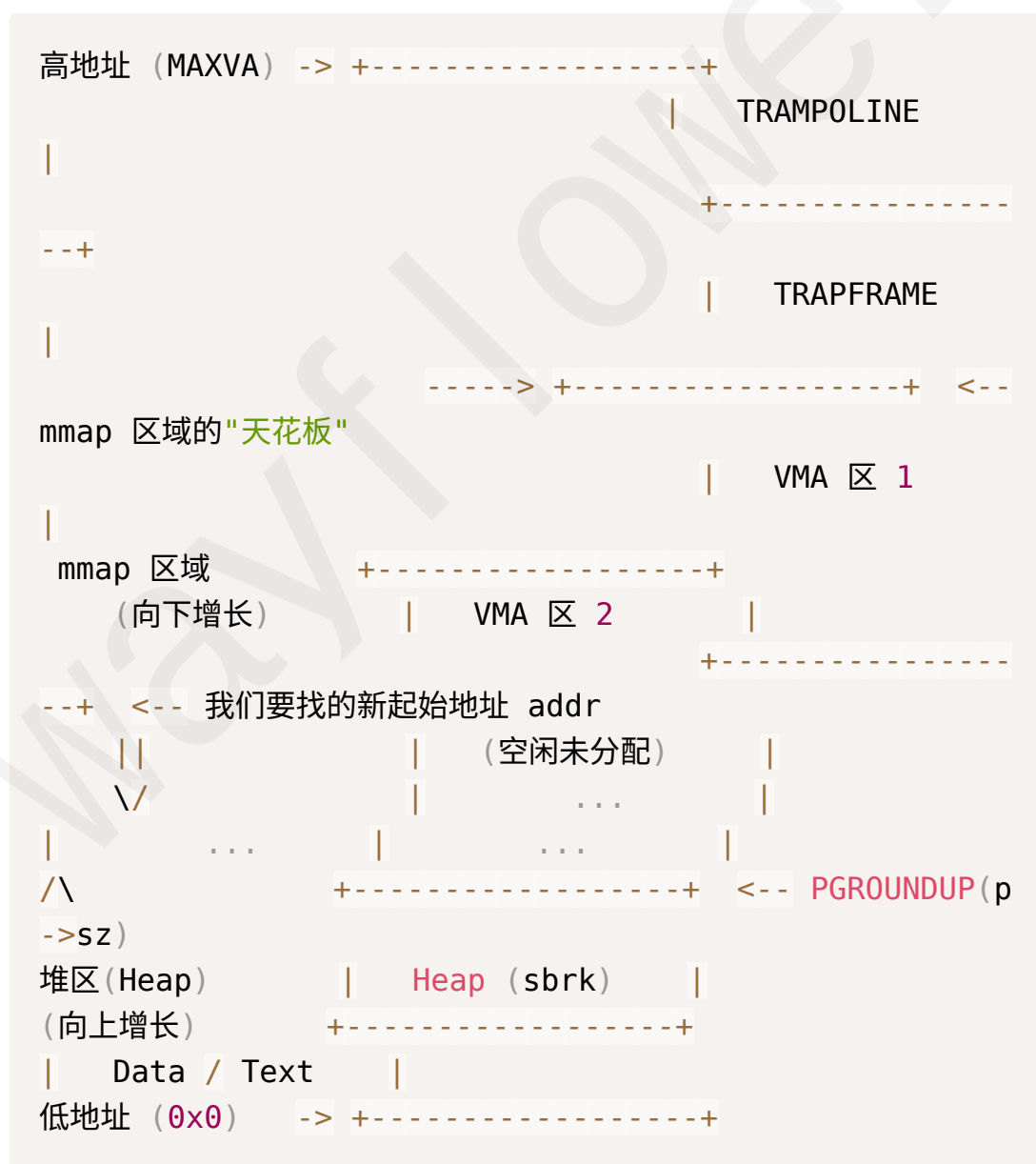
```

v->prot = prot;
v->flags = flags;
v->offset = offset;
v->f = f ? filedup(f) : NULL; // 这里一定要注意: mmap
映射的file, 其引用数必须增加

return v->addr;
}

```

在这里, 有一个宏小工具 `PGROUNDUP`, 它用来进行空间取整, 由于内存映射的最小单位是页 (4096Byte), 所以我们要对用户申请的 `length` 进行页取整。下面这个是一个进程的虚拟内存空间的简图



这样，我们就完成了 `mmap` 中对于 `vma` 这个新逻辑的注册行为。

- 如果在这一步我们就直接测试，应该会得到这样的报错。

```
===== START test_mmap =====
file len: 27
mmap content:
usertrap(): unexpected scause 0x000000000000000d pid=
31 mmap
sepc=0x00000000000001a0 stval=0x0000003fffffd000
init: starting munmap
===== START test_munmap =====
```

这是因为用户态执行到直接读取 `array` 内存这一步时，我们的 `mmap` 虽然给它注册了位置，但并没有真正把物理页给它映射过去，所以内核抛出 `scause 0xd` 缺页异常，因此我们需要正视处理一下由于 `mmap` 引发的缺页异常，让它能够分配物理页给映射，而不是直接抛出异常。

来到 `trap.c` 中，我们对 `usertrap` 这个函数进行修改

```
else if ((which_dev = devintr()) != 0)
{
    // ok
}
else if (r_scause() == 13 || r_scause() == 15 || r_
scause() == 12)
{ // Load / Store / Instruction Page Fault
    uint64 va = r_stval();
    struct vma *v = NULL;
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid && va >= p->vmass[i].addr &
& va < p->vmass[i].addr + p->vmass[i].length)
        {
            v = &p->vmass[i];
            break;
        }
    }

    if (v != NULL)
```

```

{
    uint64 va_align = PGROUNDOWN(va);
    char *mem = kalloc();
    if (mem == 0)
    {
        p->killed = 1;
    }
    else
    {
        memset(mem, 0, PGSIZE);
        if (v->f)
        {
            // 如果是从文件映射，现在把它读进来
            uint64 offset = v->offset + (va_align - v->
addr);
            uint64 n = PGSIZE;
            if (va_align + PGSIZE > v->addr + v->length)
            {
                n = v->addr + v->length - va_align;
            }

            elock(v->f->ep);
            eread(v->f->ep, 0, (uint64)mem, offset, n);
            eunlock(v->f->ep);
        }

        int prot = PTE_U | PTE_V;
        int kprot = PTE_V;
        if (v->prot & PROT_READ)
        {
            prot |= PTE_R;
            kprot |= PTE_R;
        }
        if (v->prot & PROT_WRITE)
        {
            prot |= PTE_W;
            kprot |= PTE_W;
        }
        if (v->prot & PROT_EXEC)

```

```

    {
        prot |= PTE_X;
        kprot |= PTE_X;
    }

    // 映射到用户页表
    if (mappages(p->pagetable, va_align, PGSIZE,
        (uint64)mem, prot) != 0)
    {
        kfree(mem);
        p->killed = 1;
    }
    // k210 需要同步映射到进程对应的内核页表 kpagetabl
e
    else if (mappages(p->kpagetable, va_align, PG
SIZE, (uint64)mem, kprot) != 0)
    {
        vmunmap(p->pagetable, va_align, 1, 1);
        p->killed = 1;
    }
}
}
else
{
    // 如果没找到 vma, 说明是真的越界访问, 杀死进程
    printf("\nusertrap(): page fault but vma not fo
und! va=%p\n", r_stval());
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmas[i].valid)
        {
            printf("  vma[%d]: addr=%p, length=%d\n",
i, p->vmas[i].addr, p->vmas[i].length);
        }
    }
    printf("\nusertrap(): unexpected scause %p pid
=%d %s\n", r_scause(), p->pid, p->name);
    printf("          sepc=%p stval=%p\n", r_sepc

```

```

(), r_stval());
    p->killed = 1;
}
}

```

- 但只改变这里的缺页异常处理函数，再进行测试，仍然得不到测试样例要求的
mmap content: Hello, mmap successfully!，不过我们的 usertrap 与 mmap 都已经正常实现了，这里就是打印时的问题了。这是因为原有的copy函数检测到如果地址大于堆顶，就直接拒绝copy，但我们分配的 mmap 块却远远高于这个地址，所以我们需要修改一下这几个函数的逻辑。

```

int copyout2(uint64 dstva, char *src, uint64 len)
{
    uint64 sz = myproc()->sz;
    if (dstva + len > sz || dstva >= sz)
    {
        // 检查这个拷贝的地址是否在某个有效的vma地址内
        int in_vma = 0;
        struct proc *p = myproc();
        for (int i = 0; i < NVMA; i++)
        {
            if (p->vmass[i].valid && dstva >= p->vmass[i].addr
                && dstva < p->vmass[i].addr + p->vmass[i].length)
            {
                in_vma = 1;
                break;
            }
        }
        if (!in_vma)
            return -1;
    }
    memmove((void *)dstva, src, len);
    return 0;
}

int copyin2(char *dst, uint64 srcva, uint64 len)
{
    uint64 sz = myproc()->sz;

```

```

if (srcva + len > sz || srcva >= sz)
{
    int in_vma = 0;
    struct proc *p = myproc();
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid && srcva >= p->vmass[i].addr
            && srcva < p->vmass[i].addr + p->vmass[i].length)
        {
            in_vma = 1;
            break;
        }
    }
    if (!in_vma)
        return -1;
}
memmove(dst, (void *)srcva, len);
return 0;
}

int copyinstr2(char *dst, uint64 srcva, uint64 max)
{
    int got_null = 0;
    uint64 sz = myproc()->sz;
    struct proc *proc = myproc();

    while (max > 0)
    {
        if (srcva >= sz)
        {
            int in_vma = 0;
            for (int i = 0; i < NVMA; i++)
            {
                if (proc->vmass[i].valid && srcva >= proc->vmass[i].addr
                    && srcva < proc->vmass[i].addr + proc->vmass[i].length)
                {
                    in_vma = 1;

```

```

        break;
    }
}
if (!in_vma)
    break;
}

char *p = (char *)srcva;
if (*p == '\0')
{
    *dst = '\0';
    got_null = 1;
    break;
}
else
{
    *dst = *p;
}
--max;
srcva++;
dst++;
}
if (got_null)
{
    return 0;
}
else
{
    return -1;
}
}

```

这样，如果这个拷贝的地址确实在某个有效的 `vma` 地址内，那就可以正常进行拷贝了。

- 自此，mmap的工作告一段落。

▼ munmap:

- 看看需要做什么：

```

void test_munmap(void){
    TEST_START(__func__);
    char *array;
    const char *str = " Hello, mmap successfully!";
    int fd;

    fd = open("test_mmap.txt", O_RDWR | O_CREATE);
    write(fd, str, strlen(str));
    fstat(fd, &kst);
    printf("file len: %d\n", kst.st_size);
    array = mmap(NULL, kst.st_size, PROT_WRITE | PROT
_READ, MAP_FILE | MAP_SHARED, fd, 0);
    //printf("return array: %x\n", array);

    if (array == MAP_FAILED) {
        printf("mmap error.\n");
    }else{
        //printf("mmap content: %s\n", array);

        int ret = munmap(array, kst.st_size);
        printf("munmap return: %d\n",ret);
        assert(ret == 0);

        if (ret == 0)
            printf("munmap successfully!\n");
        }
        close(fd);

    TEST_END(__func__);
}

```

`munmap` 负责将 `mmap` 的映射全部解除，并且在解除之前，看看是否要写回物理空间。其实我们在这里不需要过多关注测试样例，只要按照 `mmap` 的逻辑，将 `munmap` 实现回收的功能即可。这里就不再过多分析了。

```

uint64
sys_munmap(void)
{

```



```

uint64 addr, length;
if (argaddr(0, &addr) < 0 || argaddr(1, &length) <
0)
{
    return -1;
}

struct proc *p = myproc();
struct vma *v = NULL;

for (int i = 0; i < NVMA; i++)
{
    // 允许部分或完整 unmap, 只要 addr 匹配就行
    if (p->vmas[i].valid && addr >= p->vmas[i].addr &
& addr < p->vmas[i].addr + p->vmas[i].length)
    {
        v = &p->vmas[i];
        break;
    }
}

if (v == NULL)
    return -1;

if (v->flags & MAP_SHARED)
{
    vma_writeback(p->pagetable, addr, length, v);
}

// 这里调用 vmunmap 会将我们之前按需分配好的物理页面彻底还
给内核
vmunmap(p->pagetable, addr, PGROUNDUP(length) / PGS
IZE, 1);

if (addr == v->addr && length == v->length)
{
    // 全退还
    if (v->f)

```

```
{
    fclose(v->f);
    v->f = NULL;
}
v->valid = 0;
}
else
{
    // 处理 OSComp 中可能出现的部分截断 munmap (例如头部的
    partial unmap)
    // 根据需要做更精确的设计, 但最简单的是整体释放, 这里只做
    简单的偏移
    v->addr = addr + length;
    v->length -= length;
    v->offset += length;
}

return 0;
}
```

lab4

要求：获取助教提供的新的xv6-k210-template；完成进程调度算法 `test_proc_rr`
`test_proc_priority` `test_proc_mlfq`

▼ 前置操作（切换新的仓库）

- 按照助教给的教程，从最下面的“系统性讲解（第一次必看！）”的第一个视频看下去就很直观了。
- 或者我这里也有一些视频抄送版本：
 1. 退出你在lab1-3的仓库，目前我的位置是 `ubuntu@os-lab:~`，直接在这个目录下运行指令，将template克隆下来。然后别进到这个目录里，我修改了文件名为 `os-part4`。

```
git clone https://gitlab.eduxiji.net/pku230121066
6/xv6-k210-template.git
```

2. 进入目录并执行一个fetch

```
cd os-part4

git fetch origin
```

视频里还有一个指令 `git checkout part4-scheduler`，它的作用主要是切换当前我们编辑的代码为part4-scheduler这个分支的，这个在后面通过不同测试样例时是需要的。

3. 接下来，在我们目前的part4-scheduler分支下，运行这个代码，进入到xv6中，如果观察os-part4的文件中，`Makefile`的最下面多了一个 `make run_test`，这个指令就是在本地运行测试样例的指令。如果添加一个参数，变成 `make run_test SCHEDULER_TYPE=PRIORITY`，这样就可以挨个样例测试了

```
docker run -ti --rm -v ./:/xv6 -w /xv6 --privilege
d=true docker.educg.net/cg/os-contest:2024p6 /bin/
bash
make run_test
```

这里我在 `.bashrc` 中添加了一个简化的指令，它可以直接定位到lab4所在的位置并且进入xv6，保存以后删除当前的powershell再重新开一个，直接输入 `inpart4` 就能进入xv6了

```
alias inpart4='cd os-part4 && docker run -ti --rm
-v ./:/xv6 -w /xv6 --privileged=true docker.educg.
net/cg/os-contest:2024p6 /bin/bash'
```

4. 助教说这个part4建议我们用不同的branch来保护，先观察上一步测试样例的反馈

```
Starting test program: test_proc_rr
Scheduler type: Round Robin
init: starting test_proc_rr
pid 3 test_proc_rr: unknown sys call 54322
pid 4 test_proc_rr: unknown sys call 54322
pid 5 test_proc_rr: unknown sys call 54322
testing output size:152, contents:
Testing RR Scheduler - Basic
RR Scheduler Process 3 completed
RR Scheduler Process 2 completed
RR Scheduler Process 1 completed
RR Basic Test Completed
init: process pid=2 exited
init: test execution completed, starting judger
Judger: Starting evaluation
Test1 output:
Testing RR Scheduler - Basic
RR Scheduler Process 3 completed
RR Scheduler Process 2 completed
RR Scheduler Process 1 completed
RR Basic Test Completed
Expected order: 3 2 1 0 0
Test1 PASSED
SCORE: 1
init: judger completed
make[1]: Leaving directory '/xv6'
```

可以看到Test1失败了，因为有一个未知的系统调用54322，接下来的步骤实际上和之前lab做的很像，我们查看 `sysnum.h` 中的定义，有 `#define SYS_set_timeslice 54322`，但 `syscall.c` 中没有对其进行注册，所以注册以后，我们给一个简单的占位函数，把它放在 `syscall.c` 下面（注意这里你打开的文件应该来自于 **os-lab4**），这里的 `timeslice` 也需要在进程定义中声明

```
uint64
sys_set_timeslice(void)
{
    int timeslice;
    struct proc *p = myproc();

    if (argint(0, &timeslice) < 0)
    {
        return -1;
    }

    if (timeslice < 0)
    {
        return -1;
    }

    p->timeslice = timeslice;

    return 0;
}

// Per-process state
struct proc
{
    ...
    int tmask;                // trace mask
    int timeslice;
};
```

5. 再 `make run_test` 就可以看到虽然说没通过测试点，但已经没有 `unknown syscall` 了。

6. 接下来，为了这个rr的测试点，建立一个branch，如果运行下面这个指令，现在你应该在新建好的这个branch上了。这个指令的意思是在当前这个branch的基础下新建一个分支并切换过去，所以如果我们还要建part4-priority分支，那就切换回一开始的part4-scheduler分支再建立就行。

```
git checkout -b part4-rr
```

这时候你可以运行 `git branch`，可以看到在刚刚创建的branch前面有一个*

7. 然后去希冀平台的gitlab，我们建一个新的仓库，并选择public权限，去掉README这一项，建好以后，复制下面这个代码，最后的url部分可以从gitlab的仓库中，点击clone按钮直接复制，应以https开头

```
git remote set-url origin https://gitlab.eduxiji.net/pkuxxxxxxxxxx/os-part4.git
git remote -v
```

然后你运行 `git remote -v` 就可以看到现在已经切换到了你自己的仓库了

8. 运行下面的代码，将这些改动push到你的仓库中的part4-rr分支，接下来，之后的push就只需要git push，而不用设置这条的origin了；如果在之后的提交中，希望切换到另一个分支，并push到另一个分支上，记得还是需要在第一次push时设置一下。

```
git add .
git commit -m "your comment"
git push -u origin part4-test
```

9. 提交的时候需要在校园网下进入这个网页<http://10.129.81.45:8080/>，在这次的lab中选择part4，填写好要交的分支名字与gitlab的网址，提交就行了。想查看提交记录，就进入查询页面，选择“用户（uid）”，输入自己的学号就能查得到。

▼ test_proc_rr

- 看一下测试样例：

```
#define ITERATIONS 1000000
#define LOOP 100
```

```

void task(int id) {
    volatile long long count = 0;
    for(int loop = 0; loop < LOOP; loop++) {
        for(int i = 0; i < ITERATIONS; i++) {
            count += (long long)i * (long long) i;
            if(i % (ITERATIONS/2) == 0) {
                // printf("RR Scheduler Process %d: i
iteration %d\n", id, i);
            }
        }
    }

    char buffer[50];
    int pos = 0;
    const char* parts[] = {"RR Scheduler Process ",
"0", " completed\n"};
    for (int i = 0; parts[0][i] != '\0'; i++) {
        buffer[pos++] = parts[0][i];
    }
    buffer[pos++] = '0' + id;
    for (int i = 0; parts[2][i] != '\0'; i++) {
        buffer[pos++] = parts[2][i];
    }

    write(1, buffer, pos);
}

/*
 * Desc:
 * We fork three processes, and set timeslice 1 to P1,
2 to P2, 3 to P3.
 * The three processes just do the same work, and will
last a long enough time
 * to encounter several timer interrupts.
 *
 * Expected:

```

```
* P3 will finish the job first, then P2, and P1 is the last.  
* The judge program will check the appearance and order of `Process {\d} completed`  
* to make sure you have implemented the Round-Robin algorithm with given timeslice.  
*/
```

```
int main() {  
    printf("Testing RR Scheduler - Basic\n");  
  
    int pid1, pid2, pid3;  
    if((pid1=fork())==0) {  
        set_timeslice(1);  
        task(1);  
        exit(0);  
    }  
    if((pid2=fork())==0) {  
        set_timeslice(2);  
        task(2);  
        exit(0);  
    }  
    if((pid3=fork())==0) {  
        set_timeslice(3);  
        task(3);  
        exit(0);  
    }  
    wait(0);  
    wait(0);  
    wait(0);  
  
    printf("RR Basic Test Completed\n");  
    exit(0);  
}
```

三个进程都进行了同一个任务，但是三个进程分配到的时间片不一样大，其中进程3的时间片分配最多，理论上来说应该是task3 → task2 → task1的顺序。

但如果只是在内核中把 `sys_set_timeslice()` 实现了，现在的顺序仍然是123的初始fork顺序。

```
init: test execution completed, starting judger
Judger: Starting evaluation
Test1 output:
Testing RR Scheduler - Basic
RR Scheduler Process 1 completed
RR Scheduler Process 2 completed
RR Scheduler Process 3 completed
RR Basic Test Completed
Expected order: 3 2 1 0 0
Test1 FAILED
SCORE: 0
```

- 在kernel/main.c中，我们得到了CPU 进行完了所有的初始化工作（页表、陷入机制、并发锁、设备、磁盘等）后，最后一行代码就是调用scheduler()。

```
// Per-CPU process scheduler.
// Each CPU calls scheduler() after setting itself up.
// Scheduler never returns. It loops, doing:
//  - choose a process to run.
//  - swtch to start running that process.
//  - eventually that process transfers control
//    via swtch back to the scheduler.
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();           // 获取当前
    执行这段代码的 CPU 结构体
    extern pagetable_t kernel_pagetable; // 声明全局
    的内核页表

    c->proc = 0;                        // 初始化，
    当前 CPU 没有运行任何进程
    for (;;)                             // 经典的无
    限循环：调度器永远在工作
```

```

{
    // Avoid deadlock by ensuring that devices can in
    // interrupt.
    intr_on(); // 开启中
    断。非常重要！如果当前没有进程可运行，必须允许响应时钟/设备中
    断，否则系统就死锁了。

    int found = 0; // 标记这一
    轮遍历是否找到了可运行的进程
    for (p = proc; p < &proc[NPROC]; p++) // 遍历整个
    进程表（默认为 64 个槽位）
    {
        acquire(&p->lock); // 在读取/修
        改进程状态前，必须加锁，防止多核资源竞争

        if (p->state == RUNNABLE) // 【关键】
        看看这个进程是不是“准备好运行”了？
        {
            // 找到了一个可运行的进程！
            p->state = RUNNING; // 把状态改
            为“正在运行”
            c->proc = p; // 登记：当
            前 CPU 正在运行进程 p

            // 切换到该进程专属的内核页表
            w_satp(MAKE_SATP(p->kpagetable));
            sfence_vma(); // 刷新 TLB
            缓存，确保新页表生效

            // 【核心动作：上下文切换】
            // 保存当前 CPU 的调度器上下文到 c->context，
            // 加载进程 p 的上下文 (p->context)，然后直接跳转到
            p 上次暂停的地方！
            switch(&c->context, &p->context);

            // =====
            // 【注意！】代码执行到上面那行 swtch 时，就跳出去
            了！这里的进程已经开始运行了。

```

```

        // 等到未来的某个时刻，该进程调用了 yield() 或 sleep() 放弃 CPU，
        // 会发生反向的 switch()，代码才会重新回到下面这一行继续执行！
        // =====

        // 进程运行结束或挂起，回到了调度器：切换回系统全局的内核页表
        w_satp(MAKE_SATP(kernel_pagetable));
        sfence_vma();

        c->proc = 0; // 登记：当前 CPU 又变空闲了
        found = 1; // 标记：这轮遍历我们至少切过去执行了一个进程
    }
    release(&p->lock); // 释放进程锁，让别的 CPU 也可以碰这个进程
}

// 如果遍历了一整圈进程表，一个准备好的进程都没找到（都在 wait/sleep 甚至全死光了）
if (found == 0)
{
    intr_on(); // 再次确保中断打开
    asm volatile("wfi"); // Wait For Interrupt: 让 CPU 进入低功耗休眠，直到下一个硬件中断（比如时钟滴答声）把它叫醒，再接着循环找进程。
}
}
}
}

```

每个 `cpu` 初始化以后都会永不返回地进入 `scheduler` 的死循环，不断寻找可以运行的进程，其中 `intr_on()` 会允许内核接受中断，防止死锁，这一步的原因可以这样解释：

▼ `intr_on()` 与死锁：

1. 灾难场景假设（如果没有 `intr_on()`）
假设系统中有 2 个进程 P1 和 P2 正在运行（假设我们只有 1 个 CPU）：
2. P1 想读取磁盘文件：P1 调用系统调用，陷入内核。在内核里它请求读取磁盘。因为磁盘很慢，P1 需要等待磁盘硬件读完数据，于是它调用 `sleep()` 把自己的状态设为 `SLEEPING`，并且让出 CPU（这会去调用 `sched()` 然后 `swtch()` 跳回 `scheduler()` 的 `for(;;)` 循环）。
P2 也想读磁盘：调度器收回 CPU 后，循环发现了 P2（它是 `RUNNABLE` 的），于是 `swtch()` 给 P2。P2 运行了一会儿，也想读磁盘，同样它也调用 `sleep()`，变成了 `SLEEPING` 状态，并让出 CPU 回到了调度器。
现在，可怕的情况出现了：整个系统里所有的进程（P1 和 P2）都处于 `SLEEPING` 状态！没有一个是 `RUNNABLE` 的！
3. 在 `scheduler` 遍历进程表时，它会把当前正接受检查的进程上锁（`acquire(&p->lock);`），但是现在没有允许中断，所以每个进程都是 `SLEEPING`，如果这个时候磁盘硬件读完数据，试图发送一个硬件中断告诉 cpu 可以唤醒 P1
4. 但是由于 P1 在之前的检查中上锁了，这个中断没办法对 P1 的状态进行修改，`scheduler` 不停地 `acquire` 和 `release` 进程锁，很多情况下 `context` 切换回来的那一瞬间这个锁可能都是关闭的，这样 P1 和 P2 就会陷入互相等待。cpu 的中断处理函数一直被屏蔽，就会陷入死锁。

找到一个 `RUNNABLE` 的进程以后，调度器将这个进程的状态改为 `RUNNING`，cpu 的当前运行进程改成这个进程 p，刷新为 p 的内核页表。此后调度器会运行 `swtch` 这个函数，它对应的是一个 `swtch.S` 汇编码，其主要的作用就是交换两个 `context`，完成一个偷天换日的过程：

▼ `swtch` 在调度过程中：

```
# Context switch
#
# void swtch(struct context *old, struct context
# *new);
#
# Save current registers in old. Load from new.

.globl swtch
```

```

swtch:
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    sd s1, 24(a0)
    sd s2, 32(a0)
    sd s3, 40(a0)
    sd s4, 48(a0)
    sd s5, 56(a0)
    sd s6, 64(a0)
    sd s7, 72(a0)
    sd s8, 80(a0)
    sd s9, 88(a0)
    sd s10, 96(a0)
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    ld s1, 24(a1)
    ld s2, 32(a1)
    ld s3, 40(a1)
    ld s4, 48(a1)
    ld s5, 56(a1)
    ld s6, 64(a1)
    ld s7, 72(a1)
    ld s8, 80(a1)
    ld s9, 88(a1)
    ld s10, 96(a1)
    ld s11, 104(a1)

    ret

```

在调用 `swtch(&c->context, &p->context)` 时，这个汇编会将传入的第一个参数（即当前 `cpu` 正在使用的上下文）与 `cpu` 刚设置好的新的进程 `p` 的 `context` 进行交换，这样，现在 `cpu` 就开始使用调度器调度来的 `p` 的上下文了。

- 在ics中学习的x86框架中，执行 `call` 指令后，CPU会在硬件层面上将下一条指令的地址压栈，然后再跳转到目标函数，执行 `ret` 指令时，CPU则自动从栈顶弹出该指令并跳转。
- 在xv6的RISCV框架中，并没有这种自动压栈的硬件操作，执行函数调用时，CPU会把下一条指令的地址存入一个专门的寄存器`ra`，然后执行跳转。而在执行 `ret` 指令时，实际上 `cpu` 做了这样一件事： `jalr x0,0(ra)`，也就是简单地跳转到了`ra`存的地址。

所以在**p1**与**p2**调度的过程中，我们看到：

1. scheduler第一次调度，发现p2是RUNNABLE的，此时调用 `swtch(&c->context, &p->context);` 会把当前 `cpu` 的物理寄存器`ra`的内容存到`a0`（即第一个参数 `oldcontext`）的起始位置 `ra` 变量中，然后存其他的上下文。这时，存的`ra`实际上是内核的下一步，也就是 `scheduler` 函数中，调用 `swtch` 之后的下一步。
2. 完成这种“偷天换日”的技巧：把`a1`（即被调度上来，新换入cpu中的 **p2**的 `context`）的起始位置加载到了 `cpu` 的真正物理寄存器`ra`中，改变了 `scheduler` 函数中调用 `swtch` 函数之后本应该的返回指令，因此接下来 `ret` 的时候，就直接进入了另一个进程**p2**的运行中。现在我们记清楚：此时 `cpu->context` 中的 `ra` 变量，存的指令是 `scheduler` 函数中，调用完 `swtch` 的下一条指令。
3. 接下来，当这个进程**p2**的时间片结束了或是直接 `yield` 出去，交由内核进行调度时，会有这个过程：

```
// Give up the CPU for one scheduling round.
void yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}

// Switch to scheduler. Must hold only p->lock
// and have changed proc->state. Saves and restores
// intena because intena is a property of this
```

```

// kernel thread, not this CPU. It should
// be proc->intena and proc->noff, but that would
// break in the few places where a lock is held
// but
// there's no process.
void sched(void)
{
    int intena;
    struct proc *p = myproc();

    if (!holding(&p->lock))
        panic("sched p->lock");
    if (mycpu()->noff != 1)
        panic("sched locks");
    if (p->state == RUNNING)
        panic("sched running");
    if (intr_get())
        panic("sched interruptible");

    intena = mycpu()->intena;
    swtch(&p->context, &mycpu()->context);
    mycpu()->intena = intena;
}

```

`yield` 完成 **p2** 锁的持有以及状态的改变，`sched` 负责检查工作并设置 `intena`，又回去调用 `swtch` 了。

4. 这时候，`swtch.S` 又进行一次调换，我们注意这里的传参顺序是这样的：`swtch(&p->context, &mycpu()->context);`，现在，**p2** 进程正在使用的物理寄存器 `ra` 内容被保存在它自己的 `context` 中的 `ra` 变量，然后当前 `cpu` 的 `context` 里之前在 `scheduler` 中调用并保存好的 `ra`，就换入了真实的物理寄存器 `ra`，那么下一步，就回到了我们在 `scheduler` 中，第一次调用 `swtch` 函数的下一步了。
5. 下一步，内核又把当前的页表指针，从进程专属的内核页表又换回了全局的内核页表了。自此，一次调度就结束了，然后 `scheduler` 因为死循环，进入下一次调度，在这次调度中，调度器发现 **p1** 是 `RUNNABLE` 的，那么 **p1** 就会从头执行一次这个过程。

- 接下来，我们看看这个初始的 `scheduler` 在做什么，它实际上就是很忠实地按进程创建的顺序来遍历进程表，找到一个 `RUNNABLE` 的就换上来了。但我们需要时间片长的进程3最先完成，这时候，我们来看一下除了进程自己调用 `yield`、`sleep` 以外，内核自己决定进行调度的过程：

```
void
usertrap(void)
{
    // printf("run in usertrap\n");
    int which_dev = 0;

    ...

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2)
        yield();

    usertrapret();
}

void
kerneltrap() {
    int which_dev = 0;

    ...

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2 && myproc() != 0 && myproc()->state == RUNNING) {
        yield();
    }

    // the yield() may have caused some traps to occur,
    // so restore trap registers for use by kernelvec.
    S's sepc instruction.
    w_sepc(sepc);
    w_sstatus(sstatus);
}
```


在内核态处理中断，以及用户态处理中断时，这个 `which_dev == 2` 的判断，就是在看这次中断是否来自时钟引发的中断，如果这个中断的类型满足，那么内核调用 `yield`，完成一次调度。

- 受到目前助教在测试样例中的 `set_timeslice()` 启发，在现有的基础上，我们只需要对 `usertrap` 和 `kerneltrap` 的逻辑做一点调整就好。当前的逻辑中，所有进程的 `timeslice` 都被当成了1，如果一开始设置好这个进程的 `timeslice`，那就应该在每次时钟中断时，检查现有的时间片用了多少，还剩多少，因此在proc中再添加一个成员 `int ticks;`。

严谨一点，我们要在一个进程的全周期中，都添加有关这个变量的维护，包括 `allocproc` `fork` `freeproc`。

```
static struct proc *
allocproc(void)
{
    ...
    p->context.sp = p->kstack + PGSIZE;
    p->ticks = 0;
    p->timeslice = 0;

    return p;
}

int fork(void)
{
    ...
    np->timeslice = p->timeslice;
    np->ticks = 0;

    np->state = RUNNABLE;
    ...
}

static void
freeproc(struct proc *p)
{
    ...
    p->xstate = 0;
    p->ticks = 0;
```

```

    p->timeslice = 0;
    p->state = UNUSED;
}

```

接下来，在usertrap中，添加这个逻辑。

```

void usertrap(void)
{
    ...
    // give up the CPU if this is a timer interrupt.
    if (which_dev == 2)
    {
        if (p->timeslice > 0)
        {
            p->ticks++;
            if (p->ticks >= p->timeslice)
            {
                p->ticks = 0;
                yield();
            }
        }
        else
        {
            yield();
        }
        yield();
    }
    usertrapret();
}

void usertrapret(void)
{
    ...
    if (which_dev == 2 && myproc() != 0 && myproc()->state == RUNNING)
    {
        if (myproc()->timeslice > 0)
        {

```

```

    myproc()->ticks++;
    if (myproc()->ticks >= myproc()->timeslice)
    {
        myproc()->ticks = 0;
        yield();
    }
}
else
    yield();
}
// the yield() may have caused some traps to occur,
// so restore trap registers for use by kernelvec.
S's sepc instruction.
w_sepc(sepc);
w_sstatus(sstatus);
}

```

你还要记得，如果在这次准备yield这个进程，要把它ticks数置为0。

这样就得到了这个样例的1分。

▼ test_proc_priority

- 记着运行 `git checkout part4-priority` 到你建的另一个分支上。现在你再看这些文档，就跟上一个测试样例完全没关系了，你做的修改全部在之前的分支上。这个分支是从 `part4-priority` 建立的。
- 运行这个，以只测试该节点

```
make run_test SCHEDULER_TYPE=PRIORITY
```

- 看一下要求：

```

/*
 * Desc:
 * We fork three processes, and set priority 10 to P1,
 * 20 to P2, 30 to P3.
 * The three processes just do the same work, and will
 * last a long enough time
 * to encounter timer interrupt and yield.
 *

```

```
* Expected:  
* P1(with the highest priority) will finish the job f  
irst, then P2, and P3 is the last.  
* The judge program will check the appearance and ord  
er of `Process {\d} completed`  
* to make sure you have implemented the priority algo  
rithm with different priorities.  
*/
```

```
int main() {  
    printf("Testing Priority Scheduler - Basic\n");  
  
    int pid1, pid2, pid3;  
    if((pid1=fork())==0) {  
        set_priority(10);    // highest one  
        task(1);  
        exit(0);  
    }  
    if((pid2=fork())==0) {  
        set_priority(20);  
        task(2);  
        exit(0);  
    }  
    if((pid3=fork())==0) {  
        set_priority(30);  
        task(3);  
        exit(0);  
    }  
    wait(0);  
    wait(0);  
    wait(0);  
  
    printf("Priority Basic Test Completed\n");  
    exit(0);  
}
```

依然是三个进程执行一样的任务，在这里根据优先级不同，`priority` 越小优先级越高，按此顺序完成执行即可。

- 还是先注册，在 `proc.h` 里给 `proc` 添加一个 `priority` 的成员，并实现 `sys_setpriority`，再严谨一点，除了管理 `priority` 的全周期，我们在 `fork` 中，也让子进程的优先级设置为和父进程一样的。

```
uint64
sys_set_priority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
    p->priority = priority;
    return 0;
}
```

```
struct proc
{
    ...
    int tmask;                // trace mask
    int priority;             // process priority for
    priority scheduler
};
```

```
int fork(void)
{
    ...
    np->priority = p->priority; // set priority of the
    child same as parent

    np->state = RUNNABLE;
    ...
}
```

- 如果我们直接运行 `make run_test SCHEDULER_TYPE=PRIORITY`，这样大概率你会得到一个比较随机的结束情况，但如果试的次数足够多，应该有概率出现1/2/3的顺序。
- 在这个样例中，我们实际上不用对trap函数做什么调整，我们在 `scheduler` 中调整一下寻找下一个进程的逻辑即可。

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    extern pagetable_t kernel_pagetable;

    c->proc = 0;
    for (;;)
    {
        // Avoid deadlock by ensuring that devices can in
        // terrupt.
        intr_on();

        int found = 0;
        int highest_pri = 1000000;

        for (p = proc; p < &proc[NPROC]; p++)
        {
            acquire(&p->lock);
            if (p->state == RUNNABLE && p->priority < highest_pri)
            {
                highest_pri = p->priority;
            }
            release(&p->lock);
        }

        for (p = proc; p < &proc[NPROC]; p++)
        {
            acquire(&p->lock);
            if (p->state == RUNNABLE && p->priority == highest_pri)
```

```

    {
        // Switch to chosen process. It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        // printf("[scheduler]found runnable proc with pid: %d\n", p->pid);
        p->state = RUNNING;
        c->proc = p;
        w_satp(MAKE_SATP(p->kpagetable));
        sfence_vma();
        swtch(&c->context, &p->context);
        w_satp(MAKE_SATP(kernel_pagetable));
        sfence_vma();
        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;

        found = 1;
    }
    if (p->state == RUNNABLE)
    {
        found = 1; // 如果有一个RUNNABLE的进程，就不让cpu
        // 休息，虽然不能执行，但还要重新跑一次比较的循环
        // 除非系统真的没有进程了，这时就让CPU休息一下，等中断来唤醒它
    }
    release(&p->lock);
}
if (found == 0)
{
    intr_on();
    asm volatile("wfi");
}
}
}

```

这里会有个问题，比方说极限一点，在两个fork之间突然产生了一个新的优先级更高的进程，那么这个逻辑就会忽略掉这个进程而去找之前那个确定好的优先级的进程了。此外，这里没有选择pid作为筛选逻辑，因为当两个进程的优先级相同时，如果用pid筛选，就会出现另一个进程饿死的情况。Gemini给出的解释是：如果要彻底考虑这种情况，就需要建立链表，在每次调度时给一个大锁，然后挑出来优先级最高的并运行，再释放这个锁。不过即使遇到我们之前那个问题，新建的进程也只是错过了一次调度，并不会造成很大的损失。所以就先这么办吧。

▼ test_proc_mlfq

- 阅读助教的文档，我们发现这个任务完全不用真的实现一个什么队列，我们甚至可以“不使用队列，而是在上一个测试样例的实现基础上继续实现优先级变化的统计逻辑即可。”
- 直接在part4-priority为基础创建一个新的branch。
- 运行这个，以只测试该节点

```
make run_test SCHEDULER_TYPE=MLFQ
```

- 会直接出现这个字样，我们去实现一下要求的 `#define SYS_get_priority 54324`

```
Starting test program: test_proc_mlfq
Scheduler type: MLFQ
init: starting test_proc_mlf
pid 7 test_proc_mlfq: unknown sys call 54324
pid 4 test_proc_mlfq: unknown sys call 54324
pid 6 test_proc_mlfq: unknown sys call 54324
```

```
uint64
sys_set_priority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
```



```

    p->priority = priority;
    return 0;
}

uint64
sys_get_priority(void)
{
    struct proc *p = myproc();
    return p->priority;
}

```

再看看运行结果：

```

init: starting test_proc_mlfq
testing output size:469, contents:
Testing MLFQ Scheduler - Basic
MLFQ Scheduler Process 5 with initial priority 3 and
final priority 3 completed
MLFQ Scheduler Process 2 with initial priority 1 and
final priority 1 completed
MLFQ Scheduler Process 4 with initial priority 5 and
final priority 5 completed
MLFQ Scheduler Process 3 with initial priority 2 and
final priority 2 completed
MLFQ Scheduler Process 1 with initial priority 10 and
final priority 10 completed
MLFQ with Priorities Test Completed
init: process pid=2 exited
init: test execution completed, starting judger
Judger: Starting evaluation
Test3 output:
Testing MLFQ Scheduler - Basic
MLFQ Scheduler Process 5 with initial priority 3 and
final priority 3 completed
MLFQ Scheduler Process 2 with initial priority 1 and
final priority 1 completed
MLFQ Scheduler Process 4 with initial priority 5 and
final priority 5 completed

```

```
MLFQ Scheduler Process 3 with initial priority 2 and
final priority 2 completed
MLFQ Scheduler Process 1 with initial priority 10 and
final priority 10 completed
MLFQ with Priorities Test Completed
Expected order: 2 0 0 0 1
Test3 FAILED
SCORE: 0
```

还是没什么头绪

- 看看测试样例的要求:

```
... (前面就是一个安全的打印函数而已)
void cpu_intensive_task(int id, int priority) {
    volatile long long count = 0;
    for(int loop = 0; loop < LOOP*10; loop++) {
        for(int i = 0; i < ITERATIONS; i++) {
            count += (long long)i * (long long) i;
            if(i % (ITERATIONS/2) == 0) {
                // printf("CPU Process %d (prio %d):
iteration %d\n", id, priority, i);
            }
        }
    }
    int final_prio = get_priority();
    write_mlfq_completion(id, priority, final_prio);
}

void io_intensive_task(int id, int priority) {
    for(int i = 0; i < 30; i++) {
        // printf("IO Process %d (prio %d): iteration
%d\n", id, priority, i);
        sleep(1); // Simulate I/O operation
    }
    int final_prio = get_priority();
    write_mlfq_completion(id, priority, final_prio);
}
```

```

void mixed_task(int id, int priority) {
    volatile long long count = 0;
    for(int i = 0; i < 20; i++) {
        // Do some computation
        for(int j = 0; j < ITERATIONS/10; j++) {
            count += (long long)j * (long long) j;
        }
        // printf("Mixed Process %d (priority %d): it
eration %d\n", id, priority, i);
        sleep(1); // Some I/O
    }
    int final_prio = get_priority();
    write_mlfq_completion(id, priority, final_prio);
}

```

```

/*
* Desc:
* We fork five processes with different characteristics and priorities:
* Priority: smaller number = higher priority
* - P1: CPU-intensive, low priority (10) - should be demoted
* - P2: I/O-intensive, high priority (1) - should stay in high queues
* - P3: CPU-intensive, high priority (2) - initially high but may be demoted
* - P4: I/O-intensive, medium priority (5) - should be promoted
* - P5: Mixed workload, high priority (3)
*
* Expected:
* The highest priority I/O-bound processes (P2) should finish first,
* and the lowest priority CPU-bound (P1) should complete last.
* CPU-intensive process P1 & P3 will have an ending priority lower than initial,
* while I/O-intensive process P4 will have an higher

```

```

priority than the initial one.
*/

int main() {
    printf("Testing MLFQ Scheduler - Basic\n");

    int pid1, pid2, pid3, pid4, pid5;

    // Low priority CPU-bound (may be demoted)
    if((pid1=fork())==0) {
        set_priority(10);    // Lowest priority (large
                             st number)
        cpu_intensive_task(1, 10);
        exit(0);
    }

    // Highest priority I/O-bound (should stay in high
    queues)
    if((pid2=fork())==0) {
        set_priority(1);    // Highest priority (smallest
                             number)
        io_intensive_task(2, 1);
        exit(0);
    }

    // High priority CPU-bound (initially high but may
    be demoted)
    if((pid3=fork())==0) {
        set_priority(2);    // High priority
        cpu_intensive_task(3, 2);
        exit(0);
    }

    // Medium priority I/O-bound (initially low but may
    be promoted)
    if((pid4=fork())==0) {
        set_priority(5);    // Medium priority
        io_intensive_task(4, 5);
    }
}

```

```

        exit(0);
    }

    // High priority mixed workload
    if((pid5=fork())==0) {
        set_priority(3);    // High priority
        mixed_task(5, 3);
        exit(0);
    }

    for (int i = 0; i < 5; i++) {
        wait(0);
    }

    printf("MLFQ with Priorities Test Completed\n");
    exit(0);
}

```

在这个测试样例中，有三种不同的任务：

1. cpu密集型：它会一直执行大量的乘法，永远不交出cpu，这个任务在mlfq的规则里，优先级应当很低。
2. io密集型：它只干一点活，然后就交出cpu，这是调度器很喜欢的任务，为了保证交互响应，它应当被保持高优先级或者提高它的优先级。
3. mixed任务：做一点乘法，然后sleep交出cpu，它的优先级应该属于中游水平。

按照文档的要求，我们最后应该有**P2**最先结束，而最低优先级的**P1**应最后完成。CPU密集型的**P1**和**P3**最终优先级会低于初始值，而I/O密集型的**P4**最终优先级会高于初始值。

- 阅读文档，这里io密集型任务调用sleep其实是在模拟cpu等待io，当累积时间/比例超过一定程度后就会获得优先级的提升，而cpu密集型，则会根据它被“强制下线”的次数来降低优先级。
 - 所以在proc结构体中，我们还要声明一些新的成员变量来满足这个判断过程。

```

struct proc
{

```

```

...
    int priority;                // process priority
for priority scheduler
    int now_priority;           // now priority for
mlfq scheduler
    int io_ticks;               // ticks doing I/O,
for mlfq scheduler
    int cpu_ticks;              // ticks doing CPU
work, for mlfq scheduler
};

```

`now_priority` 就是在所有进程的调度与运行中的增减以后的实时优先级。

`priority` 则是通过 `sys_setpriority()` 设定的初始优先级。

`io_ticks` 记录了这个进程做了多少个io任务的时间片（即 `sleep` /主动触发 `sched` 了多少次）。

`cpu_ticks` 记录了这个进程做了多少个cpu任务的时间片（即被强制下线了多少次）。

- 因此我们还得修改一些函数，包括进程全周期的维护函数，以及 `sys_getpriority` 和 `sys_setpriority`。

```

uint64
sys_set_priority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
    p->priority = priority;
    p->now_priority = priority; // set now_priority
same as initial priority
    return 0;
}

uint64
sys_get_priority(void)

```

```

{
    struct proc *p = myproc();
    return p->now_priority;
}

```

这里的 `fork` 有点要注意，新得到的子进程，它的优先级虽然与父进程相同，但它的 `io_ticks` 与 `cpu_ticks` 都要置为0，因为它没有经历过调度。

```

int fork(void)
{
    ...
    pid = np->pid;

    np->priority = p->priority; // set priority of the child same as parent

    np->now_priority = p->now_priority; // set now_priority of the child same as parent

    np->io_ticks = 0; // set io_ticks of the child to 0

    np->cpu_ticks = 0; // set cpu_ticks of the child to 0

    np->state = RUNNABLE;

    release(&np->lock);

    return pid;
}

```

- 接下来我们开始对这些进程进行赏罚机制。
 - 在之前我们知道如果一个进程被cpu强制下线，那么会由trap函数通过调用 `yield` 间接调用 `shed`，而在io密集型的任务中，我们则是直接调用了 `sleep` 来模拟。因此我们可以直接在 `yield` 以及 `sleep` 的函数上做文章。

- 我在这里选择的策略是一个进程每经过固定次数的被调度运行后，检查主动 `sleep` 与被动 `yield` 的比例，根据比例决定升降 `priority`，你也可以直接在 `sleep` 中检查 `io_ticks` 的次数并升高 `priority`。这里我新增了一个评测函数，每次 `sleep` 和 `yield` 时都进行一次检验。

```
void evaluate_priority(struct proc *p, int total_ticks)
{
    int cpu_percent = (p->cpu_ticks * 100) / total_ticks;

    // 判定规则：
    if (cpu_percent >= 70)
    {
        // CPU 动作占 70% 以上，典型的贪婪型 -> 降级
        if (p->now_priority < 100)
        {
            p->now_priority++;
        }
    }
    else if (cpu_percent <= 30)
    {
        // CPU 动作低于 30% (意味着 IO 占了 70% 以上)，典型的交互型 -> 升级
        if (p->now_priority > 1)
        {
            p->now_priority--;
        }
    }
    else
    {
        // CPU 占比在 30% 到 70% 之间 (混合型)，不奖不罚，维持原状
    }
    // 结算后重置计数器，开始新一轮的考核周期
    p->cpu_ticks = 0;
    p->io_ticks = 0;
}
```


在 `yield` 与 `sleep` 中添加相应逻辑:

```
void yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->cpu_ticks += 1;
    int total_ticks = p->io_ticks + p->cpu_ticks;
    if (total_ticks >= 30)
    {
        evaluate_priority(p, total_ticks);
    }
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}

void sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();
    if (lk != &p->lock)
    {
        // DOC: sleeplock0
        acquire(&p->lock); // DOC: sleeplock1
        release(lk);
    }

    p->io_ticks += 1;
    int total_ticks = p->io_ticks + p->cpu_ticks;
    if (total_ticks >= 30)
    {
        evaluate_priority(p, total_ticks);
    }
    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    // Tidy up.
    p->chan = 0;
}
```

```

// Reacquire original lock.
if (lk != &p->lock)
{
    release(&p->lock);
    acquire(lk);
}
}

```

最后把 `scheduler` 里面关于 `priority` 的判断变成关于 `now_priority` 的判断，并添加一些有关 `base_priority` 的判断，就行了。

```

void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    extern pagetable_t kernel_pagetable;

    c->proc = 0;
    for (;;)
    {
        // Avoid deadlock by ensuring that devices can
        interrupt.
        intr_on();

        int found = 0;
        int highest_pri = 1000000;
        int base_pri = 1000000;

        for (p = proc; p < &proc[NPROC]; p++)
        {
            acquire(&p->lock);
            if (p->state == RUNNABLE && p->now_priority
                < highest_pri)
            {
                highest_pri = p->now_priority;
                base_pri = p->priority;
            }
            else if (p->state == RUNNABLE && p->now_prio

```

```

rity == highest_pri && p->priority < base_pri)
{
    base_pri = p->priority;
}
release(&p->lock);
}

for (p = proc; p < &proc[NPROC]; p++)
{
    acquire(&p->lock);
    if (p->state == RUNNABLE && p->now_priority
== highest_pri && p->priority == base_pri)
    {
        // Switch to chosen process. It is the pr
ocess's job
        // to release its lock and then reacquire
it
        // before jumping back to us.
        // printf("[scheduler]found runnable proc
with pid: %d\n", p->pid);
        p->state = RUNNING;
        c->proc = p;
        w_satp(MAKE_SATP(p->kpagetable));
        sfence_vma();
        swtch(&c->context, &p->context);
        w_satp(MAKE_SATP(kernel_pagetable));
        sfence_vma();
        // Process is done running for now.
        // It should have changed its p->state bef
ore coming back.
        c->proc = 0;

        found = 1;
    }
    if (p->state == RUNNABLE)
    {
        found = 1; // 如果有一个RUNNABLE的进程，就不让
cpu休息，虽然不能执行，但还要重新跑一次比较的循环

```

```
// 除非系统真的没有进程了，这时就让C  
PU休息一下，等中断来唤醒它
```

```
    }  
    release(&p->lock);  
}  
if (found == 0)  
{  
    intr_on();  
    asm volatile("wfi");  
}  
}  
}
```

lab5

要求：在原先os-part4仓库中完成内存相关调用 `test_mem_lazy_allocation`
`test_mem_cow`

▼ 前置操作

- 完全不需要再git clone新的仓库，我们直接在os-part4这个目录下，运行这个指令

```
git fetch origin
git checkout part5-mem
```

切换到part5-mem分支下，再运行 `git checkout -b part5-lazy` 创建一个新的分支

▼ mem_lazy_allocation

- 先看一下要求：

```
#include "test.h"

#define SIZE (1 << 14) // 16KB
#define PAGES (SIZE / 4096) // 4KB per page

/*
 * Desc:
 * We test lazy allocation by allocating a large memory region but only
 * actually using a portion of it. We then check the physical page count
 * to ensure only the used pages are allocated.
 *
 * Expected:
 * Initially, no extra pages should be allocated. After accessing the first
 * few pages, only those pages should be allocated. The judge program will
 * check the physical page count at each step to verify
```

```

y lazy allocation.
*/

int main() {
    printf("Testing Lazy Allocation\n");

    // Get initial physical page count
    int initial_pages = getpgcnt();
    printf("Initial physical pages: %d\n", initial_pages);

    // Allocate a large memory region (should not allocate physical pages yet)
    char *mem = sbrk(SIZE);
    if (mem == (char*)-1) {
        printf("sbrk failed\n");
        exit(1);
    }

    int after_alloc_pages = getpgcnt();
    printf("Physical pages after sbrk: %d (should be same as initial)\n", after_alloc_pages);

    if (after_alloc_pages != initial_pages) {
        printf("ERROR: Physical pages increased without access!\n");
        exit(1);
    }

    // Access first page (should trigger page fault and allocate one page)
    mem[0] = 'A';
    int after_first_access = getpgcnt();
    printf("Physical pages after first access: %d (should be +1)\n", after_first_access);

    if (after_first_access != initial_pages + 1) {
        printf("ERROR: Expected %d pages, got %d\n",

```

```

initial_pages + 1, after_first_access);
    exit(1);
}

// Access a page in the middle
mem[SIZE/2] = 'B';
int after_mid_access = getpgcnt();
printf("Physical pages after mid access: %d (should be +2)\n", after_mid_access);

if (after_mid_access != initial_pages + 2) {
    printf("ERROR: Expected %d pages, got %d\n",
initial_pages + 2, after_mid_access);
    exit(1);
}

// Access last page
mem[SIZE-1] = 'C';
int after_last_access = getpgcnt();
printf("Physical pages after last access: %d (should be +3)\n", after_last_access);

if (after_last_access != initial_pages + 3) {
    printf("ERROR: Expected %d pages, got %d\n",
initial_pages + 3, after_last_access);
    exit(1);
}

printf("Lazy Allocation Test Completed Successfully\n");
exit(0);
}

```

这个测试样例中，重复调用了 `getpgcnt()` 用来检测是否在 `sbrk` 要求分配时，系统只进行了 `sz` 的调整，而没有真正执行物理页的分配，只有在三次真正向 `mem` 数组写入时才会分配真正的物理页。而 `mem` 数组在起初，本应由 `sbrk` 分配一个 16KB=4Page 大小的虚拟空间，但因为是懒分配，这些虚拟地址实际上并没有映射到真实的物理空间中。

- 在这个测试样例中，要实现懒分配，参照助教给出的指引，应该先实现 `getpgcnt` 用于统计全局已分配物理页数。运行 `make run_test TYPE=LAZY`，我们发现如下报错

```
pid 2 test_mem_lazy_a: unknown sys call 54325
pid 2 test_mem_lazy_a: unknown sys call 54325
pid 2 test_mem_lazy_a: unknown sys call 54325
```

查询 `sysnum.h`，发现 `#define SYS_getpgcnt 54325`，而 `syscall.c` 中并没有完成相应注册，所以我们添加这个 `sys_getpgcnt()` 的注册信息，然后在 `sysproc.c` 中进行完整的实现。

- 根据助教的提示，我们知道需要在 `kalloc.c` 中调整一些逻辑，用于统计全局的分配情况，而不是直接在当前进程中遍历页表。
 - ▼ 接下来要分析一下 `kalloc.c` 中的具体内容，看一下 xv6 是怎么内存管理的，以及我们该怎么下手操作。

```
// Physical memory allocator, for user processes,
// kernel stacks, page-table pages,
// and pipe buffers. Allocates whole 4096-byte
// pages.
```

```
#include "include/types.h"
#include "include/param.h"
#include "include/memlayout.h"
#include "include/riscv.h"
#include "include/spinlock.h"
#include "include/kalloc.h"
#include "include/string.h"
#include "include/printf.h"
```

```
void freerange(void *pa_start, void *pa_end);
```

```
extern char kernel_end[]; // first address after kernel.
```

```
struct run {
```



```

    struct run *next;
};

struct {
    struct spinlock lock;
    struct run *freelist;
    uint64 npage;
} kmem;

void
kinit()
{
    initlock(&kmem.lock, "kmem");
    kmem.freelist = 0;
    kmem.npage = 0;
    freerange(kernel_end, (void*)PHYSTOP);
#ifdef DEBUG
    printf("kernel_end: %p, phystop: %p\n", kernel_end, (void*)PHYSTOP);
    printf("kinit\n");
#endif
}

void
freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char*)PGROUNDUP((uint64)pa_start);
    for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
        kfree(p);
}

// Free the page of physical memory pointed at
// by v,
// which normally should have been returned by
// a
// call to kalloc(). (The exception is when

```

```

// initializing the allocator; see kinit above.)
void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa <
kernel_end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    kmem.npage++;
    release(&kmem.lock);
}

// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r) {
        kmem.freelist = r->next;
        kmem.npage--;
    }
}

```

```

    release(&kmem.lock);

    if(r)
        memset((char*)r, 5, PGSIZE); // fill with j
unk
    return (void*)r;
}

uint64
freemem_amount(void)
{
    return kmem.npage << PGSHIFT;
}

```

- `extern char kernel_end[];`：这是一个外部定义的符号，由于物理内存分配器不能把内核自己正在用的内存分配出去。所以，可供分配的空闲物理内存，是从 `kernel_end` 开始，一直到物理内存的最高点 `PHYSTOP` 这里为 `0x80600000`。
- `struct run`：这是空闲链表 `freelist` 的节点结构体，它的前8Byte是指向下一个节点的指针，这样，每个空闲物理页的前8Byte就会自然地用来存放下一个空闲页的指针，由于这个页本身就是空闲的，这部分空间就被很好地利用了。**注意**，在这个 `kalloc.c` 中，传递的指针虽然都是 `(void*)`，但已经是**直接指向真实物理页地址**的指针了，因此这个整个脚本已经是虚拟内存页表与真实物理页的桥接。
- `struct kmem`：全局内存管理者，`freelist` 是全局空闲页链表的起始地址，`npage` 则是还有多少空闲页。
- `kinit()`：初始化 `kmem`，调用 `freerange`，将 `kernel_end[]` 起始到达物理内存最高点 `PHYSTOP` 处全部释放。
- `freerange(void *pa_start, void *pa_end)`：把给定范围内的物理内存，切成一个个 4096 字节（4KB，`PGSIZE`）的标准页，然后循环调用 `kfree` 把它们挂进空闲链表。
- `kfree(void *pa)`：
 - **垃圾填充（Catch dangling refs）**：`memset(pa, 1, PGSIZE)`。把要释放的这一整页全部填满 `1`。这是一种极佳的**防御性编程**。如果有恶意程序或写错的内核代码还在使用（Use-

after-free) 这块被释放的内存，读出来的全是一堆乱码（垃圾数据），很容易让程序立即崩溃报错，从而及早发现 Bug。

- **链表头插法：** 把这个物理页强转成 `struct run*`，然后采用头插法（LIFO，后进先出）挂入 `kmem.freelist`，最后 `npage++`。
- **`kalloc(void)`（这是几乎所有新分配/复制页表的核心函数）：**
 - **取走头部：** 加锁后，直接把 `kmem.freelist` 指向的第一个物理页拿出来（从链表头上摘除节点），然后 `npage--`。
 - **再次垃圾填充：** 如果分配成功，`memset((char*)r, 5, PGSIZE)` 把这页填满 5。原因同上，防止新分配的内存里残留着上一个进程的密码、密钥等机密数据，同时也防止未初始化就直接使用引发的随机 Bug。
- **`freemem_amount(void)`：** 把空闲页数 `npage` 左移 `PGSHIFT`（乘以 4096），得出空闲内存的总字节数。
- 之后我们发现这里可以直接用 `kmem` 的成员 `npage`，但是我们仍不知道总页数是多少，如果直接使用现有的 `freemem_amount() >> PGSHIFT`，则返回的是空闲页面数。因此在这里我们再声明一个全局变量 `total_pages` 并在 `init` 的时候对整个物理空间可分配的页面数做一个统计，补充一个接口函数，就大功告成。

```
uint64 total_pages = 0;

void freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char *)PGROUNDUP((uint64)pa_start);
    for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
    {
        total_pages++;
        kfree(p);
    }
}

uint64
allocated_pages(void)
```

```

{
    acquire(&kmem.lock);
    uint64 pages = total_pages - kmem.npage;
    release(&kmem.lock);
    return pages;
}

```

记得在 `kalloc.h` 中声明这个新的函数，否则不能在 `sysproc.c` 中调用。
在 `sysproc.c` 中，我们添加函数定义：

```

uint64
sys_getpgcnt(void)
{
    return allocated_pages();
}

```

之后可以看到结果是这样的：

```

init: starting test_mem_lazy_allocation
testing output size:157, contents:
Testing Lazy Allocation
Initial physical pages: 40
Physical pages after sbrk: 44 (should be same as initial)
ERROR: Physical pages increased without access!
init: process pid=2 exited
init: test execution completed, starting judger
Judger: Starting evaluation
Test2 output:
Testing Lazy Allocation
Initial physical pages: 40
Physical pages after sbrk: 44 (should be same as initial)
ERROR: Physical pages increased without access!

```

在第一次 `sbrk` 的时候，直接分配了4个完整的物理页，所以与要求不符，但我们确实已经实现了 `sys_getpgcnt`。

- 下一步，我们继续调整 `sysproc.c` 中的 `sbrk` 逻辑，让它在被调用时只改变 `myproc()-sz`，而不真正分配物理页面。
 - `sys_sbrk` 的核心函数 `growproc()`，它在做的事情实际上也就是：调整 `sz`；在需要更多内存时调用 `uvmalloc`，在需要更少内存时调用 `uvmdealloc`。在这里我们就先不管少内存时的解映射。如果只是改变 `sz`，那我们直接把前面有关 `uvmalloc` 部分的调用去掉就好了，因为真正的分配过程要在用到时，触发缺页异常再来处理。

```
int growproc(int n)
{
    uint sz;
    struct proc *p = myproc();

    sz = p->sz;
    if (n > 0)
    {
        // 懒分配实现
        sz = sz + n;
    }
    else if (n < 0)
    {
        sz = uvmdealloc(p->pagetable, p->kpagetable, sz, sz + n);
    }
    p->sz = sz;
    return 0;
}
```

- 下一步，我们要处理 `sbrk` 中，涉及到解除映射的部分，也就是 `uvmdealloc` 的部分，这块先研究一下 `uvmalloc` 和 `uvmdealloc` 这两个函数：

▼ `uvmalloc`：

```
// Allocate PTEs and physical memory to grow process from oldsz to
// newsz, which need not be page aligned. Returns new size or 0 on error.
uint64
```

```

uvmalloc(pagetable_t pagetable, pagetable_t kpaget
able, uint64 oldsz, uint64 newsz)
{
    char *mem;
    uint64 a;

    if(newsz < oldsz)
        return oldsz;

    oldsz = PGROUNDUP(oldsz);
    for(a = oldsz; a < newsz; a += PGSIZE){
        // 每次循环前进一页 (4KB)。调 kalloc 要一块真实的物理内存。
        // 拿到后, 必须 memset 清零。
        mem = kalloc();
        if(mem == NULL){
            uvmdealloc(pagetable, kpagetable, a, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        // 第 1 次映射: 映射到用户页表
        if (mappages(pagetable, a, PGSIZE, (uint64)mem, PTE_W|PTE_X|PTE_R|PTE_U) != 0) {
            kfree(mem);
            uvmdealloc(pagetable, kpagetable, a, oldsz);
            return 0;
        }
        // 第 2 次映射: 映射到内核页表
        if (mappages(kpagetable, a, PGSIZE, (uint64)mem, PTE_W|PTE_X|PTE_R) != 0){
            int npages = (a - oldsz) / PGSIZE;
            vmunmap(pagetable, oldsz, npages + 1, 1);
            // plus the page allocated above.
            vmunmap(kpagetable, oldsz, npages, 0);
            return 0;
        }
    }
}

```

```

return newsz;
}

```

- 在 `kalloc` 这一页成功后，会进行两次 `mappages`：
 - **给用户用**：带有 `PTE_U` (User) 权限，这意味着 CPU 在用户态 (U Mode) 时可以读写执行它。
 - **给内核用**：没有 `PTE_U` 权限！为什么要去掉？因为 RISC-V 架构的安全机制规定，内核态 (S Mode) 默认情况下**绝对禁止**访问带有 `PTE_U` 标志的页面（除非设置 `sstatus.SUM` 位）。不带 `PTE_U`，内核页表把这个地址当成自己的特权内存，内核就可以直接用指针对其进行读写了。
- 而第二次映射到内核失败时，则会触发回滚机制
 - `vmunmap(pagetable, ..., 1)`：把用户页表的映射删掉。注意最后的参数 `1`，代表连带释放底层的物理内存 `kfree`。这里要清理 `npages + 1` 页（包含刚才刚映射进去的那一页）。
 - `vmunmap(kpagetable, ..., 0)`：把内核页表的映射删掉。注意最后的参数 `0`，代表只删页表项，不去二次 `kfree` 物理内存。

▼ `uvmdealloc`：

```

// Deallocate user pages to bring the process size
// from oldsz to
// newsz.  oldsz and newsz need not be page-aligne
// d, nor does newsz
// need to be less than oldsz.  oldsz can be large
// r than the actual
// process size.  Returns the new process size.
uint64
uvmdealloc(pagetable_t pagetable, pagetable_t kpag
etable, uint64 oldsz, uint64 newsz)
{
    if(newsz >= oldsz)
        return oldsz;
    // 注意到，如果newsz和oldsz在对齐后的内存大小相等，则仍不
    // 触发vmunmap的解除映射
    if(PGROUNDUP(newsz) < PGROUNDUP(oldsz)){
        int npages = (PGROUNDUP(oldsz) - PGROUNDUP(new

```



```

sz)) / PGSIZE;
    // 解除这些页面的内核映射和用户映射
    vmunmap(kpagetable, PGROUNDUP(newsz), npages,
0);
    vmunmap(pagetable, PGROUNDUP(newsz), npages,
1);
}

return newsz;
}

```

- 接下来，我们应该修改 `vmunmap` 的定义

```

void vmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if ((va % PGSIZE) != 0)
        panic("vmunmap: not aligned");

    for (a = va; a < va + npages * PGSIZE; a += PGSIZE)
    {
        if ((pte = walk(pagetable, a, 0)) == 0)
            // panic("vmunmap: walk");
            continue; // 懒分配修改 1: 如果没有分配过页表目录，直接跳过当前页

        if ((*pte & PTE_V) == 0)
            // panic("vmunmap: not mapped");
            continue; // 懒分配修改 2: 如果页表项存在但无效（没挂物理页），直接跳过当前页

        if (PTE_FLAGS(*pte) == PTE_V)
            panic("vmunmap: not a leaf");
    }
}

```

```

    if (do_free)
    {
        uint64 pa = PTE2PA(*pte);
        kfree((void *)pa);
    }
    *pte = 0;
}
}

```

这样，在解除空头支票的时候，不会在 `walk` 找不到对应物理页面时报错了。

- 下一步，我们需要处理 `trap.c` 中的中断处理过程，当产生缺页异常时，要给这个引发异常的地址真正分配一个物理页。新增 `lazy_handler` 处理函数，每次发生Page Fault时（`r_scause` 的值可以判断）调用。处理函数校验Page Fault的地址是否是**进程已分配地址空间内的合法地址**，如果是合法地址，使用 `kalloc` **分配页面并重新映射到页表**。

现在如果我们再次运行这个测试，则会出现这样的报错，这个报错的 `scause` 就是写缺页异常的错误号了。

```

usertrap(): unexpected scause 0x000000000000000f p
id=2 test_mem_lazy_a
sepc=0x00000000000000a4 stval=0x0000000000003000

```

随后，因为我们需要给缺页异常以新的处理逻辑，我们需要在用户态抛出异常，被检查到时，跳转的逻辑，这时 `scause==0xf` 就不必直接引发异常并杀死进程了。

```

void usertrap(void)
{
    ...
    else if ((which_dev = devintr()) != 0)
    {
        // ok
    }
    else if (r_scause() == 0xf) // 添加有关写缺页的异常处理
    {
        uint64 vmaddr = r_stval();
    }
}

```

```

        if (vmaddr >= p->sz) // 超限检查, 如果这个引发缺
// 异常的虚拟地址超出了进程空间大小
        {
            p->killed = 1;
        }
        else
        {
            if (lazy_handler(vmaddr, p) < 0) // 这里就是
// 真正要进行的懒分配处理器
            {
                p->killed = 1;
            }
        }
    }
    else
    ...
}

```

接下来, 在该函数之前, 我们要完成 `lazy_handler` 的定义, 这里我们其实可以直接仿照 `uvmmalloc` 的逻辑, 只不过我们在这时只需要处理单页的缺页问题, 因此有很多内容与参数可以省去。

```

int lazy_handler(uint64 va, struct proc *p)
{
    char *mem;
    // 这里的a就不用像uvmmalloc中那样作为一页一页遍历的指针
    // 了
    uint64 a = PGROUNDDOWN(va);

    mem = kalloc();
    if (mem == NULL)
    {
        // 如果物理内存真没了, 直接报错就行
        // 不同于uvmmalloc中, 如果分配到这一页物理内存用完, 要
        // 把之前的全部解映射
        // 这里什么都还没映射, 不需要 dealloc
        return -1;
    }
}

```

```

memset(mem, 0, PGSIZE);

// 第 1 次映射：映射到用户页表
if (mappages(p->pagetable, a, PGSIZE, (uint64)mem,
PTE_W | PTE_X | PTE_R | PTE_U) != 0)
{
    kfree(mem); // 映射失败，只需把刚拿到的这一页物理内存还回去
    return -1;
}

// 第 2 次映射：映射到内核页表
if (mappages(p->kpagetable, a, PGSIZE, (uint64)mem,
PTE_W | PTE_X | PTE_R) != 0)
{
    // 内核映射失败，必须撤销刚才成功的用户态映射
    // 注意：只 unmap 这一页 (a)，数量是 1，最后一个参数 1 表示连带 kfree 释放底层物理内存
    vmunmap(p->pagetable, a, 1, 1);
    return -1;
}

return 0;
}

```

此外，要记得在开头添加这几个声明，否则我们在新函数中的调用是无法被正确编译的：

```

#include "include/kalloc.h"
#include "include/vm.h"
#include "include/string.h"

```

这样这个节点就顺利通过！

▼ mem_cow

- 对于cow的过程，在我的**课程笔记/寻址逻辑**中有一些思考。
- 在part5-mem下创建新分支 `git checkout -b part5-cow` 。

- 仍旧先实现 `getpgcnt`，这个不再赘述。
- 接下来，看一下需要做什么，在下面的代码中标出得分规则

```
#include "test.h"

#define SIZE (1 << 12) // 4KB (one page)
#define PGSIZE (1 << 12)

/*
 * Desc:
 * We test copy-on-write by forking a process and having both parent and child
 * read from the same memory. Then we modify the memory in one process and check
 * that the physical page count increases appropriately.
 *
 * Expected:
 * After fork, physical page count should remain the same (shared pages).
 * After writing to a page in either process, the page should be copied and
 * the physical page count should increase by one.
 */

int main() {
    printf("Testing Copy-on-Write\n");

    // Get initial physical page count
    int initial_pages = getpgcnt();
    printf("Initial physical pages: %d\n", initial_pages);

    // Allocate and initialize a page
    char *mem = sbrk(SIZE);
    if (mem == (char*)-1) {
        printf("sbrk failed\n");
        exit(1);
    }
}
```

```

    }

    int write_sum = 0;

    for (int i = 0; i < SIZE; i++) {
        mem[i] = i % 256;
        write_sum += mem[i];
    }

    int after_init_pages = getpgcnt();
    printf("Physical pages after initialization: %d\n", after_init_pages);

    // 正确实现getpgcnt以后，在第一次sbrk分配一页空间并进行
    // 写操作后，
    // 现在的已分配物理页数应当比刚开始多1（因此这里不要求一定
    // 得用lazy分配才行）
    if (after_init_pages != initial_pages + 1) {
        printf("ERROR: Expected %d pages, got %d\n",
            initial_pages + 1, after_init_pages);
        exit(1);
    }

    int sz = getprocsz();
    int delta_without_cow =
        (sz / PGSIZE)    // 为子进程完全创建与父进程一样数
        + 2              // 这是给用户态的页表目录，由于RIS
        + 2              // CV在这里用的是三级页表，因此添加二级页表和三级页表目录，要单独
        + 2              // 分配两页
        + 2              // 这是给内核态的页表，这是xv6在这
        + 1              // 的变体，为了内核方便读写用户内容，当前进程用户内存也会映射到内核
        + 2              // 页表，跟上面的+2一个意义
        + 1              // 进程用户态的根页表，也就是一级
        + 2              // 页表，这个页表的基地址一般就存在SATP中了，allocproc中
        + 2              // mapping trampoline，在地址
        + 2              // 最高端，跨越了很大虚拟空间，因此不能与用户数据共用页表目录，要额
        + 2              // 外分配一级二级页表目录

```

```

        + 1 // trapframe per proc, 这一步
的额外分配页表过程可以在proc.c的allocproc找到
        + 1 // kernel page table, 内核态根
页表
        + 1 + 2 // +1: 真正的内核栈页面; +2: 对于
该内核栈的两级页表目录
    ;

```

```

// Fork a process
int pid = fork();
if (pid < 0) {
    printf("fork failed\n");
    exit(1);
}

int after_fork_pages = getpgcnt();
printf("Physical pages after fork: %d\n", after_f
ork_pages);

```

// 如果没有实现cow, 那么这一步应该由前者等于后者, 则无法通过测试点

```

if (after_fork_pages >= after_init_pages + delta_
without_cow) {
    printf("ERROR: Page count changed too much fr
om %d to %d after fork without write\n", after_init_p
ages, after_fork_pages);
}

```

```

if (pid == 0) {
    // Child process
    // Read from the shared page (should not trig
ger copy)
    int child_before_read = getpgcnt();
    printf("Physical pages before child read: %d
\n", child_before_read);

```

```

    int sum = 0;
    for (int i = 0; i < SIZE; i++) {

```

```

        sum += mem[i];
    }
    printf("Child read sum: %d\n", sum);

    // 之前写进父进程页面的内容应该可以被子进程完全正确地
    读入，否则说明cow的映射在读时有问题
    if (sum != write_sum) {
        printf("ERROR: Data corruption. Sum should be %d, but got %d\n", write_sum, sum);
        exit(2);
    }

    int child_after_read = getpgcnt();
    printf("Physical pages after child read: %d (should be same)\n", child_after_read);

    // 读完页面应该不触发写权限异常，因为cow是可读不可写的权限
    if (child_after_read != child_before_read) {
        printf("ERROR: Page count changed after read\n");
        exit(3);
    }

    // Write to the page (should trigger copy)
    mem[0] = 0xFF;
    int child_write_pages = getpgcnt();
    printf("Physical pages after child write: %d (should be increased)\n", child_write_pages);

    // 写的时候应当触发越权，即缺页异常，并复制一个新页，
    再将子进程映射过去
    if (child_write_pages < after_fork_pages + 1)
    {
        printf("ERROR: Expected at least %d pages, got %d\n", after_fork_pages + 1, child_write_pages);
        exit(1);
    }

```



```

    }

    exit(0);
} else {
    // Parent process
    int status ;
    wait(&status);
    printf("wait status:%d\n", status);
    int before_write_pages = getpgcnt();

    // Check that parent's page is unchanged
    int sum = 0;
    for (int i = 0; i < SIZE; i++) {
        sum += mem[i];
    }
    printf("Parent read sum: %d\n", sum);

    // 如果正确实现子进程cow, mem[0] = 0xff不会影响到父进程的加和
    if (sum != write_sum) {
        printf("ERROR: Data corruption. Sum should be %d, but got %d\n", write_sum, sum);
        exit(1);
    }

    int after_read_pages = getpgcnt();
    printf("Physical pages after child exit: %d (should be same as before read)\n", after_read_pages);

    // 父进程读那些cow页面也不应该触发异常, 因此前后物理页面数应当不变
    if (after_read_pages != before_write_pages) {
        printf("ERROR: Expected %d pages, got %d\n", before_write_pages, after_read_pages);
        exit(1);
    }
}

```

```

        // Parent writes to the page (should not trigger copy as child is gone)
        mem[0] = 0xAA;
        int parent_write_pages = getpgcnt();
        printf("Physical pages after parent write: %d (should be same)\n", parent_write_pages);

        // 子进程已经离去，因此父进程应当不再存在cow的不可写问题，因此不会触发缺页，而是直接在现有页面写
        if (parent_write_pages != after_read_pages) {
            printf("ERROR: Page count changed after parent write\n");
            exit(1);
        }
    }

    printf("Copy-on-Write Test Completed Successfully\n");
    exit(0);
}

```

如果直接运行，在第一步就会得到这样的结果：

```

init: starting test_mem_cow
testing output size:771, contents:
Testing Copy-on-Write
Initial physical pages: 40
Physical pages after initialization: 41 (should be + 1)
Parent proc pages: 4
Physical pages after fork: 57
ERROR: Page count changed too much from 41 to 57 after fork without write
Parent proc pages: 4
Physical pages after fork: 57
ERROR: Page count changed too much from 41 to 57 after fork without write
Physical pages before child read: 57

```

```

Child read sum: 522240
Physical pages after child read: 57 (should be same)
Physical pages after child write: 57 (should be increased)
ERROR: Expected at least 58 pages, got 57
wait status:1
Parent read sum: 522240
Physical pages after child exit: 41 (should be same as before read)
Physical pages after parent write: 41 (should be same)
Copy-on-Write Test Completed Successfully
init: process pid=2 exited

```

在这里我看了一下父进程本身需要的页面数，进行计算后，子进程在cow实现前，会造成所有物理页面数增加到57页的情况，这是合理的。

- 我们需要关注 `fork` 的具体实现，可以发现最核心的复制逻辑在于 `allocproc` 这一步，顺藤摸瓜，在这个函数中，我们又找到了 `proc_pagetable`，这个函数是用来创建用户态页表的，但这个函数并不包含将用户态的内存信息copy到子进程这一步，不过会把trampoline的页面建立映射。

```

extern char trampoline[]; // trampoline.S
...

// Create a user page table for a given process,
// with no user memory, but with trampoline pages.
pagetable_t
proc_pagetable(struct proc *p)
{
    pagetable_t pagetable;

    // An empty page table.
    pagetable = uvmcreate();
    if(pagetable == 0)
        return NULL;

    // map the trampoline code (for system call return)
    // at the highest user virtual address.

```

```

// only the supervisor uses it, on the way
// to/from user space, so not PTE_U.
if(mappages(pagetable, TRAMPOLINE, PGSIZE,
            (uint64)trampoline, PTE_R | PTE_X) < 0)
{
    uvmfree(pagetable, 0);
    return NULL;
}

// map the trapframe just below TRAMPOLINE, for tra
mpoline.S.
if(mappages(pagetable, TRAPFRAME, PGSIZE,
            (uint64)(p->trapframe), PTE_R | PTE_W)
< 0){
    vmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return NULL;
}

return pagetable;
}

```

其中，第一次建立映射是将 `trampoline.S` 的真实物理地址利用 `mappages` 挂到 `TRAMPOLINE` 定义的用户态最高地址空间处，并设置好可读可执行的权限，这里可以再看看 `mappages` 的具体实现情况

```

// Create PTEs for virtual addresses starting at va t
hat refer to
// physical addresses starting at pa. va and size mig
ht not
// be page-aligned. Returns 0 on success, -1 if walk
() couldn't
// allocate a needed page-table page.
int
mappages(pagetable_t pagetable, uint64 va, uint64 siz
e, uint64 pa, int perm)
{
    uint64 a, last;

```

```

pte_t *pte;

a = PGROUNDDOWN(va);
last = PGROUNDDOWN(va + size - 1);

for(;;){
    if((pte = walk(pagetable, a, 1)) == NULL)
        return -1;
    if(*pte & PTE_V)
        panic("remap");
    *pte = PA2PTE(pa) | perm | PTE_V;
    if(a == last)
        break;
    a += PGSIZE;
    pa += PGSIZE;
}
return 0;
}

```

综合上面的具体实例，这个函数的过程就是：将传入的 `va`（`TRAMPOLINE` 定义的用户态最高地址）和 `size` 先进行处理，给出被映射的虚拟地址起始页地址 `a` 与结束页地址 `last`；由于物理地址 `pa` 一定是页对齐的，因此无需处理；随后用 `walk` 函数，在三级页表中层层解析，找到最低一层页表给出的 `pte`，这时的 `pte` 就已经是指向物理地址的 `pte` 了；如果这个 `pte` 的 `PTE_V` 即 valid 位为 1，则触发重分配的报错，否则将需要映射到虚拟地址的真实物理地址 `pa` 以及其它权限都放入这个 `pte`。

- 接下来我们需要调整这一套逻辑，因为我们要引入一个新的权限位，即 `PTE_COW`。选中 `PTE_V`，跳转到定义，我们在 `riscv.h` 中添加这一条定义 `#define PTE_COW (1L << 8)`，当这个位被设置时，该 `pte` 对应的物理页即为一个 COW 状态下的被多个进程同时共享的映射。

```

#define PTE_V (1L << 0) // valid
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // 1 -> user can access
#define PTE_COW (1L << 8) // 咱们自己定义的 COW 标志位！

```

根据助教的流程，下一步我们在 `kalloc.c` 中维护一个全局的数组，用于记录每个真实物理页面被多少个进程同时映射，当只有一个计数时，该页面不再是 `cow` 的，当计数清零时，这个页面才会被 `kfree` 彻底回收，加入空闲队列。

```
struct
{
    struct spinlock lock;
    struct run *freelist;
    uint64 npage;
    uint64 ref_cnt[PHYSTOP / PGSIZE];
} kmem;
```

`PHYSTOP` 为所有物理地址的终点，而物理页要对齐 `PGSIZE`，因此这个数组的大小这样定义。事实上，我们真正可以操作的部分，即可以用于进程分配页表的部分，还是从 `kernel_end` 开始往后的物理内存空间。

接下来我们还需要维护这个 `ref_cnt`，这首先需要我们对它进行初始化（这里要对 `freerange` 进行操作），并且在 `kalloc` 时增加引用计数，在 `kfree` 时减少引用计数的同时，判断是否降为0后进行彻底回收。

```
void freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char *)PGROUNDUP((uint64)pa_start);
    for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
    {
        // 注意到在freerange中最后每一个物理页都会调用一次kfree
        // 这时会自动执行ref_cnt的减少，因此我们在调用前将其设为1，这样在初始化结束后
        // 每个物理页的引用数才会是0
        acquire(&kmem.lock);
        kmem.ref_cnt[(uint64)p / PGSIZE] = 1;
        total_pages++;
        release(&kmem.lock);
        kfree(p);
    }
}
```

```

// Free the page of physical memory pointed at by v,
// which normally should have been returned by a
// call to kalloc(). (The exception is when
// initializing the allocator; see kinit above.)
void kfree(void *pa)
{
    struct run *r;

    if (((uint64)pa % PGSIZE) != 0 || (char *)pa < kern
el_end || (uint64)pa >= PHYSTOP)
        panic("kfree");
    acquire(&kmem.lock);
    if (--kmem.ref_cnt[(uint64)pa / PGSIZE] > 0)
    {
        release(&kmem.lock);
        return;
    }
    release(&kmem.lock);

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run *)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    kmem.npage++;
    release(&kmem.lock);
}

// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
    struct run *r;

```

```

    acquire(&kmem.lock);
    r = kmem.freelist;
    if (r)
    {
        kmem.freelist = r->next;
        kmem.npage--;
    }
    release(&kmem.lock);

    if (r)
    {
        memset((char *)r, 5, PGSIZE); // fill with junk
        acquire(&kmem.lock);
        kmem.ref_cnt[(uint64)r / PGSIZE] = 1;
        release(&kmem.lock);
    }
    return (void *)r;
}

```

除此之外，页面的被引用数还会在 `fork` 中被增加，因此需要补充一个函数，作为接口暴露给 `proc.c` 使用。

```

// 新增：安全地增加引用计数
void kaddref(void *pa)
{
    acquire(&kmem.lock);
    kmem.ref_cnt[(uint64)pa / PGSIZE]++;
    release(&kmem.lock);
}

```

▼ 实际上，`kernel_end` 分隔开的，低物理地址的位置存放着内核态的全部代码，所有进程的内核态都共享这部分内容，那为什么还要在 `proc` 中声明一个 `kpagetable`，在 `fork` 的时候大费周章地把所有进程都共享的内核态代码页表再复制一份出来呢？因为在这个 `xv6` 的实现中，为了方便该进程的内核态直接使用用户态数据，它在 `proc` 中定义的 `kpagetable` 会在用户态的 `pagetable` 映射完成后也进行一次映射，如此，虽然增加了物理内存开销，但是会节省内核直接使用用户态数据的时间。


```

// initialize kernel pagetable for each process.
pagetable_t
proc_kpagetable()
{
    pagetable_t kpt = (pagetable_t) kalloc();
    if (kpt == NULL)
        return NULL;
    // 这里的kernel_pagetable就是vm.c开头定义与初始化的内
    核态不变的共享代码的页表部分
    memmove(kpt, kernel_pagetable, PGSIZE);

    // remap stack and trampoline, because they shar
    e the same page table of level 1 and 0
    char *pstack = kalloc();
    if(pstack == NULL)
        goto fail;
    if (mappages(kpt, VKSTACK, PGSIZE, (uint64)pstac
    k, PTE_R | PTE_W) != 0)
        goto fail;

    return kpt;

fail:
    kvmfree(kpt, 1);
    return NULL;
}

```

因此，在 `kpagetable` 中，第一部分是所有进程共享的内核态代码的页表项（由 `memmove` 直接复制），第二部分才是给这个内核态开始映射一段内核栈等等。

而在 `allocproc` 时的用户态页表，创建的过程就是 `proc.c` 定义的这个函数实现的：

```

// Create a user page table for a given process,
// with no user memory, but with trampoline pages.
pagetable_t
proc_pagetable(struct proc *p)

```

```

{
    pagetable_t pagetable;

    // An empty page table.
    // 这块跟proc_kpagetable其实都是kalloc了一个新页作为
    页表
    pagetable = uvmcreate();
    if(pagetable == 0)
        return NULL;

    // map the trampoline code (for system call return)
    // at the highest user virtual address.
    // only the supervisor uses it, on the way
    // to/from user space, so not PTE_U.
    // 将trampoline映射到用户态内存空间的最顶端TRAMPOLINE
    处
    if(mappages(pagetable, TRAMPOLINE, PGSIZE,
                (uint64)trampoline, PTE_R | PTE_X) <
0){
        uvmfree(pagetable, 0);
        return NULL;
    }

    // 为trapframe创建页表项，映射到
    // map the trapframe just below TRAMPOLINE, for
    trampoline.S.
    if(mappages(pagetable, TRAPFRAME, PGSIZE,
                (uint64)(p->trapframe), PTE_R | PTE_
W) < 0){
        vmunmap(pagetable, TRAMPOLINE, 1, 0);
        uvmfree(pagetable, 0);
        return NULL;
    }

    return pagetable;
}

```

注意：其实 `mappages` 做的事情就是把给的物理地址与 `flags` 组合，并从第一个参数 `pagetable`（一般来说就是一级页表基址）开始，`walk` 到一个由第二个参数 `va` 解码出来的 `pte`，并修改 `pte` 的内容，这就是映射的过程。

- 不过在我们的cow中，完全没有必要对内核这部分的映射关系做调整，像助教说的，我们只需要实现对数据页面的Copy-on-Write就能节省fork时的页面复制开销，通过测试样例。
- 接下来我们对 `fork` 下手。在原版的 `fork` 中，会先调用 `allocproc`，为子进程分配一个空白的 `proc`，并且给它 `kalloc` 一个 `trapframe` 页、一个用户态页表以及一个内核态页表，这部分我们不需要改动。在接下来，进行数据页面的复制时，引发了这个函数

```
// Given a parent process's page table, copy
// its memory into a child's page table.
// Copies both the page table and the
// physical memory.
// returns 0 on success, -1 on failure.
// frees any allocated pages on failure.
int
uvmcopy(pagetable_t old, pagetable_t new, pagetable_t
knew, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i = 0, ki = 0;
    uint flags;
    char *mem;

    while (i < sz){
        if((pte = walk(old, i, 0)) == NULL)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == NULL)
            goto err;
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(new, i, PGSIZE, (uint64)mem, flags) !
= 0) {
```

```

        kfree(mem);
        goto err;
    }
    i += PGSIZE;
    if(mappages(knew, ki, PGSIZE, (uint64)mem, flags
& ~PTE_U) != 0){
        goto err;
    }
    ki += PGSIZE;
}
return 0;

err:
vmunmap(knew, 0, ki / PGSIZE, 0);
vmunmap(new, 0, i / PGSIZE, 1);
return -1;
}

```

它会从旧页表基地址（父进程的用户态一级页表基地址）开始 `walk`，找到 `sz` 页数量内每一页对应 `pte`，并转化为真实的物理页地址；随后用 `mem` 进行新物理页的分配，然后 `memmove` 复制进新的页；这时 `mem` 是新分配的真实物理页的物理地址，`mappages` 会把 `mem` 映射到传入的第二个参数 `new`（子进程的用户态一级页表基地址，这个成员变量是一个 `uint64` 的指针，被定义在 `proc` 中，并被 `allocproc` 初始化好了），同时还会映射一次到 `knew`（子进程内核态一级页表基址）。

所以修改这个逻辑，就只需要让原先的映射逻辑由“将新的物理页映射到子进程页表”改为“将父进程的物理页映射过去”，同时，记得进行 `PTE_W` 与 `PTE_COW` 的设置。

```

int uvmcopy(pagetable_t old, pagetable_t new, pagetable_t knew, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i = 0, ki = 0;
    uint flags;

    while (i < sz)
    {

```

```

    if ((pte = walk(old, i, 0)) == NULL)
        panic("uvmcopy: pte should exist");
    if ((*pte & PTE_V) == 0)
        panic("uvmcopy: page not present");
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    if (flags & PTE_W)
    {
        flags &= ~PTE_W; // remove write permission
        flags |= PTE_COW; // set COW flag
        *pte |= flags;    // update parent's PTE
    }
    if (mappages(new, i, PGSIZE, (uint64)pa, flags) != 0)
    {
        goto err;
    }
    i += PGSIZE;
    if (mappages(knew, ki, PGSIZE, (uint64)pa, flags & ~PTE_U) != 0)
    {
        goto err;
    }
    ki += PGSIZE;
    kaddref((void *)pa); // 增加物理页的引用计数
}
return 0;

err:
    vmunmap(knew, 0, ki / PGSIZE, 0);
    vmunmap(new, 0, i / PGSIZE, 1);
    return -1;
}

```

- 接下来我们进行 `trap.c` 的处理，在现在的cow机制中，如果我们对页面进行读是没问题，但是写的时候会触发缺页异常，这时候需要我们把 `cow_handler` 的相关内容实现

```

void usertrap(void)
{
    ...
    else if ((which_dev = devintr()) != 0)
    {
        // ok
    }
    else if (r_scause() == 15)
    {
        if (cow_handler(p, r_stval()) != 0)
        {
            p->killed = 1;
        }
    }
    else
    {
        ...
    }
}

```

- 接下来我们看看cow_handler应该怎么实现

```

int cow_handler(struct proc *p, uint64 va)
{
    // 首先检查引发缺页异常的页是否合法
    va = PGROUNDDOWN(va);
    if (va >= p->sz)
        return -1;

    // 找到引发缺页的对应pte并检查权限
    pte_t *pte = walk(p->pagetable, va, 0);
    if (pte == 0)
        return -1;
    if ((*pte & PTE_V) == 0)
        return -1;
    if ((*pte & PTE_U) == 0)
        return -1;
    // 这说明并非由COW引起，那就是纯纯的越权
    if ((*pte & PTE_COW) == 0)
        return -1;
}

```

```
uint64 pa = PTE2PA(*pte);
uint flags = PTE_FLAGS(*pte);
```

// 这一步是检查，如果子进程都结束了对该页面的cow映射，那该页面被引量是1

// 此时应该完成该进程内核态和用户态页面PTE_W的恢复工作，否则在最后一个测试点不能通过

```
uint64 ref_cnt = kgetref((void *)pa);
if (ref_cnt == 1)
{
    flags = (flags & ~PTE_COW) | PTE_W;
    *pte = PA2PTE(pa) | flags;

    pte_t *kpte = walk(p->kpagetable, va, 0);
    if (kpte)
    {
        *kpte = PA2PTE(pa) | (flags & ~PTE_U);
    }
    sfence_vma();
    return 0;
}
```

// 不然就是真的需要复制到新的页面

```
char *mem = kalloc();
if (mem == 0)
    return -1;
```

// 进行内容的复制，将父进程物理页的内容完全复制到新开辟的mem中

```
memmove(mem, (char *)pa, PGSIZE);
```

// 恢复页面的PTE_W权限，不必使用mappages重新建立映射，直接修改pte的权限就行

```
flags = (flags & ~PTE_COW) | PTE_W;
*pte = PA2PTE(mem) | flags;

pte_t *kpte = walk(p->kpagetable, va, 0);
if (kpte)
{
```

```

    *kpte = PA2PTE(mem) | (flags & ~PTE_U);
}

// 这里其实并没有把父进程的那个页面直接释放掉，如果还有别的
// 进程仍然在cow映射
// 则这个调用只能起到减少ref的作用
kfree((void *)pa);

sfence_vma();

return 0;
}

```

其实这个调用中，很多的函数如 `walk` 等等，如果我们直接放在 `trap.c` 里，是需要补充一大堆引用的，所以我们直接放在 `vm.c` 里即可，并且记住在 `vm.h` 中添加一个函数声明。

- 最后大功告成，运行 `make run_test`，可以看到现在 `fork` 以后，比实现cow之前少了4页，而这4页就是我们之前得到的父进程的size，也就是说我们的cow确实将子进程的那些页面直接映射到父进程那些数据页面的物理页上，而不是分配一个新的物理页了。

lab6

要求：在原先os-part4仓库中完成内存相关调用 `test_vm_fifo` `test_vm_lru`

▼ 前置操作

- 运行这个指令，切换到part6的分支上，并在这个分支上创建一个新的分支

```
git checkout part6-page-replace
```

- 看助教的参考步骤，我们知道在前两步，两个测试点要做的事情都一样，所以我们不必现在就进行分支，在这个part6的主分支上，将步骤一和步骤二完成。

▼ 步骤一：

- 实现 `set_max_page_in_mem`、`get_page_swap_count` 两个系统调用：在进程管理结构里添加相应字段，其中 `max_page_in_mem` 用于限制单个进程最多能使用的mmap映射区域的页面数，而 `page_swap_count` 用于记录进程在mmap映射区域发生的换出行为次数。（具体逻辑可以看测试样例的注释说明）

▼ set_max_page_in_mem：

- 这是一个系统调用，因此我们仍需要再 `syscall.c` 中对齐进行注册，接下来，我们要在 `sysproc.c` 中完成它的定义。
- 这个调用用于限制单个进程最多可以使用的 `mmap` 映射区域，超过了这个数量时，则触发页面调换。既然涉及到了进程的内容，我们去 `proc.h` 中，查看一下有关进程和 `mmap` 的定义情况。

```
#define MAX_VMA 16 // 每个进程最多16个映射区域

struct VMA {
    uint64 vm_start;
    uint64 vm_end;
    int prot;
    int flags;
    uint64 vm_off;
    struct VMA *vm_next, *vm_prev;
};
```

```

. . .

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    struct proc *parent;           // Parent process
    void *chan;                     // If non-zero,
    sleeping on chan                // If non-zero,
    int killed;                     // If non-zero,
    have been killed                // Exit status to
    int xstate;                     // be returned to parent's wait
    int pid;                        // Process ID

    // these are private to the process, so p->lock
    // need not be held.
    uint64 kstack;                  // Virtual address
    of kernel stack                // Size of process
    uint64 sz;                      // memory (bytes)
    pagetable_t pagetable;          // User page table
    pagetable_t kpagetable;         // Kernel page table
    struct trapframe *trapframe;    // data page for
    trampoline.S                   // switch() here
    struct context context;          // to run process
    struct file *ofile[NFILE];      // Open files
    struct dirent *cwd;              // Current directory
    char name[16];                  // Process name
    (debugging)                     // trace mask
    int tmask;

```

```

    struct VMA head;
};

```

这里我们发现，在最开始，定义的vma最大数量是一个宏，这个我们没法修改，但事实上我们这里需要实现的是一个新的变量，它用于记录当前内存中最多可以使用多少个vma，所以接下来我们就必须给 `struct proc` 自己添加一个额外的变量 `int max_page_in_mem`，同时要给出一个记录目前有多少个vma在内存中的变量 `int cur_page_in_mem`，并且需要我们自己追踪在进程全过程中的它的周期。

进入 `proc.c`，我们寻找这个变量的全周期。（不过如果我们直接阅读测试样例，你就发现其实做这个变量的管理也没那么关键，因为不涉及 `fork` 等操作，所以接下来的很多操作只是出于健壮考虑）

首先应该是进程创建时，对这个变量进行初始化。

```

static struct proc *
allocproc(void){
    ...
    p->kstack = VKSTACK;
    p->max_page_in_mem = MAX_VMA;
    p->cur_page_in_mem = 0;
    p->head.vm_prev = &p->head;
    p->head.vm_next = &p->head;
    ...
}

```

其次是在回收进程时将其清空。

```

static void
freeproc(struct proc *p)
{
    ...
    p->pid = 0;
    p->parent = 0;
    p->max_page_in_mem = 0;
    p->cur_page_in_mem = 0;
    p->name[0] = 0;
}

```

```
...  
}
```

然后是在fork时的复制。

```
// Create a new process, copying the parent.  
// Sets up child kernel stack to return as if from  
// fork() system call.  
int fork(void)  
{  
    ...  
  
    np->parent = p;  
  
    // copy tracing mask from parent.  
    np->tmask = p->tmask;  
  
    np->max_page_in_mem = p->max_page_in_mem;  
    np->cur_page_in_mem = p->cur_page_in_mem;  
  
    ...  
}
```

随后我们到测试样例中去看一下这个函数在用户态是怎么被定义的，这能告诉我们应该如何传递参数。

```
set_max_page_in_mem(4);
```

好，接下来我们只需要在 `sysproc.c` 中添加这个简单的系统调用即可。

```
uint64  
sys_set_max_page_in_mem()  
{  
    int n;  
    if (argint(0, &n) < 0)  
    {  
        return -1;  
    }  
}
```

```
myproc()->max_page_in_mem = n;
return 0;
}
```

▼ get_page_swap_count:

- 同样地，我们需要在进程的全周期添加一个新的变量 `int page_swap_count` 用于记录该进程中发生的所有的页面换出行为。记得在 `syscall.c` 中添加这个 `sys_get_page_swap_count` 的注册内容。
- 但是你要记得，在fork时，不应该将父进程的页面换出数量加进来，所以这块这么写：`np->page_swap_count = 0;`

```
uint64
sys_get_swap_count()
{
    return myproc()->page_swap_count;
}
```

▼ 步骤二:

- 由于真实交换区可能位于磁盘上，涉及到文件操作，较为麻烦，因此本实验中可以使用**内存里模拟的交换区**进行替代。实现内存里模拟的交换区，用于存放**各个进程mmap映射区域换出的页面**，在进程需要访问交换区里的页面时，进行**换入操作**。
 1. 全局交换区可参考数据结构设计：由全局数组/链表结构组织的**slot**，每个slot存放一个 `PAGESZ` 大小的数据，并维护当前slot的状态（已使用/未使用）、数据的所有者PID、数据对应的起始地址addr等。
 - a. `alloc_global_swap_slot`：从全局交换区里分配一个空闲的slot（用于之后换出页面内容）
 - b. `free_global_swap_slot`：将使用完毕的slot归还至全局交换区（换入内容后释放资源）
 2. 对进程 `mmap` 区域的每个页面，维护**标志位**表明页面**是否处于交换区**（若处于交换区，则认为是内存不够发生了换出），可以考虑修改 `struct VMA` 结构体以维护**mmap区域对应的各个Page的信息**：Page的起始地址，Page当前的状态（在交换区/在内存/未使用），Page的末次访问时间（用于LRU）和本次进入内存时间（用于FIFO），etc.；

- 参考这个要求，一个交换区是全局变量，这个交换区的每个结构体应该具有来源 `proc` 的 `PID` 等信息，因为其作为一个物理页面的模拟，它还需要能够完整记录一整个页面的数据，为了方便后续的管理，由于我们将真实交换区仍然模拟为虚拟内存中的结构，所以我们直接将其定义在 `vm.h` 下，方便这些虚存管理函数对齐操作。

这里的`alloc`与`free`我们可以参考 `allocproc` 和 `freeproc`，分配时返回一个结构体或全局交换区中的编号，释放时什么也不用返回。

```
void mock_swap_init(void);
int alloc_global_swap_slot(int pid, uint64 vaddr);
// 从全局交换区里分配一个空闲的slot（用于之后换出页面内容）
void free_global_swap_slot(int idx); // 将使用完毕
的slot归还至全局交换区（换入内容后释放资源）
struct swap_slot
{
    int used;           // 是否有效
    int pid;            // 进程ID
    uint64 vaddr;       // 该页面原本对应的虚拟地址
    char data[PGSIZE]; // 该页面数据的实际缓冲区
};
#define MAX_SWAP_SLOTS 10 // 由于测试样例只有8个页面
需要来回交换
```

因为我们的交换区是一个全局变量，可能涉及多个进程共享同一个区的现象，如果我们不能保证操作的原子性，可能出现两个进程分配到同一个 `slot` 槽位等竞态问题。所以在全局交换区变量的设计上，我们加一个锁吧。（如果我们只是为了通过测试点，这个锁应该也不用写，我看测试样例没有fork的问题）

```
struct
{
    struct spinlock lock;
    struct swap_slot slot[MAX_SWAP_SLOTS]; // 因为测试
样例只有8个页面，所以全局交换区设置为10个slot以防止越界
} mock_swap;

void mock_swap_init(void)
{
```

```

initlock(&mock_swap.lock, "mock_swap");
for (int i = 0; i < MAX_SWAP_SLOTS; i++)
{
    mock_swap.slot[i].used = 0;
    mock_swap.slot[i].pid = -1;
    mock_swap.slot[i].vaddr = 0;
    memset(mock_swap.slot[i].data, 0, PGSIZE);
}
}

```

接下来是助教给出的两个函数的定义，这两个函数作为该全局交换区的对外接口，管理这个交换区 `slot` 的分配与撤销。

```

int alloc_global_swap_slot(int pid, uint64 vaddr)
{
    acquire(&mock_swap.lock);
    for (int i = 0; i < MAX_SWAP_SLOTS; i++)
    {
        if (!mock_swap.slot[i].used)
        {
            mock_swap.slot[i].used = 1;
            mock_swap.slot[i].pid = pid;
            mock_swap.slot[i].vaddr = vaddr;
            release(&mock_swap.lock);
            return i;
        }
    }
    release(&mock_swap.lock);
    return -1; // 没有可用的slot
}

void free_global_swap_slot(int idx){
    acquire(&mock_swap.lock);
    if(idx >= 0 && idx < MAX_SWAP_SLOTS){
        mock_swap.slot[idx].used = 0;
        mock_swap.slot[idx].pid = -1;
        mock_swap.slot[idx].vaddr = 0;
        memset(mock_swap.slot[idx].data, 0, PGSIZE);
    }
}

```

```

    }
    release(&mock_swap.lock);
}

```

之后，像这种全局变量的初始化，我们把它放在main.c中即可。

```

void main(unsigned long hartid, unsigned long dtb_
pa)
{
    ...
    timerinit();    // init a lock for timer
    trapinithart(); // install kernel trap vector,
including interrupt handler
    procinit();
    mock_swap_init();
    plicinit();
    ...
}

```

- 之后，根据流程，我们需要在VMA中额外维护一个成员变量，用于展示这个 `mmap` 映射的页，在此之前，我们先看一下在这个仓库下的 `mmap` 是怎么被实现的。首先，`sys_mmap` 的作用只是解析参数，我们可以看到真正起作用的是 `mymmap`。

```

uint64 mymmap(int fd, uint64 addr, uint64 len, int
prot, int flags, uint64 offset)
{
    struct proc *p = myproc();
    struct VMA *vma;
    // mmap映射的区域从进程的堆顶向上找，并从最顶端的TRAMPOLINE|TRAPFRAME处向下
    uint64 start = p->sz, end = TRAPFRAME;
    if (start - end < len)
        addr = -1;
    // 找到目前该进程中mmap链表的最靠低地址处的头
    for (vma = p->head.vm_next; vma != &p->head; vma
= vma->vm_next)
    {

```



```

        if (end - vma->vm_end >= len)
            break;
        end = vma->vm_start;
    }
    // 看看我们希望分配的这个映射内存与目前堆顶到mmap链表底
    之间的空隙是否够大
    if (start - end < len)
        addr = -1;
    addr = end - len;
    // 这个调用实际上从全局的空闲vma池中分配出一个来接到当前
    进程的p->head下
    if ((vma = allocshare()) == 0)
        return -1;
    // 根据传入的参数初始化这个vma
    vma->vm_start = addr;
    vma->vm_end = vma->vm_start + len;
    vma->prot = prot;
    vma->flags = flags;
    vma->vm_off = offset;

    // 挂载到当前进程的head尾部前一块
    struct VMA *cnt;
    for (cnt = p->head.vm_next; cnt != &p->head; cnt
    = cnt->vm_next)
        if (cnt->vm_start < vma->vm_start)
            break;
    cnt->vm_prev->vm_next = vma;
    vma->vm_prev = cnt->vm_prev;
    vma->vm_next = cnt;
    cnt->vm_prev = vma;

    return addr;
}

```

可以看到这个调研完全没有考虑到任何分页相关的内容，它只是给vma设置好了开始结束地址，但我们在 `mmap` 之后，根据要求，需要对 `mmap` 的地方进行“页”级别的置换。也就是说我们应该在这个调用中，对我们的成员变量做维护。

先定义好这个新的结构。

```
struct VMA_page
{
    int status; // 0: not used, 1: in memory, 2: swa
    pped out
    uint64 vaddr;
    int swap_slot_idx; // 如果status为2, 记录对应
    的全局交换区
    uint64 last_in_mem_time; // 该页面进入内存的时
    间戳, 用于FIFO算法
    uint64 last_access_time; // 该页面最后一次被访问的时
    间戳, 用于LRU算法
};

struct VMA
{
    uint64 vm_start;
    uint64 vm_end;
    int prot;
    int flags;
    uint64 vm_off;
    struct VMA_page pages[10]; // 该映射区域内的页面信
    息, 测试样例就8页
    struct VMA *vm_next, *vm_prev;
};
```

随后我们处理一下这个新成员变量的全周期, 只考虑通过测试点, 我们只在 `mmap` 发生时对其进行初始化操作。

```
uint64 mymmap(int fd, uint64 addr, uint64 len, int
prot, int flags, uint64 offset)
{
    ...
    if ((vma = allocshare()) == 0)
        return -1;
    vma->vm_start = addr;
    vma->vm_end = vma->vm_start + len;
```

```

vma->prot = prot;
vma->flags = flags;
vma->vm_off = offset;

// 这块用于处理VMA_page的初始化
int npages = (len + PGSIZE - 1) / PGSIZE; // 向上取整计算需要的页面数
for (int i = 0; i < npages; i++)
{
    vma->pages[i].status = 0; // 初始状态为not used
    vma->pages[i].vaddr = vma->vm_start + i * PGSIZE;
    vma->pages[i].swap_slot_idx = -1; // 初始时没有对应的交换区slot
    vma->pages[i].last_in_mem_time = 0;
    vma->pages[i].last_access_time = 0;
}

...
}

```

▼ 步骤三：

- 接下来是实现swap。
 1. `swap_out`：将mmap映射区的页面内容拷贝到交换区的桶内，（并释放被拷贝的页面内存）
 2. `swap_in`：将交换区桶内的页面内容拷贝到进程mmap映射区指定页面上，（并预先分配内存）
- 这两个调用应该是处理缺页异常之后的操作，但应该放在 `vm.c` 中，因为我们的全局交换区也在这里定义，所以我们在 `vm.h` 中给出两个函数的定义，在 `trap.c` 中直接使用即可。

这两个函数可以参考 `uvmmalloc`，而对于 `swap_out`，由于 `vmunmap` 中已经将对应 `pte` 设为0，所以就不用我们再手动进行valid位清除。但不考虑页面的权限了，在测试样例中没有涉及太多考虑。

```

int swap_out(struct proc *p, struct VMA_page *page)

```

```

{
    int idx = alloc_global_swap_slot(p->pid, page->v
addr);
    if (idx < 0)
    {
        return -1;
    }
    // 找到该换出页的PTE，获取物理地址，复制到交换区
    pte_t *pte = walk(p->pagetable, page->vaddr, 0);
    if (pte == NULL || (*pte & PTE_V) == 0)
    {
        return -1;
    }
    uint64 pa = PTE2PA(*pte);
    memmove(mock_swap.slot[idx].data, (char *)pa, PG
SIZE);

    // 解除映射，更新PTE和页表项状态
    if (walk(p->kpagetable, page->vaddr, 0) != NULL)
    {
        vmunmap(p->kpagetable, page->vaddr, 1, 0); //
这一步时先不释放物理页，只是解映射内核页表
    }
    vmunmap(p->pagetable, page->vaddr, 1, 1);
    page->status = 2; // 标
记为交换出
    page->swap_slot_idx = idx; // 记
录交换区slot索引

    p->cur_page_in_mem--;
    p->page_swap_count++;
    return 0;
}

int swap_in(struct proc *p, struct VMA_page *page)
{
    if (page->swap_slot_idx < 0 || page->swap_slot_i
dx >= MAX_SWAP_SLOTS)

```

```

{
    return -1;
}
// 从交换区复制回物理内存，更新PTE和页表项状态
char *mem = kalloc();
if (mem == NULL)
{
    return -1;
}
memmove(mem, mock_swap.slot[page->swap_slot_idx].data, PGSIZE);
if (mappages(p->pagetable, page->vaddr, PGSIZE,
(uint64)mem, PTE_W | PTE_R | PTE_U) != 0)
{
    kfree(mem);
    return -1;
}
if (mappages(p->kpagetable, page->vaddr, PGSIZE,
(uint64)mem, PTE_W | PTE_R) != 0)
{
    vmunmap(p->pagetable, page->vaddr, 1, 1);
    kfree(mem);
    return -1;
}

int idx = page->swap_slot_idx;
free_global_swap_slot(idx); // 释放交换区slot
page->status = 1;           // 标记为在内存中
page->swap_slot_idx = -1;   // 清除交换区slot索引

p->cur_page_in_mem++;
return 0;
}

```

在 `vm.h` 中添加定义，并加上。

```

struct proc;
struct VMA_page;

```

- 之后，我们进行缺页异常相关的处理。这边的逻辑是这样：`mmap`了8个页面以后，第一次试图写入一定出现缺页异常，此时会进行下面的流程

```
void
usertrap(void)
{
    ...
    else if (r_scause() == 13 || r_scause() == 15) {
        uint64 va = r_stval();
        struct proc *p = myproc();
        va = PGROUNDDOWN(va);
        struct VMA *vma;
        // 在p的vma链表中寻找这一缺页地址的对应vma结构体
        for(vma = p->head.vm_next; vma != &p->head; vma = vma->vm_next)
            if(va >= vma->vm_start && va < vma->vm_end)
                break;
        if(vma == &p->head)
        {
            p->killed = 1;
            exit(-1);
        }
        // 只是未分配物理页，则直接分配一个物理页
        char *mem;
        if((mem = kalloc()) == 0)
        {
            p->killed = 1;
            exit(-1);
        }
        memset(mem, 0, PGSIZE);
        uint64 pa = (uint64)mem; //提取kalloc出的mem的物理地址

        int perm = PTE_U;
        if (vma->prot & PROT_READ) perm |= PTE_R;
        if(vma->prot & PROT_WRITE) perm |= PTE_W;
        if(vma->prot & PROT_EXEC) perm |= PTE_X;
        //在用户页表上添加
```

```

        if(mappages(p->pagetable, va, PGSIZE, pa, per
m) != 0)
        {
            kfree(mem);
            exit(-1);
        }
        if(mappages(p->kpagetable, va, PGSIZE, pa, per
m & ~PTE_U) != 0)
        {
            vmunmap(p->pagetable, va, 1, 0);
            kfree(mem);
            exit(-1);
        }
    }
    ...
}

```

会首先查vma表，确认这一次缺页异常不是非法访问，而是由于vma未分配物理页导致的（注意，之前调用 `sys_mmap` 时，`mymmap` 已经从空闲vma链表中拿出一个交给该进程 `p->head` 下挂载了）；之后，会用 `mappages` 查询用户页表上，该引发异常的虚拟地址对应的 `pte` 是否可用，若 `v=0`，则将 `mem` 的物理地址添加权限后，直接修改该 `pte` 内容。

这样就完成了这次缺页异常对应的mmap页的物理绑定。

在刚开始，不涉及到超出内存中可用mmap页时，引发缺页异常都应该这样做，但在实际情况中，由于物理内存（DRAM，也就是分配的物理页所在的硬件中）大小有限，需要将多余的页换入磁盘中。这就是说，我们刚设计的全局交换区实际上在模拟磁盘在这里的作用。

因此需要修改 `usertrap` 的逻辑，使得：

1. 每次由于mmap引发的缺页异常，在需要分配物理页之前检查该进程当前在内存中的物理页数是否达到了上限。
2. 达到上限时，要进行换出页面的选择，执行换出操作。
3. 判断缺页地址从属哪一个页，这个页的status为0（未进入过内存）还是2（在交换区内，需要进入内存）

```

void usertrap(void)
{

```

```

...
else if (r_scause() == 13 || r_scause() == 15)
{
    uint64 va = r_stval();
    struct proc *p = myproc();
    va = PGROUNDDOWN(va);
    struct VMA *vma;
    for (vma = p->head.vm_next; vma != &p->head; v
ma = vma->vm_next)
        if (va >= vma->vm_start && va < vma->vm_end)
            break;
    if (vma == &p->head)
    {
        p->killed = 1;
        exit(-1);
    }
    struct VMA_page *page = addr2page(&p->head, v
a);
    if (page == 0)
    {
        p->killed = 1;
        exit(-1);
    }
    if (p->cur_page_in_mem >= p->max_page_in_mem)
    {
        struct VMA_page *victim = 0;
        victim = select_victim_page(&p->head);
        if (swap_out(p, victim) < 0)
        {
            p->killed = 1;
            exit(-1);
        }
    }

    if (page->status == 2)
    {
        // 页面在交换区内，需要换入到物理内存
        if (swap_in(p, page) < 0) // 这一步自动完成了cu

```


r_page_in_mem++的操作

```
{
    p->killed = 1;
    exit(-1);
}
}
else if (page->status == 0)
{
    // 页面尚未使用过, 执行全新分配
    char *mem;
    if ((mem = kalloc()) == 0)
    {
        p->killed = 1;
        exit(-1);
    }
    memset(mem, 0, PGSIZE);
    uint64 pa = (uint64)mem; // 提取kalloc出的mem
    的物理地址

    int perm = PTE_U;
    if (vma->prot & PROT_READ)
        perm |= PTE_R;
    if (vma->prot & PROT_WRITE)
        perm |= PTE_W;
    if (vma->prot & PROT_EXEC)
        perm |= PTE_X;
    // 在用户页表上添加
    if (mappages(p->pagetable, va, PGSIZE, pa, p
    erm) != 0)
    {
        kfree(mem);
        exit(-1);
    }
    if (mappages(p->kpagetable, va, PGSIZE, pa,
    perm & ~PTE_U) != 0)
    {
        vmunmap(p->pagetable, va, 1, 0);
        kfree(mem);
    }
}
```

```

        exit(-1);
    }
    p->cur_page_in_mem++;
    page->status = 1;           // 标记为在内存中
}
}
...
}

```

这里用到了一个 `addr2page`，在 `proc.c` 中定义的。

```

struct VMA_page *addr2page(struct VMA *head, uint64 addr)
{
    struct VMA *vma;
    for (vma = head->vm_next; vma != head; vma = vma->vm_next)
    {
        if (addr >= vma->vm_start && addr < vma->vm_end)
        {
            return &vma->pages[(addr - vma->vm_start) / PGSIZE];
        }
    }
    return 0;
}

```

然后这块的 `select_victim_page` 是一个简单的按照页面顺序挑选的占位内容，目的是检查现在所有的其他逻辑是否正确。

```

struct VMA_page *select_victim_page(struct VMA *head)
{
    struct VMA *v;
    struct VMA_page *victim = 0;

    // 遍历进程内部环形双向链表上挂载的所有 VMA 块

```

```

    for (v = head->vm_next; v != head; v = v->vm_next)
    {
        int npages = (v->vm_end - v->vm_start + PGSIZE - 1) / PGSIZE;

        // 遍历每一个 VMA 内部对应的物理页面信息
        for (int i = 0; i < npages; i++)
        {
            // 只有真正在内存中的页 (status == 1) 才能作为候选者
            if (v->pages[i].status == 1)
            {
                victim = &v->pages[i];
                printf("换出页面: %d\n", i);
                break;
            }
        }
    }

    if (victim == 0)
        panic("select_victim_page: no valid victim found");

    return victim;
};

```

▼ 现在这样实现以后，按照fifo测试样例的访问顺序要求，最终会有下面的结果，换出页面数为10次，和我们推测的顺序一样，说明其他逻辑都OK，接下来按照不同的测试点要求修改 `select_victim_page` 就行

访问页面	访问后内存状态	换出页面	是否换出
0	[0]	N	N
1	[0, 1]	N	N
2	[0, 1, 2]	N	N
3	[0, 1, 2, 3]	N	N
0	[0, 1, 2, 3]	N	N

1	[0, 1, 2, 3]	N	N
4	[1, 2, 3, 4]	0	Y
5	[2, 3, 4, 5]	1	Y
0	[0, 3, 4, 5]	2	Y
1	[1, 3, 4, 5]	0	Y
6	[3, 4, 5, 6]	1	Y
7	[4, 5, 6, 7]	3	Y
0	[0, 5, 6, 7]	4	Y
1	[1, 5, 6, 7]	0	Y
2	[2, 5, 6, 7]	1	Y
3	[3, 5, 6, 7]	2	Y

- `git push` 一下，然后在这个分支基础上去创建fifo或者LRU的branch。

▼ test_vm_fifo:

- 由于之前在 `proc.h` 中，为每一页已经实现了这个变量。

```
uint64 last_in_mem_time;    // 该页面进入内存的时间戳，用于FIFO算法
```

因此我们只需要在trap.c中，处理每一页的换入时加上这个变量的维护即可。而关于时间的定义，我们可以直接使用ticks。

```
void usertrap()
{
    ...
    p->cur_page_in_mem++;
    page->status = 1; // 标记为在内存中
}
acquire(&tickslock);
page->last_in_mem_time = ticks; // 更新进入内存时间戳
release(&tickslock);
...
}
```

随后，在select_victim_page中添加这个时间判断的逻辑就OK。

```
struct VMA_page *select_victim_page(struct VMA *head)
{
    struct VMA *v;
    struct VMA_page *victim = 0;
    int oldest_time = 0x7FFFFFFF; // 初始化为最大值
    int selected = -1;

    // 遍历进程内部环形双向链表上挂载的所有 VMA 块
    for (v = head->vm_next; v != head; v = v->vm_next)
    {
        int npages = (v->vm_end - v->vm_start + PGSIZE -
1) / PGSIZE;

        // 遍历每一个 VMA 内部对应的物理页面信息
        for (int i = 0; i < npages; i++)
        {
            // 只有真正在内存中的页 (status == 1) 才能作为候选者
            if (v->pages[i].status == 1)
            {
                if (v->pages[i].last_in_mem_time < oldest_time)
                {
                    oldest_time = v->pages[i].last_in_mem_time;
                    victim = &v->pages[i]; // 更新当前最老的页面指针
                    selected = i;
                    // printf("换出页面: %d\n", i);
                }
            }
        }
    }

    printf("换出页面: %d\n", selected);
    if (victim == 0)
        panic("select_victim_page: no valid victim found");
}
```

```
return victim;
};
```

▼ test_vm_lru:

- 首先需要实现一个系统调用 `#define SYS_lru_access_notify 54334`，这在测试样例中，是每次写入页面后调用的，这个是用来告诉内核，我最后一次访问的时间是这个时候。也就是说，我们可以用这个系统调用来维护。

```
uint64 last_access_time; // 该页面最后一次被访问的时间戳，
用于LRU算法
```

参考测试样例中的调用格式。

```
for (int i = 0; i < pattern_length; i++) {
    int page_index = access_pattern[i];
    mem[page_index * PAGE_SIZE] = 'A' + page_index;
    lru_access_notify((uint64)&mem[page_index * PAGE_SIZE]);
    // notify kernel the access action
    printf("Accessed page %d\n", page_index);
    sleep(1);
}
```

需要传递一个参数，这个参数指向了mmap某个页面的首地址。

```
uint64
sys_lru_access_notify()
{
    uint64 va;
    if (argaddr(0, &va) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
    struct VMA_page *page = addr2page(&p->head, va);
    if (page == 0)
    {
```

```

        return -1;
    }
    acquire(&tickslock);
    page->last_access_time = ticks; // 更新页面的最后访问时间戳
    release(&tickslock);
    return 0;
}

```

接下来，只需要对 `select_victim_page` 做修改就可以了。

```

struct VMA_page *select_victim_page(struct VMA *head)
{
    struct VMA *v;
    struct VMA_page *victim = 0;
    int oldest_time = 0x7fffffff; // 初始为一个很大的数，确保任何页面的时间戳都比它小

    // 遍历进程内部环形双向链表上挂载的所有 VMA 块
    for (v = head->vm_next; v != head; v = v->vm_next)
    {
        int npages = (v->vm_end - v->vm_start + PGSIZE - 1) / PGSIZE;

        // 遍历每一个 VMA 内部对应的物理页面信息
        for (int i = 0; i < npages; i++)
        {
            // 只有真正在内存中的页 (status == 1) 才能作为候选者
            if (v->pages[i].status == 1)
            {
                if (v->pages[i].last_access_time < oldest_time)
                {
                    oldest_time = v->pages[i].last_access_time;
                    victim = &v->pages[i];
                }
            }
        }
    }
}

```

```
}  
if (victim == 0)  
    panic("select_victim_page: no valid victim found");  
  
return victim;  
};
```


lab7

要求：实现这些测试样例：

`test_dup`、`test_dup2`、`test_pipe`、`test_open`、`test_openat`、`test_close`、
`test_getdents`、`test_read`、`test_mkdir`、`test_chdir`、`test_unlink`、`test_mount`、
`test_umount`、`test_fstat`

之前在lab3已经顺便实现了很多调用，因此这里只剩这些测试样例：

`test_dup`、`test_dup2`、`test_pipe`、`test_getdents`、`test_mkdir`、`test_chdir`、
`test_unlink`、`test_mount`、`test_umount`

▼ 前置操作：

- 添加这几个测试点

```
char *tests[] = {  
  
    // lab7  
    "dup",  
    "dup2",  
    "pipe",  
    "getdents",  
    "mkdir_", // 注意这里有一个_!!!!  
    "chdir",  
    "unlink",  
    "mount",  
    "umount",  
};
```

- 执行make fast_renew以后，把未实现的调用号先进行注册，相应的系统调用则先用return 0占位以确保可以通过编译。

```
#define SYS_chdir 49 // 把之前已有的进行修改  
...  
#define SYS_dup3 24  
#define SYS_mkdirat 34  
#define SYS_getdents 61
```

```
#define SYS_unlink 35
#define SYS_mount 40
#define SYS_umount 39
```

▼ dup

- 默认实现好了 `dup`，只需要对应注册，以及在 `init.c` 中添加测试点即可通过。

▼ dup2

- 看一下要做什么

```
void test_dup2(){
    TEST_START(__func__);
    int fd = dup2(STDOUT, 100);
    assert(fd != -1);
    const char *str = "    from fd 100\n";
    write(100, str, strlen(str));
    TEST_END(__func__);
}
```

在帮助文档中https://github.com/oscomp/testsuite-for-oskernel/blob/main/oscomp_syscalls.md，我们知道，这里的 `dup2` 测试样例会传递 `dup3` 调用号，然后进行 `sys_dup3` 的系统调用，而这个调用相比之前的 `dup`，则实现了，将第一个参数的文件复制到指定文件描述符的功能。

- 看看原版 `dup` 是怎么做的

```
uint64
sys_dup(void)
{
    struct file *f;
    int fd;

    if (argfd(0, 0, &f) < 0)
        return -1;
    if ((fd = fdalloc(f)) < 0)
        return -1;
    filedup(f);
}
```

```

return fd;
}

```

首先将传入的第一个参数当做文件描述符来解析，用 `argfd` 直接将解析完的指向真实文件的指针传到第三个参数里，随后用 `fdalloc`，在当前进程中的 `p->ofile[]` 找一个空闲的位置安放这个文件指针，并返回 `ofile` 的索引作为新的文件描述符，最后为 `f` 这个文件增加一次引用。

- 因此在原版 `dup` 的基础上，我们只需要将 `fdalloc` 干的事情变成直接指向存放即可，因此我们不能再直接调用 `fdalloc` 了。

```

// Allocate a file descriptor for the given file.
// Takes over file reference from caller on success.
static int
fdalloc(struct file *f)
{
    int fd;
    struct proc *p = myproc();

    for (fd = 0; fd < NOFILE; fd++)
    {
        if (p->ofile[fd] == 0)
        {
            p->ofile[fd] = f;
            return fd;
        }
    }
    return -1;
}

```

但是你注意到，原版的 `fdalloc` 中，所有文件描述符会有一个上限 `NOFILE`，因此在该进程的所有文件描述符，都不可能超过这个 `NOFILE`。在测试样例中，传入了100这个文件描述符作为指定，但 `NOFILE` 却只有16，如果我们直接做，就会失败。所以在 `param.h` 中，我们需要修改这个宏定义：`#define NOFILE 110`，之后，再直接写好 `sys_dup3` 即可

```

uint64
sys_dup3(void)
{

```

```

struct file *f;
int oldfd, newfd;

if (argfd(0, 0, &f) < 0)
    return -1;
if (argint(1, &newfd) < 0)
    return -1;
if (argint(0, &oldfd) < 0)
    return -1;
struct proc *p = myproc();
if (newfd < 0 || newfd >= NOFILE || p->ofile[newfd]
    != NULL)
    return -1;
p->ofile[newfd] = f;
filedup(f);
return newfd;
}

```

▼ pipe

- 添加测试点后即可直接通过。

▼ getdents

- 看看要做什么吧

```

char buf[512];
void test_getdents(void){
    TEST_START(__func__);
    int fd, nread;
    struct linux_dirent64 *dirp64;
    dirp64 = buf;
    //fd = open(".", O_DIRECTORY);
    fd = open(".", O_RDONLY);
    printf("open fd:%d\n", fd);

    nread = getdents(fd, dirp64, 512);
    printf("getdents fd:%d\n", nread);
    assert(nread != -1);
    printf("getdents success.\n%s\n", dirp64->d_nam

```

```

e);

    /*
    for(int bpos = 0; bpos < nread;){
        d = (struct dirent *)(buf + bpos);
        printf( "%s\t", d->d_name);
        bpos += d->d_reclen;
    }
    */

    printf("\n");
    close(fd);
    TEST_END(__func__);
}

```

根据文档的描述，这里调用 `getdents` 传入的第二个参数 `dirp64` 是这样的一个结构，以 `linux_dirent64` 结构的指针解析 `buf` 这段空数组

```

struct dirent {
    uint64 d_ino;    // 索引结点号
    int64 d_off;     // 到下一个dirent的偏移
    unsigned short d_reclen;    // 当前dirent的长度
    unsigned char d_type;    // 文件类型
    char d_name[];    // 文件名
};

```

成功执行之后，则会将读出的字节数返回给 `nread`，最后，用 `dirp64` 指针解析 `buf` 区，把 `d_name` 输出。

- 这里我们就需要了解一下内核中的 `dirent` 是什么，这个在lab3中有提到过一点，简短来说：`struct dirent` 实际上代表着在物理磁盘上，这个文件的真实情况，也就是说操作系统的全局中，它的信息只对应一个文件，这是唯一的，又称为这个文件的元数据，我们只是用 `struct file` 作为打开文件表，指向该物理数据。

在 `fat32.h` 中，我们阅读这个结构体的真实模样，发现它和要求传入的结构不同。

```

struct dirent {
    char filename[FAT32_MAX_FILENAME + 1];
};

```

```

uint8  attribute;
// uint8  create_time_tenth;
// uint16 create_time;
// uint16 create_date;
// uint16 last_access_date;
uint32 first_clus;
// uint16 last_write_time;
// uint16 last_write_date;
uint32 file_size;

uint32 cur_clus;
uint   clus_cnt;

/* for OS */
uint8  dev;
uint8  dirty;
short  valid;
int     ref;
uint32 off;           // offset in the parent d
ir entry, for writing convenience
    struct dirent *parent; // because FAT32 doesn't
have such thing like inum, use this for cache trick
    struct dirent *next;
    struct dirent *prev;
    struct sleeplock lock;
};

```

它的大小高达360Bytes，而我们需要的却是经过解析后的若干个

`linux_dirent64` 小结构体。这里添加一个新的定义。

```

struct linux_dirent64
{
    uint64 d_ino;
    uint64 d_off;
    unsigned short d_reclen;
    unsigned char d_type;
    char d_name[0];
};

```

- 事实上 `getdents` 的功能，就是遍历并读取一个打开的目录中的内容，并将目录项（文件、子目录等）的信息填充到用户态提供的缓冲区中。而这里的内容，则转化为上面这个小结构体作为存储办法，一点一点填充到 `buf` 中。

由于用户态的linux标准，需要使用VFS这一套标准逻辑，但在内核态中，却是FAT32的文件系统，它不存在 `d_ino`（inode）这种概念，因此要进行标准化处理，我们就需要引入 `getdents` 作为“欺上瞒下”的处理过程。

```
uint64
sys_getdents(void)
{
    struct file *f;
    uint64 buf;
    int len;

    if (argfd(0, 0, &f) < 0 || argaddr(1, &buf) < 0
    || argint(2, &len) < 0)
        return -1;

    if (f->readable == 0 || !(f->ep->attribute & ATT
R_DIRECTORY))
        return -1;

    struct dirent de;
    int count = 0;
    int ret;
    int nread = 0;
    struct linux_dirent64 lde;

    ...

    return nread;
}
```

开头我们先进行一些基本操作，把参数解析了，并且声明一些必要变量。这里的 `lde` 作为我们一步一步解析后的标准结构体，每次承担转化与转存到 `buf` 的功能，`nread` 则表示读取了多少byte。

- 接下来我们要进行解析的过程。这里我们要先看一个关键函数，它能帮我们在FAT32这个文件系统中解析 `dirent` 。

```
int enext(struct dirent *dp, struct dirent *ep, uint off, int *count)
{
    if (!(dp->attribute & ATTR_DIRECTORY))
        panic("enext not dir");
    if (ep->valid)
        panic("enext ep valid");
    if (off % 32)
        panic("enext not align");
    if (dp->valid != 1) { return -1; }

    union dentry de;
    int cnt = 0;
    memset(ep->filename, 0, FAT32_MAX_FILENAME + 1);

    for (int off2; (off2 = reloc_clus(dp, off, 0)) != -1; off += 32) {
        if (rw_clus(dp->cur_clus, 0, 0, (uint64)&de, off2, 32) != 32 || de.lne.order == END_OF_ENTRY) {
            return -1;
        }
        if (de.lne.order == EMPTY_ENTRY) {
            cnt++;
            continue;
        } else if (cnt) {
            *count = cnt;
            return 0;
        }
        if (de.lne.attr == ATTR_LONG_NAME) {
            int lcnt = de.lne.order & ~LAST_LONG_ENTRY;

            if (de.lne.order & LAST_LONG_ENTRY) {
                *count = lcnt + 1;
            }
        }
        // plus the s-n-e;
    }
}
```



```

        count = 0;
    }
    read_entry_name(ep->filename + (lcnt -
1) * CHAR_LONG_NAME, &de);
    } else {
        if (count) {
            *count = 1;
            read_entry_name(ep->filename, &d
e);
        }
        read_entry_info(ep, &de);
        return 1;
    }
}
return -1;
}

```

FAT32文件系统中，所有的文件条目顺序排序，正常情况下会存成这样的结构，这里只给文件名留了11个字节的大小，总体的大小则正好是32字节。

```

typedef struct short_name_entry {
    char        name[CHAR_SHORT_NAME];
    uint8       attr;
    uint8       _nt_res;
    uint8       _crt_time_tenth;
    uint16      _crt_time;
    uint16      _crt_date;
    uint16      _lst_acce_date;
    uint16      fst_clus_hi;
    uint16      _lst_wrt_time;
    uint16      _lst_wrt_date;
    uint16      fst_clus_lo;
    uint32      file_size;
} __attribute__((packed, aligned(4))) short_name_e
ntry_t;

```

如果长过11字节的名字，则会由另一个结构体 `long_name_entry_t` 经过连续的存放来保管名字碎片。

而 `enext` 则替我们把这个机制隔绝了，这样，在 `enext` 运行时，经过了多少个 `entry`，就会把 `count` 设为几。但FAT32中，用户删除某一文件，只会将 `entry` 的第一个字节设为 `EMPTY_ENTRY`，但 `enext` 不会在遇到空洞时直接跳过，而是正常将 `count++`，这是为了后续运行下一次 `enext` 之前，内核可以知道真正有用的下一个 `entry` 在什么位置。总的来说，`enext` 做了三件事：跳空洞、拼长名、提核心属性，最后将相关内容存放到传来的参数 `de` 中。

所以，现在我们就可以围绕这个 `enext` 得到我们的 `getdents` 了

```
uint64
sys_getdents(void)
{
    struct file *f;
    uint64 buf;
    int len;

    if (argfd(0, 0, &f) < 0 || argaddr(1, &buf) < 0
    || argint(2, &len) < 0)
        return -1;

    if (f->readable == 0 || !(f->ep->attribute & ATT
R_DIRECTORY))
        return -1;

    struct dirent de;
    int count = 0;
    int ret;
    int nread = 0;
    struct linux_dirent64 lde;

    elock(f->ep);
    while (1)
    {
        ret = enext(f->ep, &de, f->off, &count);
        if (ret == 0)
        { // 这个文件已被删除，直接跳过
```

```

        f->off += count * 32;
        continue;
    }
    if (ret == -1) // 后面不再有目录了
        break;

    int name_len = strlen(de.filename);
    int reclen = 19 + name_len + 1; // 19 是 struct linux_dirent64不包含文件名大小的size
    reclen = (reclen + 7) & ~7; // align to 8

    if (nread + reclen > len)
    {
        break; // 再存进去一个linux_dirent64会超过buf的最大空间
    }

    f->off += count * 32;

    // 这下面的部分只是为了严谨起见，但完全可以不添加这些内容
    lde.d_ino = de.first_clus; // 用首簇号作为inode
    lde.d_off = f->off; // 如果还想getdents，下次从哪里开始读
    lde.d_reclen = reclen; // 包含真实文件名长度的该结构体大小
    lde.d_type = (de.attribute & ATTR_DIRECTORY) ? 4 : 8; // 目录文件则为4，普通文件为8

    // 这下面就是存储过程了
    // 先把不包含文件名的内容存一遍到buf
    if (copyout2(buf + nread, (char *)&lde, 19) < 0)
    {
        eunlock(f->ep);
        return -1;
    }
    // 再把文件名补充进去

```

```

        if (copyout2(buf + nread + 19, de.filename, name_len + 1) < 0)
        {
            eunlock(f->ep);
            return -1;
        }

        nread += reclen; // 更新现在读了多少字节
    }
    eunlock(f->ep);

    return nread;
}

```

- 这样，最后在输出中，我们会看到 `bin`，也就是从 `.` 开始向下解析出来的第一个 `dirent` 名字，而这里得到的504，则是linux对于8的对齐，这时，就无需再像FAT32那样32对齐了。

▼ mkdir

- 看一下要做什么

```

void test_mkdir(void){
    TEST_START(__func__);
    int rt, fd;

    rt = mkdir("test_mkdir", 0666);
    printf("mkdir ret: %d\n", rt);
    assert(rt != -1);
    fd = open("test_mkdir", O_RDONLY | O_DIRECTORY);
    if(fd > 0){
        printf("  mkdir success.\n");
        close(fd);
    }
    else printf("  mkdir error.\n");
    TEST_END(__func__);
}

```

这里会调用 `mkdirat` 而非 `mkdir`，它有更强的能力，可以实现：如果 `path` 是相对路径，则它是相对于 `dirfd` 目录而言的。如果 `path` 是相对路径，且 `dirfd` 的值为 `AT_FDCWD`，则它是相对于当前路径而言的。如果 `path` 是绝对路径，则 `dirfd` 被忽略。

如果我们看一下具体的调用情况，实际上只需要考虑传的 `dirfd` 为 `AT_FDCWD` 的情况即可。

```
int mkdir(const char *path, mode_t mode)
{
    return syscall(SYS_mkdirat, AT_FDCWD, path, mode);
}
```

- 所以我们可以基于原有 `mkdir` 的基础上修改，得到 `mkdirat`。

先看看 `mkdir`

```
uint64
sys_mkdir(void)
{
    char path[FAT32_MAX_PATH];
    struct dirent *ep;

    if (argstr(0, path, FAT32_MAX_PATH) < 0 || (ep = create(path, T_DIR, 0)) == 0)
    {
        return -1;
    }
    eunlock(ep);
    eput(ep);
    return 0;
}
```

直接利用 `path` 创建一个目录，这里的 `create` 逻辑就是很简单地找到 `path` 对应的 `parent` 节点的 `dirent`，然后在这个 `dirent` 下从物理磁盘上 `ealloc` 一个新的 `entry`，但是完全没有考虑相对路径的事。所以我们首先修改参数解析逻辑：

```
uint64
sys_mkdirat(void)
```

```

{
    char path[FAT32_MAX_PATH];
    struct dirent *ep;
    int flags;

    if (argstr(1, path, FAT32_MAX_PATH) < 0 || argint(
2, &flags) < 0 || (ep = create(path, T_DIR, flags))
== NULL)
    {
        return -1;
    }
    eunlock(ep); // 解开睡眠锁，允许其他进程访问它
    eput(ep);
    return 0;
}

```

之后，通过相对路径解析绝对路径的事情，我们在 `openat` 测试点其实已经实现过一次了，所以接下来的逻辑也都差不多，直接实现就好了。

```

uint64
sys_mkdirat(void)
{
    char path[FAT32_MAX_PATH];
    struct dirent *ep;

    // 接下来将path转换为绝对路径，方法和sys_open里处理相对路
    径的方法一样
    int dirfd, mode;

    if (argint(0, &dirfd) < 0 || argstr(1, path, FAT32_
MAX_PATH) < 0 || argint(2, &mode) < 0)
    {
        return -1;
    }

    if (path[0] == '/' || dirfd == AT_FDCWD)
    { // 绝对路径或者没有指定 dirfd (AT_FDCWD)，直接使用 pat
    h 进行查找

```

```

    ;
}
else
{
    struct file *dirf;
    if (argfd(0, 0, &dirf) < 0 || !(dirf->ep->attribute & ATTR_DIRECTORY))
    {
        return -1;
    }

    struct dirent *curr = dirf->ep;
    char fullpath[FAT32_MAX_PATH];
    char *p_out = fullpath + FAT32_MAX_PATH - 1;
    *p_out = '\0';

    int path_len = strlen(path);
    if (path_len >= FAT32_MAX_PATH)
        return -1;
    p_out -= path_len;
    memmove(p_out, path, path_len);

    while (curr != NULL && curr->parent != curr && curr->parent != NULL)
    {
        int len = strlen(curr->filename);
        if (p_out - fullpath < len + 1)
        {
            return -1;
        }
        p_out--;
        *p_out = '/';
        p_out -= len;
        memmove(p_out, curr->filename, len);
        curr = curr->parent;
    }

    if (p_out > fullpath)

```

```

    {
        p_out--;
        *p_out = '/';
    }
    safestrcpy(path, p_out, FAT32_MAX_PATH);
}

if ((ep = create(path, T_DIR, mode)) == NULL)
{
    return -1;
}

eunlock(ep); // 解开睡眠锁，允许其他进程访问它
eput(ep);
return 0;
}

```

▼ chdir

- 实现完 `mkdir` 以后添加这个测试样例即可通过。

▼ unlink

- 先看一下要做什么

```

int test_unlink()
{
    TEST_START(__func__);

    char *fname = "./test_unlink";
    int fd, ret;

    fd = open(fname, O_CREATE | O_WRONLY);
    assert(fd > 0);
    close(fd);

    // unlink test
    ret = unlink(fname);
    assert(ret == 0);
}

```



```

    fd = open(fname, O_RDONLY);
    if(fd < 0){
        printf("  unlink success!\n");
    }else{
        printf("  unlink error!\n");
        close(fd);
    }
    // It's Ok if you don't delete the inode and data
    blocks.

    TEST_END(__func__);
}

```

可以看到 `unlink` 的作用实际上与删除文件的作用类似，但需要进行链接的删除，如果全部链接都消失，这个文件就被删除了。之后，我们发现它最终调用的是 `unlinkat`，只不过在测试样例中，依然使用 `AT_FDCWD` 忽略了其他情况。所以我们还是需要解析相对路径。

- 在传统的 xv6（基于 Inode 和 EXT2 思想）中，`unlink` 只是把文件的“硬链接数（link count）”减 1，只有当链接数降为 0 且没有进程打开它时，才会真正删除数据。**但是，目前所处的 FAT32 文件系统的世界里，根本没有“硬链接”这个概念！**在 FAT32 中，`unlink` 就是极其直接的“物理抹杀”。它的核心动作只有两个：
 - **立墓碑**：把父目录中记录这个文件的条目（Directory Entry）的第一个字节改成 `0xE5`。
 - **扬骨灰**：顺着 FAT 表，把这个文件占用的所有数据簇（Clusters）全部标记为空闲。
- 所以全部的流程大概是：提取参数，但这里我们只需要处理相对当前路径这一种情况；获取到路径对应的 `dirent`，检查该 `dirent` 是否为一个非空的目录；不是则直接删除，之前我们也说过，事实上 FAT32 删除文件的办法就是把 `entry` 的头一个字节设置为 `EMPTY_ENTRY`。而在 `fat.c` 中，有一些封装好的函数可以帮助。
- 事实上，在这里的 `unlink`，其逻辑可以完全套用 `sys_remove` 的逻辑。

```

uint64
sys_unlink(void)
{

```

```

char path[FAT32_MAX_PATH];
struct dirent *ep;
int len;

if ((len = argstr(1, path, FAT32_MAX_PATH)) <= 0)
    return -1;

char *s = path + len - 1;
while (s >= path && *s == '/')
{
    s--;
}
// 这里拒绝删除 "." 和 "..", 否则会导致文件系统混乱 (虽然
// FAT32 本身就没有这两个目录项, 但为了兼容性和安全性, 我们也不允
// 许用户创建它们)
if (s >= path && *s == '.' && (s == path || *--s ==
'/'))
{
    return -1;
}

// 获取文件对应的 dirent
// 注意: 和前面mkdirat的情况一样, 这里简单的 ename(path)
// 在 dirfd == AT_FDCWD 时
// 刚好能够通过 myproc()->cwd 或者绝对路径来正确解析并帮我们
// 通过这个测试点。
if ((ep = ename(path)) == NULL)
{
    return -1;
}

elock(ep);
// 如果是目录, 而且目录非空, 不能删除 (unlink主要针对文件,
// 但有些文件系统统一处理)
if ((ep->attribute & ATTR_DIRECTORY) && !isdirempty
(ep))
{
    eunlock(ep);
}

```

```

    eput(ep);
    return -1;
}

// 按照 FAT32 原理，将项目标记为0xE5（实际上eremove实现了
// 这一步）
    elock(ep->parent);
    eremove(ep);
    eunlock(ep->parent);

    eunlock(ep);
    eput(ep);

    return 0; // unlink 成功返回0
}

```

▼ mount/umount

- 检查要做的事情之后，我们发现其实只需 `return 0` 就可以完全通过了。

lab8

要求：在原先os-part4仓库中完成信号量相关调用 `test_ipc_producer_consumer`
`test_ipc_philosopher`

▼ 前置操作

- 运行这个指令，切换到part8的分支上

```
git checkout part8-sem
```

- 进入这个仓库下，如果直接运行 `make run_test`，会提示缺少系统调用，在我们注册好四个调用后，用 `return 0` 占位，这个时候再运行似乎是可以完美通过两个测试点的。

```
extern uint64 sys_sem_p(void);
extern uint64 sys_sem_v(void);
extern uint64 sys_sem_create(void);
extern uint64 sys_sem_destroy(void);
```

- 不过参考助教提供的主要步骤，我们可以只在这一个分支下完成我们的所有任务，也就是说实现了上述这四个系统调用后，两个样例均可直接通过。其中 `create` 和 `destroy` 分别管理着信号量的生命周期。

▼ sys_sem_create

- 在 xv6-riscv 操作系统中，为协调多个进程对共享资源的并发访问，防止出现数据竞争和不一致的状态，其内核已经提供了**自旋锁**。自旋锁为**忙等待**的方式，但阻塞cpu不利于单核系统，因此需要我们提供 `sem` 去完成更灵活的操作。

这里的 `sem`，应以自旋锁或睡眠锁为基础，添加计数功能。

- ▼ 我们看一下xv6自己的自旋锁是什么情况。

```
// Mutual exclusion lock.
struct spinlock {
    uint locked;           // Is the lock held?

    // For debugging:
    char *name;            // Name of lock.
};
```

```

    struct cpu *cpu;    // The cpu holding the lock.
};

// Initialize a spinlock
void initlock(struct spinlock*, char*);

// Acquire the spinlock
// Must be used with release()
void acquire(struct spinlock*);

// Release the spinlock
// Must be used with acquire()
void release(struct spinlock*);

// Check whether this cpu is holding the lock
// Interrupts must be off
int holding(struct spinlock*);

```

实际上，这里的自旋锁非常简单，它只支持互斥的操作，因此其结构中只包含了一个 `locked` 来判断是否处于等待阶段，在这个逻辑下，它只需要由 `acquire` 来控制加锁，并阻塞其他进程，`release` 来控制解锁。

```

// Acquire the lock.
// Loops (spins) until the lock is acquired.
void
acquire(struct spinlock *lk)
{
    push_off(); // disable interrupts to avoid deadloc
    k.
    if(holding(lk))
        panic("acquire");

    // On RISC-V, sync_lock_test_and_set turns into an
    atomic swap:
    //   a5 = 1
    //   s1 = &lk->locked
    //   amoswap.w.aq a5, a5, (s1)
    //  你可以看到下面这一条，只要lk不是解锁状态，就死循环等待

```

```

while(__sync_lock_test_and_set(&lk->locked, 1) !=
0)
    ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen strictly after the lock is acquired.
    // On RISC-V, this emits a fence instruction.
    __sync_synchronize();

    // Record info about lock acquisition for holding() and debugging.
    lk->cpu = mycpu();
}

```

- 我们可以依托现有的自旋锁来完成我们信号量的构造，事实上，不同于自旋锁，我们的信号量 `sem` 完全由计数器 `value` 这个判断指标来决定是否阻塞该进程。
 - `NSEM` 表示这个系统下最多可以同时支持的不同信号数量。

```

#define NSEM 128

struct sem
{
    int value;           // 信号量的值
    struct spinlock lock; // 保护信号量的自旋锁
    int valid;           // 信号量是否正在被使用
};

```

```

struct
{
    struct sem sems[NSEM]; // 信号量数组
    struct spinlock lock;   // 保护信号量数组的自旋锁
} sem_pool;

```

- 参考之前lab6有关交换区的结构体定义流程，我们也需要一个全局初始化的函数 `void seminit()`，并将其放在 `main.c` 中即可。

```
void seminit()
{
    initlock(&sem_pool.lock, "sem_pool_lock");
    int i;
    for (i = 0; i < NSEM; i++)
    {
        initlock(&sem_pool.sems[i].lock, "semaphore");
        sem_pool.sems[i].value = 0;
        sem_pool.sems[i].valid = 0;
    }
}
```

- 参考测试样例中 `sem_create` 的用法，我们知道它会向内核传递一个参数，用于初始化信号量的计数器。考虑到 `sem_p` 和 `sem_v` 中传递的参数为一个整数，并且在测试样例中，锁的数据类型也是一个int，我们知道这里的create应该起到了一个分配 `sem_pool` 中信号量的作用，需要返回一个索引值（或者说信号）。

```
int sem_create(int initial_value)
{
    acquire(&sem_pool.lock);
    for (int i = 0; i < NSEM; i++)
    {
        if (sem_pool.sems[i].valid == 0)
        {
            sem_pool.sems[i].valid = 1;
            sem_pool.sems[i].value = initial_value;
            release(&sem_pool.lock);
            return i;
        }
    }
    release(&sem_pool.lock);
    return -1; // No available semaphore
}
```

- 在 `sysproc.c` 中，包装好这个函数即可。

```
uint64
sys_sem_create(void)
{
    int initial_value;
    if (argint(0, &initial_value) < 0)
    {
        return -1;
    }
    return sem_create(initial_value);
}
```

▼ sys_sem_destroy

- 如果要回收一个信号量，我们需要检查是否有其他进程在等待这个信号量而睡眠，假设我们什么都不做直接回收，那么等待中的进程切换回来执行下一次等待时，就会试图获取一个完全不存在的內容。
- 因此我们应该在回收信号量前将所有进程都唤醒。在 `proc.c` 中，我们找到 `wakeup` 的定义，它可以将所有 `p->chan` 的睡眠进程全部唤醒。

```
void
wakeup(void *chan)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == SLEEPING && p->chan == chan) {
            p->state = RUNNABLE;
        }
        release(&p->lock);
    }
}
```

不过这里的 `chan` 是什么？在系统中，它是一个没有明确类型的指针变量，我们将其理解为 `struct proc` 下的一个成员变量，用于记录该进程 `SLEEP` 状态的来源即可。


```

void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    if(lk != &p->lock){ //DOC: sleeplock0
        acquire(&p->lock); //DOC: sleeplock1
        release(lk);
    }

    // Go to sleep.
    p->chan = chan;    // 在这一步，sleep将该进程的chan设置
    // 为传入的第一个参数。
    p->state = SLEEPING;

    sched();

    // Tidy up.
    p->chan = 0;

    // Reacquire original lock.
    if(lk != &p->lock){
        release(&p->lock);
        acquire(lk);
    }
}

```

- 我们在之后的信号量pv操作中，会把信号量作为睡眠来源传入 `sleep`，所以我们也只需要对 `wakeup` 传入同样参数即可。

```

int sem_destroy(int sem_id)
{
    if (sem_id < 0 || sem_id >= NSEM)
    {
        return -1; // Invalid semaphore ID
    }
    acquire(&sem_pool.lock);
    struct sem *s = &sem_pool.sems[sem_id];
}

```

```

    acquire(&s->lock);
    if (s->valid == 0)
    {
        release(&s->lock);
        release(&sem_pool.lock);
        return -1; // Semaphore not in use
    }
    sem_pool.sems[sem_id].valid = 0;
    wakeup(s); // Wake up any waiting processes
    release(&s->lock);
    release(&sem_pool.lock);
    return 0; // Success
}

```

- 在 `sysproc.c` 中同样添加一个包装即可。

▼ sys_sem_p

- 接下来是P操作，它在课程中用于减少信号量的计数，当信号量计数归零时，我们不使用自旋锁的acquire机制忙等待，要主动调用 `sleep` 让出cpu。不用 `yield` 的原因是我们配套使用 `sleep` / `wakeup` 来管理信号量。

```

int sem_p(int sem_id)
{
    if (sem_id < 0 || sem_id >= NSEM)
    {
        return -1; // Invalid semaphore ID
    }
    struct sem *s = &sem_pool.sems[sem_id];
    acquire(&s->lock);
    while (s->value <= 0)
    {
        sleep(s, &s->lock); // Sleep until the semaphore
        is available
    }
    s->value--;
    release(&s->lock);
    return 0;
}

```

- 同样在 `sysproc.c` 中添加包装。

▼ `sys_sem_v`

- 这个调用更加简单，别忘了最后在 `sysproc.c` 中添加包装。

```
int sem_v(int sem_id)
{
    if (sem_id < 0 || sem_id >= NSEM)
    {
        return -1; // Invalid semaphore ID
    }
    struct sem *s = &sem_pool.sems[sem_id];
    acquire(&s->lock);
    s->value++;
    // 这一步一定要有!
    wakeup(s);

    release(&s->lock);
    return 0;
}
```

为什么还要添加 `wakeup(s)` 这一步？如果执行了 `s->value++`，下一次切换到另一个阻塞在 `s` 的进程时，`while` 检查 `s->value` 不就不会持续 `sleep` 了吗？

其实并非，在 `sleep` 之后，进程的状态将永远被设置为 `SLEEP`，不能被调度，因此其它等待该信号量的进程永远无法被唤醒。